Artificial Intelligence Review

https://doi.org/10.1007/s10462-026-11640-4

Article in Press

Reinforcement Learning and Model-based Planning in Practice: A Survey of Algorithmic Rationale and Domain Applications

Received: 3 February 2026

Accepted: 3 July 2026

Published online: 14 August 2026

Cite this article as: Ghasemi M., Moosavi A.H. & Ebrahimi D. Reinforcement Learning and Model-based Planning in Practice: A Survey of Algorithmic Rationale and Domain Applications. *Artif Intell Rev* (2026). https://doi.org/10.1007/s10462-026-11640-4

Majid Ghasemi, Amir Hossein Moosavi & Dariush Ebrahimi

We are providing an unedited version of this manuscript to give early access to its findings. Before final publication, the manuscript will undergo further editing. Please note there may be errors present which affect the content, and all legal disclaimers apply.

If this paper is publishing under a Transparent Peer Review model then Peer Review reports will publish with the final article.

ARTICLE IN PRESS

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

ARTICLE IN PRESS

Reinforcement Learning and Model-based Planning in Practice: A Survey of Algorithmic Rationale and Domain Applications

Majid Ghasemi[1†], Amir Hossein Moosavi[2†], Dariush Ebrahimi[2*†]

[1*]Electrical and Computer Engineering, University of Waterloo, Waterloo, Ontario, Canada.

[2]Department of Computer Science, Wilfrid Laurier University, Waterloo, Ontario, Canada.

*Corresponding author(s). E-mail(s): debrahimi@wlu.ca;
Contributing authors: majid.ghasemi@uwaterloo.ca; s.amirhosein.mn@gmail.com;
†These authors contributed equally to this work.

ARTICLE IN PRESS

Abstract

Reinforcement Learning (RL) is a foundational framework in Artificial Intelligence (AI) that enables agents to acquire optimal decision-making strategies through interactions with their environments. Building on principles of trial-and-error learning, RL adapts dynamically by leveraging feedback in the form of rewards or penalties. This paper provides a comprehensive survey of RL, its integration with Deep Learning into Deep Reinforcement Learning (DRL), and the emerging field of model-based planning. It systematically examines a wide range of algorithms, from classical tabular methods to state-of-the-art DRL techniques, while evaluating their scalability, sample efficiency, and applicability in various domains. The survey offers an in-depth analysis of the strengths and limitations of these approaches, along with practical guidance for selecting and implementing RL algorithms. Key challenges, such as convergence, stability, and the exploration-exploitation trade-off, are thoroughly addressed to tackle real-world constraints. Additionally, this work explores the transformative impact of RL across diverse fields, including robotics, autonomous systems, intelligent transportation, energy management, and financial modeling. By bridging theoretical advancements and practical applications, this survey aims to provide researchers and practitioners with a comprehensive understanding of RL's capabilities for solving complex and dynamic problems in AI and beyond.

Keywords: Reinforcement Learning, Deep Reinforcement Learning, Model-free, Model-based, Actor-Critic, Q-learning, Deep Q-Network (DQN), Twin Delayed Deep Deterministic Policy Gradient (TD3), Proximal Policy Optimization (PPO), Trust Region Policy Optimization (TRPO)

1 Introduction

Reinforcement Learning (RL) is a branch of Artificial Intelligence (AI) in which an agent learns decision-making strategies through interactions with an environment, aiming to maximize cumulative rewards over time [1]. Since the 1950s, RL has undergone significant advancements, beginning with Richard Bellman's contributions to Dynamic Programming (DP), which laid the theoretical foundations for modern approaches [2, 3]. The field expanded with the introduction of more advanced methods, such as Temporal-Difference (TD) learning [4, 5], and the development of strategies addressing the exploration-exploitation dilemma [6, 7]. The integration of RL with Deep Learning (DL) has further accelerated its progress, leading to the emergence of Deep Reinforcement Learning

ARTICLE IN PRESS

(DRL), which enables researchers to address increasingly complex problems [8, 9]. RL has demonstrated remarkable effectiveness in sequential decision-making tasks across various domains, including game playing [10–13], robotics [14–17], and autonomous systems, particularly within Intelligent Transportation Systems (ITS) [18–22].

This survey explores the practical applications of various RL approaches across multiple domains, including but not limited to robotics [14, 23, 24], optimization [25, 26], energy efficiency and power management [27–29], networks [30–32], dynamic and partially observable environments [33–36], video games [37], real-time systems and hardware implementations [38, 39], financial portfolio management [40], ITS [18, 41, 42], signal processing [43], benchmark tasks [44], data management and processing [45, 46], as well as multi-agent and cloud-based systems [47–50]. Furthermore, this survey provides an in-depth discussion of RL algorithms, covering both traditional tabular methods and state-of-the-art techniques.

1.1 Related Work

A rich body of literature has explored various facets of Reinforcement Learning (RL), ranging from its historical foundations to its modern intersections with deep neural networks. Foundational studies, such as [6], offer a comprehensive computer science perspective on RL's early development and core principles. Building on this, several works dive into Deep Reinforcement Learning (DRL) [51–54], providing detailed analyses of how deep learning architectures have propelled RL's theoretical progress and practical capabilities.

Beyond general DRL, existing surveys often focus on specific algorithmic paradigms. Model-based RL, for instance, is widely recognized for its ability to improve sample efficiency by simulating trial-and-error learning within predictive models, thereby minimizing the need for costly real-world interactions [55–57]. Conversely, model-free approaches and their domain-specific applications have also been thoroughly reviewed; for example, [58] evaluates model-free RL in financial portfolio management, while [59] examines hybrid methods that combine model-free RL with imitation learning from expert demonstrations to enhance overall learning efficiency.

More recently, the scope of RL surveys has expanded to encompass highly specialized and cutting-edge domains. For example, [60] provides an exhaustive review of RL applications within Mobile Edge Computing (MEC) networks, detailing strategies for offloading, caching, and communication. In tandem with these networking advancements, a highly relevant trend is the integration of Large Language Models (LLMs) as advanced decision-making agents. As the field evolves, the intersection of RL and LLMs has become increasingly prominent, particularly through alignment techniques like Reinforcement Learning from Human Feedback (RLHF). Capturing this paradigm shift, [61] offers a comprehensive survey on decision-making LLMs tailored for wireless communication networks. This work illustrates how natural language processing and RL principles are jointly leveraged to optimize strategic reasoning and protocol design, providing a complete landscape of contemporary decision-making frameworks.

Despite this extensive literature, a critical gap remains. To the best of our knowledge, no existing work has systematically analyzed the strengths, weaknesses, and practical nuances of various RL algorithms across a wide spectrum of applied research domains. Selecting an appropriate RL algorithm is a multifaceted challenge dictated by problem characteristics, state and action space dimensions, environmental stochasticity, and data availability. Furthermore, practical considerations such as implementation feasibility, ease of debugging, convergence guarantees, and stability play pivotal roles in algorithmic selection.

This survey is motivated by the need to bridge this gap. By providing a comprehensive evaluation of multiple studies across diverse domains, this paper aims to help researchers make informed, practical algorithmic choices. Comparing target problems with existing, systematically evaluated research not only demystifies the selection process but also conserves valuable time and computational resources by reducing reliance on costly trial-and-error approaches.

The remainder of this paper is structured as follows: In Section 2, we present a general overview of RL before introducing various algorithms. Subsequently, we examine different RL algorithms, discussing relevant studies while analyzing their strengths and limitations. These algorithms are broadly classified into three main categories: value-based methods, policy-based methods, and actor-critic methods. Section 3 begins with an exploration of value-based methods, which are categorized into four key components: DP, tabular model-free methods, approximation model-free methods, and tabular model-based methods. Section 4 then covers policy-based methods, followed by Section 5,

ARTICLE IN PRESS

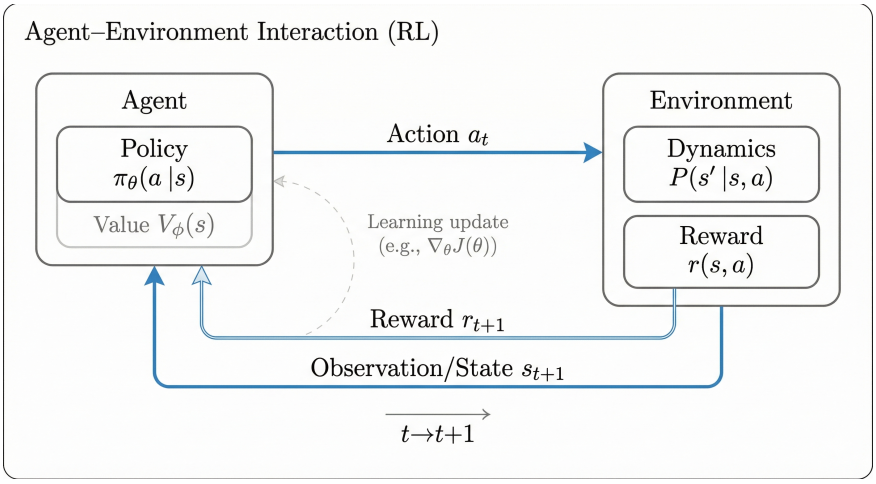


Figure 1: Agent-environment interaction in RL.

which provides an in-depth discussion on actor-critic methods. Section 6 examines recent trends in RL and DRL, including Reinforcement Learning from Human Feedback (RLHF). In Section 8, we summarize the paper and outline its scope. Finally, Section 9 synthesizes key findings and discusses future research directions.

2 General Overview of RL

This section provides a general overview of RL, assuming readers possess a basic understanding of the subject.

The RL framework is characterized by several key components that define the learning process. At its core, RL captures the essential elements of real-world problems encountered by an agent that interacts with its environment to achieve a specific objective. Such an agent must be able to perceive the environment's state to some extent and take actions that influence it [1]. In this context, an action represents a decision the agent must make, while a state encompasses the factors the agent considers in making that decision (Figure 1). In addition to the agent and environment, an RL system consists of four primary sub-components: a policy, a reward signal, a value function, and, optionally, a model of the environment. The reward signal dictates the agent's behavior within the environment by providing a scalar signal at each time step, guiding the agent to maximize cumulative rewards. A policy determines the agent's behavior at any given time, effectively mapping states to actions. The value function represents the agent's estimation of the long-term reward associated with visiting a particular state (or state-action pair) and continuing to follow its policy. Lastly, the model of the environment predicts the environment's behavior or can be used more broadly to infer its dynamics [1, 62, 63].

RL methodologies can be broadly classified into two categories: model-free and model-based approaches. Model-free methods learn directly from interactions with the environment without requiring prior knowledge of its dynamics. In contrast, model-based methods construct a model of the environment's dynamics, leveraging it to optimize and refine policies [3, 6]. Each of these categories presents distinct advantages and limitations, which are discussed in detail later in Sections 3.1 and 3.2.

Next, we examine two fundamental components of RL. First, we discuss the Markov Decision Process (MDP), which serves as the foundational framework for structuring the learning environment and guiding the agent's decision-making process. Following this, we address the exploration-exploitation dilemma, a critical aspect of RL that manages the trade-off between acquiring new information and maximizing rewards [1].

2.1 Markov Decision Process (MDP)

Before detailing the components of an MDP, it is important to clarify the notational conventions used throughout this survey. Following standard RL literature, we use uppercase letters (e.g., S_t, A_t, R_t) to denote random variables representing the state, action, and reward at a specific time step t.

ARTICLE IN PRESS

Conversely, we use their lowercase counterparts (e.g., s, a, r) to represent specific values, elements of a set, or actual realizations of these random variables during an episode.

MDP is defined by a tuple (S, A, P, R, γ), where: The MDP is a sequential decision-making framework where costs and transition functions depend directly on the system's current state and actions. The primary objective of an MDP is to derive an optimal policy for the decision-maker.

MDP is defined by a tuple (S, A, P, R, γ), where:

- S is a finite set of states,
- A is a finite set of actions,
- $P : S \times A \times S \to [0, 1]$ is the state transition probability function, where $P(s'|s, a)$ denotes the probability of transitioning to state s' from state s by taking action a,
- $R : S \times A \to \mathbb{R}$ is the reward function, where $R(s, a)$ denotes the immediate reward received after taking action a in state s,
- $\gamma \in [0, 1]$ is the discount factor that determines the importance of future rewards.

The agent's behavior is governed by a policy $\pi : S \to A$, which maps states to actions. The objective is to determine an optimal policy π^* that maximizes the expected cumulative reward, commonly referred to as the return (G_t), defined as:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R(s_{t+k}, a_{t+k}) \tag{1}$$

A key component in solving an MDP is the use of value functions, which estimate the expected return. The state-value function $V_\pi(s)$ under policy π represents the expected return when starting from state s and subsequently following policy π [64, 65]:

$$V_\pi(s) = \mathbb{E}_\pi[G_t|s_t = s] \tag{2}$$

Similarly, the action-value function $Q_\pi(s, a)$ is the expected return starting from state s, taking action a, and thereafter following policy π:

$$Q_\pi(s, a) = \mathbb{E}_\pi[G_t|s_t = s, a_t = a] \tag{3}$$

Equations 2 and 3 are collectively known as the Bellman equations. To determine the optimal policy π^*, RL algorithms iteratively refine value functions based on experience [5, 66]. In Q-learning, which is discussed later in Subsection 3.2.2, the update rule for the action-value function is:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\left(R(s_t, a_t) + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\right) \tag{4}$$

where α is the learning rate and γ is the discount factor. For a comprehensive discussion on MDPs and RL, readers are encouraged to refer to [1, 3, 6, 67, 68]

2.2 Exploration vs Exploitation

Exploration and exploitation represent a fundamental trade-off in RL. The purpose of exploration is to discover the effects of untried actions, while exploitation focuses on selecting actions that, based on the agent's current knowledge, yield high rewards at a given time step.

Exploration techniques are broadly classified into two categories: undirected [69] and directed [70, 71] exploration methods. Undirected approaches, such as semi-uniform (ϵ-greedy) exploration [72–74] and Boltzmann exploration [75–77], attempt to explore the entire state-action space by assigning a nonzero probability to all possible actions [1, 78]. In contrast, directed exploration methods, including the E^3 algorithm [79] and exploration bonuses [80], leverage past experience statistics to guide exploration more efficiently [79, 80].

Effectively balancing exploration and exploitation is critical in RL, and various strategies have been developed to address this challenge. For instance, [81] introduced a model-based RL approach that dynamically adjusts exploration and exploitation in changing environments. Using Bayesian inference with a forgetting mechanism, the method estimates state-transition probabilities and adapts the balance parameter based on action-outcome variations and environmental shifts. Additionally,

ARTICLE IN PRESS

[82] proposed Decoupled RL (DeRL), which trains separate policies for exploration and exploitation, compatible with both on-policy and off-policy RL algorithms. By decoupling these policies, DeRL enhances robustness and sample efficiency, particularly in sparse reward environments. More advanced techniques, such as entropy-regularized reward functions, were explored in [83], where the study examined the trade-off between exploration and exploitation in continuous-time RL. The optimal balance was achieved through a Gaussian distribution for the control policy, with exploitation represented by the mean and exploration by the variance. Furthermore, established methods like Upper Confidence Bound (UCB) [1] and Thompson Sampling [84] are commonly employed to manage this trade-off in simpler settings, such as multi-armed bandit problems [1, 7, 8].

2.3 Taxonomy and Orthogonal Dimensions of RL Algorithms

Classifying RL algorithms can often lead to conceptual overlap, as the landscape is defined by three primarily orthogonal dimensions. To ensure conceptual clarity, we establish the following classification axes before delving into specific algorithms:

- **Dimension 1: Optimization Target (Value-based, Policy-based, Actor-Critic).** This dimension dictates what the algorithm is trying to learn. Value-based methods learn an action-value function to implicitly derive a policy. Policy-based methods directly parameterize and optimize the policy itself. Actor-Critic methods hybridize the two, using a value function (critic) to guide the policy updates (actor).
- **Dimension 2: Environmental Dynamics (Model-free vs. Model-based).** Model-free algorithms learn solely through trial-and-error interactions with the environment. Model-based algorithms, conversely, learn or utilize a predictive model of the environment's transition dynamics and reward function to plan and simulate experiences.
- **Dimension 3: State/Action Representation (Tabular vs. Function Approximation).** Tabular methods store exact values for discrete state-action pairs in a matrix. Function approximation methods (often Deep RL) utilize parameterized functions, such as neural networks, to generalize across massive or continuous state-action spaces.

While these dimensions are orthogonal (e.g., an algorithm can be simultaneously Value-based, Model-free, and Approximate), structuring a sequential manuscript requires choosing a primary axis. For narrative clarity, the remainder of this survey is structured primarily along **Dimension 1 (Optimization Target)**. Section 3 explores Value-based methods, subdivided by their representation (Tabular vs. Approximation) and model reliance. Sections 4 and 5 subsequently explore Policy-based and Actor-Critic methods, which in modern literature predominantly utilize Function Approximation and Model-free frameworks. Table 1 illustrates how the algorithms discussed in this survey intersect across these three dimensions.

ARTICLE IN PRESS

Table 1: Orthogonal Taxonomy of Discussed RL Algorithms

Optimization Target	Representation	Model-Free	Model-Based
Value-Based	Tabular	MC, TD(0), SARSA, Q-Learning, Expected SARSA	Dynamic Programming (DP), MCTS, Prioritized Sweeping, Dyna-Q
	Approximation	DQN, Double DQN, Dueling DQN	(Modern variations of Dyna)
Policy-Based	Tabular	-	-
	Approximation	REINFORCE, TRPO, PPO	-
Actor-Critic	Tabular	-	-
	Approximation	A3C, A2C, DDPG, TD3	-

ARTICLE IN PRESS

In the following sections, we analyze various RL algorithms, discussing relevant studies while evaluating their advantages and limitations. These algorithms fall into three main categories: value-based methods, policy-based methods, and actor-critic methods. Section 3 begins with value-based methods, which are further classified into three core components: tabular model-free, tabular model-based, and approximation model-free methods. Section 4 then examines the sole method under policy-based approaches, namely, approximation model-free. Subsequently, Section 5 provides an in-depth discussion of actor-critic methods. Section 8 explains how to navigate this survey for maximum benefit. Finally, Section 9 synthesizes the key findings of this paper.

3 Value-based Methods

Value-based methods in RL focus on estimating the value of states or state-action pairs to facilitate decision-making. A fundamental aspect of this approach is learning a value function, which represents the expected long-term reward for a given state under a specific policy. These methods iteratively refine value estimates based on observed rewards and transitions. Notable examples include Q-learning and State-Action-Reward-State-Action (SARSA), which will be discussed later in this chapter. The objective of value-based methods is to determine an optimal policy by maximizing the value function, enabling the agent to select actions that yield the highest cumulative rewards.

Value-based methods are categorized into two main types: tabular model-based and tabular model-free methods. Model-based approaches rely on an explicit or learned model of the environment's dynamics, predicting state transitions and rewards in response to an agent's actions. In contrast, model-free methods do not depend on an environmental model but instead learn a policy or value function directly through interactions with the environment.

To provide a clear pedagogical progression, this section is organized systematically along two primary dimensions: the reliance on an environmental model, and the method of state-action representation. We begin in Section 3.1 with **Model-Based Algorithms: From Tabular Foundations to Search-Based Planning**, exploring how agents leverage known or learned environmental dynamics through Dynamic Programming and heuristic search. Section 3.2 transitions to **Tabular Model-free Algorithms**, detailing how agents learn purely from environmental interaction without a predictive model. Finally, Section 3.3 introduces **Approximation Model-free Algorithms**, demonstrating how deep neural networks overcome the curse of dimensionality inherent in tabular methods.

3.1 Model-Based Algorithms: From Tabular Foundations to Search-Based Planning

Tabular model-based algorithms are a class of RL techniques designed for problems with finite and discrete state and action spaces. These methods explicitly maintain and update a table or matrix that represents the environment's dynamics and reward structure. A defining characteristic of model-based algorithms is the construction of an environmental model to assist in decision-making. Key aspects of tabular model-based techniques include model representation, planning and policy evaluation, and value iteration [1]. In model representation, the environment's dynamics are captured in tabular form, where transition probabilities are expressed as $P(s'|s,a)$ and the reward function as $R(s,a)$. These elements specify the likelihood of transitioning to a new state s' and the expected reward obtained by taking action a in state s. Planning and policy evaluation involve iteratively updating the value function $V(s)$ using the Bellman equation until convergence. The algorithm considers all possible future states and corresponding actions to compute the optimal policy. Similarly, value iteration refines the value function $V(s)$ through iterative updates via the Bellman equation, assessing all potential future states and their associated actions to determine the optimal policy.

Tabular model-based algorithms are generally categorized into two groups: Dynamic Programming and Model-based planning.

3.1.1 Dynamic Programming (DP)

DP methods are foundational approaches for solving MDPs when the complete model of the environment is available. These methods operate iteratively, utilizing the Bellman equations to derive optimal policies (see Figure 2). Two primary DP-based methods are **policy iteration** and **value iteration**.

ARTICLE IN PRESS

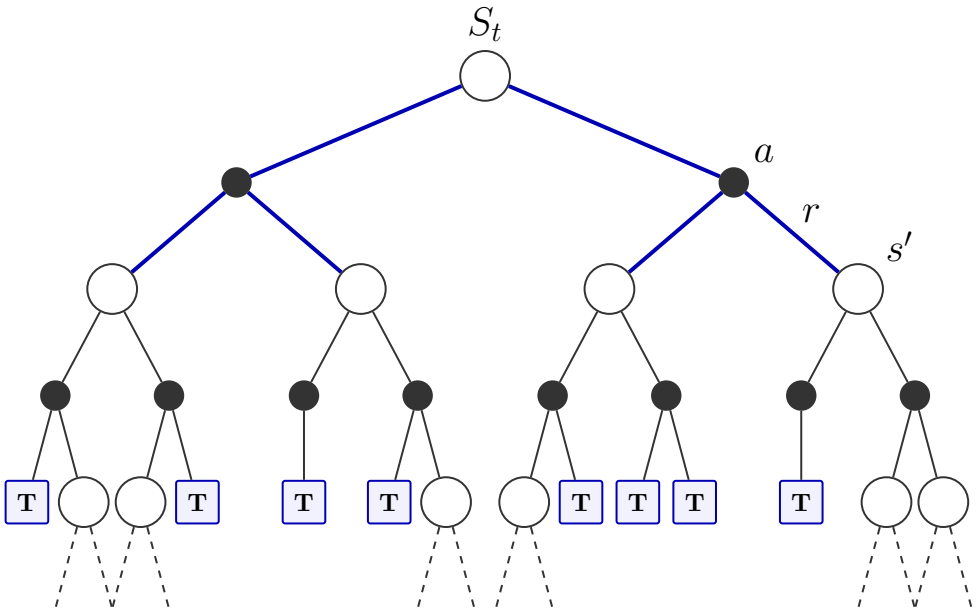


Figure 2: An illustration of the Dynamic Programming (DP) backup approach. The update sweeps across all possible actions and their subsequent next states, representing a shallow but maximally wide backup.

Algorithm 1 Policy Iteration

1: Initialize policy π arbitrarily
2: **repeat**
3: Perform policy evaluation to update the value function V_π
4: Perform policy improvement to update the policy π
5: **until** policy π converges

ARTICLE IN PRESS

Policy Iteration

Policy iteration is an iterative approach that refines the policy until it converges to the optimal solution. It comprises two key steps: policy evaluation and policy improvement [1]. **Policy evaluation** calculates the value function $V_\pi(s)$ for a given policy π. This involves solving the Bellman expectation equation for the current policy:

$$V_\pi(s) = \sum_{a \in A} \pi(a|s) \sum_{s' \in S} P(s'|s,a)[R(s,a,s') + \gamma V_\pi(s')] \tag{5}$$

This step iteratively refines the value of each state under the current policy until convergence is achieved [3]. **Policy improvement**, on the other hand, improves the policy by making it greedy with respect to the current value function:

$$\pi'(s) = \arg\max_a \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V_\pi(s')] \tag{6}$$

In this step, the policy is updated by choosing actions that maximize the expected value based on the current value function [55]. Alg. 1 provides an overview of the implementation of policy iteration.

Value Iteration

Value iteration is a DP method that iteratively updates the value of each state to directly compute the optimal value function. It integrates policy evaluation and policy improvement into a single step. The process consists of two key steps: value update and policy extraction. **Value update** revises the value function for each state based on the Bellman optimality equation:

$$V(s) = \max_a \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V(s')] \tag{7}$$

ARTICLE IN PRESS

Algorithm 2 Value Iteration

```
Initialize the value function V(s) arbitrarily
repeat
    for each state s do
        Update V(s) using the Bellman optimality equation
    end for
until the value function V(s) converges
Extract the optimal policy π* from the converged value function
```

Table 2: Comparison of Policy Iteration and Value Iteration

Aspect	Policy Iteration	Value Iteration
Convergence	Typically fewer iterations needed	May require more iterations
Complexity per Iteration	More complex (requires policy evaluation)	Simpler (single update step)
Stability	More stable due to separate steps	Can be less stable
Ease of Implementation	More complex	Simpler
Computational Cost	Higher per iteration	Lower per iteration

This step iterates over all states, updating their values based on the maximum expected return across all possible actions. **Policy extraction** happens once the value function has converged; the optimal policy can be extracted by selecting actions that maximize the expected value:

$$\pi^*(s) = \arg\max_a \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V(s')] \tag{8}$$

Alg. 2 demonstrates the implementation of value iteration. Compared to policy iteration, value iteration is simpler to implement since it merges evaluation and improvement into a single process. Additionally, it is often more efficient in practice, as it eliminates the need for separate policy evaluation steps [85]. However, value iteration may require a large number of iterations to achieve convergence, particularly in problems with extensive state spaces. Moreover, due to its combined update steps, it can be less stable than policy iteration in certain cases [1]. A detailed comparison between policy iteration and value iteration is presented in Table 2.

3.1.2 Model-based Planning

Model-based planning in RL refers to the approach where the agent constructs and uses a model of the environment to plan its actions and make decisions. This model can either be learned from interactions with the environment or be predefined if the environment's dynamics are already known. Model-based methods can be more sample-efficient as they use learned models to simulate various scenarios, enabling the agent to gather insights without needing to interact directly with the environment every time. Model-based planning helps the agent effectively balance exploration (trying new actions) and exploitation (using known effective actions). By simulating different actions and their potential outcomes, the agent can make more informed decisions, optimizing its behavior. The learned model allows the agent to easily adapt to environmental changes. Through model updates and re-planning, the agent can dynamically respond to new situations, maintaining high performance even in changing environments [1, 55, 86].

There are three primary approaches in model-based planning: Monte Carlo Tree Search (MCTS), prioritized sweeping, and Dyna-Q. We begin by analyzing MCTS in the following paragraphs.

Monte Carlo Tree Search (MCTS)

MCTS is a powerful algorithm designed to tackle sequential decision-making problems under uncertainty. It has found wide application across various domains, particularly in-game AI, where it has achieved notable success. MCTS blends the precision of tree search with the generality of random sampling (Monte Carlo simulation). The core process of MCTS is conceptually simple, as illustrated

ARTICLE IN PRESS

in Figure 3. This technique has been crucial in the development of AI systems such as AlphaGo and AlphaZero, which have demonstrated superhuman performance in games like Go, chess, and shogi.

It is crucial to clarify that while MCTS is introduced here sequentially following tabular methods, it is a highly versatile, general search framework rather than a strictly tabular algorithm. As evidenced by landmark systems like AlphaGo and AlphaZero, MCTS seamlessly integrates with function approximation. In these advanced architectures, deep neural networks are employed to guide the tree selection phase (via policy networks) and to evaluate leaf nodes (via value networks), allowing MCTS to scale to environments with massive, high-dimensional state spaces. Consequently, MCTS transcends the tabular-approximate divide, serving as a powerful planning engine across both classical and deep reinforcement learning contexts.

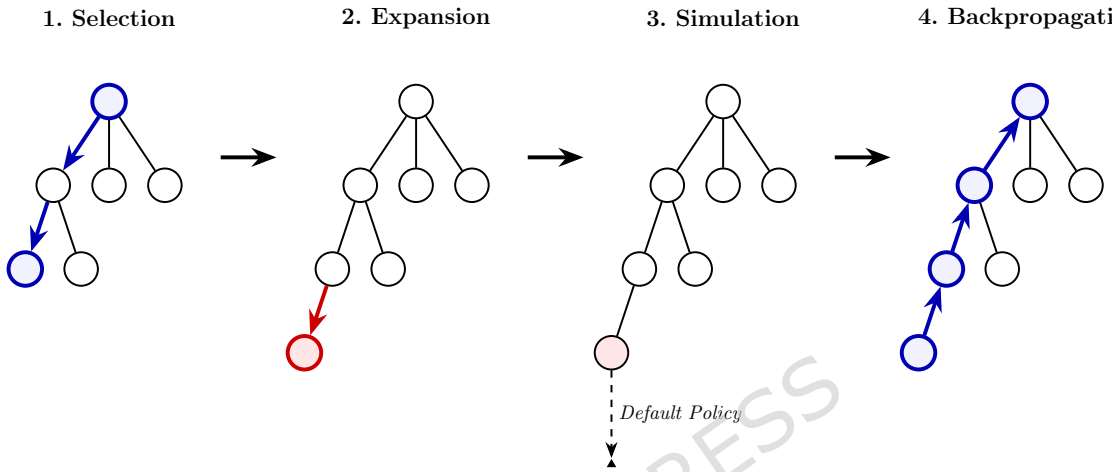


Figure 3: The four sequential phases of the Monte Carlo Tree Search (MCTS) algorithm.

MCTS operates by incrementally building a search tree in an asymmetric manner, as illustrated in Figure **??**. Each iteration of MCTS involves four primary steps [1]:

1. **Selection**: Starting from the root node, the algorithm recursively selects child nodes until it reaches a leaf node (lines 14-17). This process balances exploration and exploitation, typically using a UCB formula to prioritize moves with high estimated value and uncertainty.
2. **Expansion**: If the leaf node is not a terminal node, it is expanded by adding one or more child nodes (line 18). This step explores new, previously unconsidered moves or actions.
3. **Simulation**: From the expanded node, a simulation (or playout) is executed to the end of the game using a default policy, often random (lines 19-21). The simulation provides an estimated outcome that helps approximate the value of the moves from the expanded node.
4. **Backpropagation**: The result of the simulation is propagated back through the tree, updating the values of all nodes along the path (line 22). This refines the estimated value of moves based on the new data.

These steps are repeated until the computational budget (time or iterations) is exhausted. The move with the highest value at the root node is selected as the optimal move. Based on the overview in [87], Alg. 3 provides a comprehensive overview of MCTS.

Key Advantages of MCTS are domain independence (MCTS does not require domain-specific knowledge, only the rules of the game or problem.), scalability (It can handle very large state spaces by focusing computational effort on the most promising parts of the search tree.), and flexibility (It can be adapted to various types of problems, including those with stochastic elements) [1, 88, 89].

Study [90] presented the Multi-objective MCTS (MO-MCTS) algorithm, an extension of MCTS designed for multi-objective sequential decision-making. The key innovation lies in using the hypervolume indicator to replace the UCB criterion, enabling the algorithm to handle multi-dimensional rewards and discover several Pareto-optimal policies within a single tree. The MO-MCTS algorithm stood out for its ability to effectively manage multi-objective optimization by integrating the hypervolume indicator, which provided a comprehensive measure of the solution set's quality. This approach allowed the algorithm to balance exploration and exploitation in multi-dimensional spaces, making

ARTICLE IN PRESS

Algorithm 3 MCTS [87]

Input:
    PD: problem's dimension
    FG: fitness goal
    RL: roll-out length
Output:
    n*: the best solution found
    f: the maximum performance level reached
    t: the total number of roll-outs performed
Create root node n_r and add it to the tree
Make n_r the actual node n
$n^* \leftarrow$ null
$f \leftarrow 0$
$t \leftarrow 0$
while move_available(n) **do**
    while not is_leaf(n) **do**
        $n \leftarrow$ best_child(n)
    end while
    $n \leftarrow$ add_random_child_node(n)
    RL $\leftarrow$ rand(PD, n)
    $f, n' \leftarrow$ roll_out(n, RL)
    $t \leftarrow t + 1$
    back_propagate(n, f)
    if $f >$ FG **then**
        $n^* \leftarrow n'$
        break
    end if
    $n \leftarrow n_r$
end while
return n^*, f, t

ARTICLE IN PRESS

it more efficient in discovering Pareto-optimal solutions compared to traditional scalarized RL methods. The use of the hyper-volume indicator as an action selection criterion ensured that the algorithm could capture a diverse set of optimal policies, addressing the limitations of linear-scalarization methods that failed in non-convex regions of the Pareto front. The experimental validation on the Deep Sea Treasure (DST) problem and grid scheduling tasks demonstrated that MO-MCTS achieved superior performance and scalability, matching or surpassing state-of-the-art non-RL-based methods despite higher computational costs. However, the reliance on accurate computation of the hyper-volume indicator introduced significant computational overhead, particularly in high-dimensional objective spaces. The complexity of maintaining and updating the Pareto archive could also pose challenges, especially as the number of objectives increases. While the algorithm showed robust performance in deterministic settings, its scalability and efficiency in highly stochastic or dynamic environments remained to be fully explored. The need for domain-specific knowledge to define the reference point and other hyper-volume-related parameters could also limit the algorithm's generalizability. Additionally, the method's computational intensity might hinder its real-time applicability, requiring further optimization or approximation techniques to reduce processing time.

A novel approach to Neural Architecture Search (NAS) using MCTS was introduced in [91]. The authors proposed a method that captured the dependencies among layers by modeling the search space as a Monte-Carlo Tree, enhancing the exploration-exploitation balance and efficiently storing intermediate results for better future decisions. The method was validated through experiments on the NAS-Bench-Macro benchmark and the ImageNet dataset. The primary strength of this approach lies in its ability to incorporate dependencies between different layers during architecture search, which many existing NAS methods have overlooked. By utilizing MCTS, the proposed method effectively balanced exploration and exploitation, leading to more efficient architecture sampling. The use of a Monte-Carlo Tree allowed for the storage of intermediate results, significantly improving the search efficiency and reducing the need for redundant computations. The introduction of node

ARTICLE IN PRESS

communication and hierarchical node selection techniques further refined the evaluation of numerous nodes, ensuring more accurate and efficient search processes. Experimental results demonstrated that the proposed method achieved higher performance and search efficiency compared to state-of-the-art NAS methods, particularly on the NAS-Bench-Macro and ImageNet datasets. On the other hand, the reliance on the accurate modeling of the search space as a Monte-Carlo Tree introduced additional complexity, particularly in terms of computational overhead and memory requirements. The method's performance heavily depended on the proper tuning of hyperparameters, such as the temperature term and reduction ratio, which could be challenging to optimize for different datasets and tasks. While the experiments showed promising results, further validation in more diverse and complex real-world scenarios was necessary to fully assess the scalability and robustness of the approach. The additional computational cost associated with maintaining and updating the Monte-Carlo Tree and performing hierarchical node selection could impact the method's applicability in real-time applications or resource-constrained environments.

Study [49] introduced multi-agent MCTS (MAMCTS), a novel extension of MCTS tailored for cooperative multi-agent systems. The primary innovation was the integration of different evaluations, which significantly enhanced coordination strategies among agents. The performance of MAMCTS was demonstrated in a multi-agent path-planning domain called the multi-agent Grid-world (MAG), showcasing substantial improvements over traditional reward evaluation methods. The MAMCTS approach effectively leveraged different evaluations to prioritize actions that contributed positively to the overall system, leading to significant improvements in learning efficiency and coordination among agents. By combining MCTS with different rewards, the algorithm balanced exploration and exploitation, ensuring that agents could efficiently navigate the search space to find optimal policies. The experimental results in the 100x100 MAG environment demonstrated that MAMCTS outperformed both local and global reward methods, achieving up to 31.4% and 88.9% better performance, respectively. This superior performance was consistent across various agent and goal configurations, highlighting the scalability and robustness of the approach. The use of a structured search process and prioritized updates ensured that the algorithm could handle large-scale multi-agent environments effectively. The reliance on accurate computation and maintenance of different evaluations introduced additional computational overhead, particularly in environments with a large number of agents and goals. The method's performance was sensitive to the accuracy of these evaluations, which might be challenging to maintain in highly dynamic or unpredictable environments. While the simulations provided strong evidence of the method's efficacy, further validation in more diverse real-world scenarios was necessary to fully assess its scalability and practical utility. The complexity of managing multiple agents and ensuring synchronized updates could also pose challenges, particularly in real-time applications where computational resources are limited. Additionally, the paper focused primarily on cooperative settings, and the applicability of MAMCTS to competitive or adversarial multi-agent environments remained an area for future exploration.

A MO-MCTS algorithm tailored for real-time games was developed in [92]. It focused on balancing multiple objectives simultaneously, leveraging the hyper-volume indicator to replace the traditional UCB criterion. The algorithm was tested against single-objective MCTS and Non-dominated Sorting Genetic Algorithm (NSGA)-II, showcasing superior performance in benchmarks like DST and the multi-objective Physical Traveling Salesman Problem (MO-PTSP). The MO-MCTS algorithm excelled in handling multi-dimensional reward structures, efficiently balancing exploration and exploitation. By incorporating the hyper-volume indicator, the algorithm could discover a diverse set of Pareto-optimal solutions, effectively addressing the limitations of linear-scalarization methods in non-convex regions of the Pareto front. The empirical results on DST and MO-PTSP benchmarks demonstrated the algorithm's ability to converge to optimal solutions quickly, outperforming both single-objective MCTS and NSGA-II in terms of exploration efficiency and solution quality. The use of a weighted sum and Euclidean distance mechanisms for action selection further enhanced the adaptability of MO-MCTS to various game scenarios, providing a robust framework for real-time decision-making. Nonetheless, the reliance on the hyper-volume indicator introduced significant computational overhead, which could limit the algorithm's scalability in high-dimensional objective spaces. The need for maintaining a Pareto archive and computing the hyper-volume for action selection added to the complexity, potentially impacting real-time performance. While the algorithm showed strong results in the tested benchmarks, its applicability to more complex and dynamic real-time games required further validation. The computational intensity of the approach might hinder its practicality in resource-constrained environments, necessitating the exploration of optimization techniques to reduce processing time without compromising performance. Additionally, the paper

ARTICLE IN PRESS

primarily focused on deterministic settings, leaving the performance in stochastic or highly variable environments less explored.

Researchers in [93] explored hybrid algorithms that integrated MCTS with mini-max search to leverage the strategic strengths of MCTS and the tactical precision of minimax. The authors proposed three hybrid approaches: employing minimax during the selection/expansion phase, the rollout phase, and the backpropagation phase of MCTS. These hybrids aimed to address the weaknesses of MCTS in tactical situations by incorporating shallow minimax searches within the MCTS framework. The hybrid algorithms presented in the paper offered a promising combination of MCTS's ability to handle large search spaces with mini-max's tactical accuracy. By integrating shallow mini-max searches, the hybrids could better navigate shallow traps that MCTS might overlook, leading to more robust decision-making in games with high tactical demands. The experimental results in games like Connect-4 and Breakthrough demonstrated that these hybrid approaches could outperform standard MCTS, particularly in environments where tactical precision was crucial. The use of mini-max in the selection/expansion phase and the backpropagation phase significantly improved the ability to avoid blunders and recognize winning strategies early, enhancing the overall efficiency and effectiveness of the search process. However, the inclusion of mini-max searches introduced additional computational overhead, which could slow down the overall search process, especially for deeper mini-max searches. The performance improvements were heavily dependent on the correct tuning of parameters, such as the depth of the mini-max search and the criteria for triggering these searches. While the hybrids showed improved performance in the tested games, their scalability and effectiveness in more complex and dynamic real-world scenarios remained to be fully validated. The reliance on game-specific characteristics, such as the presence of shallow traps, might limit the generalizability of the results. Further exploration was needed to assess the impact of these hybrids in a broader range of domains and under different conditions.

Authors in [94] introduced the Option-MCTS (O-MCTS) algorithm, which extended the MCTS by incorporating high-level action sequences, or "options," aimed at achieving specific subgoals. The proposed algorithm aimed to enhance general video game playing by utilizing higher-level planning, enabling it to perform well across a diverse set of games from the General Video Game AI competition. Additionally, the paper introduced Option Learning MCTS (OL-MCTS), which applied a progressive widening technique to focus exploration on the most promising options. The integration of options into MCTS was a significant advancement, allowing the algorithm to plan more efficiently by considering higher-level strategies. This higher abstraction level helped the algorithm deal with complex games that required achieving multiple subgoals. The use of options reduced the branching factor in the search tree, enabling deeper exploration within the same computational budget. The empirical results demonstrated that O-MCTS outperformed traditional MCTS in games requiring sequential subgoal achievements, such as collecting keys to open doors, showcasing its strength in strategic planning. The introduction of OL-MCTS further improved performance by learning which options were most effective, thus focusing the search on more promising parts of the game tree and improving efficiency. On the other hand, the reliance on predefined options and their proper tuning could be a limitation, as the performance of O-MCTS heavily depended on the quality and relevance of these options to the specific games being played. The initial computational overhead associated with constructing and managing a large set of options might impact the algorithm's performance, particularly in games with numerous sprites and complex dynamics. The progressive widening technique in OL-MCTS, while beneficial for focusing exploration, introduced additional complexity and overhead, potentially reducing real-time applicability. Further validation was needed to assess the scalability and robustness of these algorithms in a wider range of real-world game scenarios, where the diversity and unpredictability of game mechanics might present new challenges.

An extension of MCTS, incorporating heuristic evaluations through implicit mini-max backups, was investigated in [95]. The approach aimed to combine the strengths of MCTS and mini-max search to improve decision-making in strategic games by maintaining separate estimations of win rates and heuristic evaluations and using these to guide simulations. The integration of implicit minimax backups within MCTS significantly enhanced the quality of simulations by leveraging heuristic evaluations to inform decision-making. This hybrid approach addressed the limitations of pure MCTS in domains where tactical precision was crucial, effectively balancing strategic exploration with tactical accuracy. By maintaining separate values for win rates and heuristic evaluations, the algorithm could better navigate complex game states, leading to stronger play performance. The empirical results in games like Kalah, Breakthrough, and Lines of Action demonstrated substantial improvements

ARTICLE IN PRESS

over standard MCTS, validating the effectiveness of implicit mini-max backups in diverse strategic environments. The method also showed robust performance across different parameter settings, highlighting its adaptability and potential for broader application in game-playing AI. However, the reliance on accurate heuristic evaluations introduced complexity in environments where such evaluations were not readily available or were difficult to compute. The additional computational overhead associated with maintaining and updating separate value estimates might impact the algorithm's efficiency, particularly in real-time applications. While the approach showed significant improvements in specific games, further validation was necessary to assess its scalability and robustness in more complex and dynamic scenarios. The dependence on domain-specific knowledge for heuristic evaluations might limit the generalizability of the method to a wider range of applications. Additionally, the complexity of tuning the parameter that weighted the influence of heuristic evaluations could pose challenges in optimizing the algorithm for different environments.

Authors in [96] explored the application of MCTS to the game of Lines of Action (LoA), a two-person zero-sum game known for its tactical depth and moderate branching factor. The authors proposed several enhancements to standard MCTS to handle the tactical complexities of LoA, including game-theoretical value proving, domain-specific simulation strategies, and effective use of progressive bias. The key strength of this paper lies in its innovative enhancements to MCTS, enabling it to handle the tactical and progression properties of LoA effectively. By incorporating game-theoretical value proving, the algorithm could identify and propagate winning and losing positions more efficiently, reducing the computational burden of extensive simulations. The use of domain-specific simulation strategies significantly improved the quality of the simulations, leading to more accurate evaluations and better overall performance. The empirical results demonstrated that the enhanced MCTS variant outperformed the world's best $\alpha\beta$-based LoA program, marking a significant milestone for MCTS in handling highly tactical games. The detailed analysis and systematic approach to integrating domain knowledge into MCTS provided a robust framework for applying MCTS to other complex games. Despite its advantages, the approach introduced additional computational overhead due to the need for maintaining and updating multiple enhancements, such as the progressive bias and game-theoretical value proving. The reliance on domain-specific knowledge for simulation strategies and the progressive bias limited the generalizability of the method to other games without similar properties. While the algorithm performed well in the controlled environment of LoA, its scalability and robustness in more dynamic and less structured environments remained to be fully explored. The complexity of tuning various parameters, such as the progressive bias coefficient and the simulation cutoff threshold, could also pose challenges, particularly for practitioners without deep domain knowledge. Further validation in diverse real-world scenarios was necessary to assess the practical applicability and long-term benefits of the proposed enhancements.

Authors in [97] explored the application of MCTS to the popular collectible card game "Hearthstone: Heroes of Warcraft." Given the game's complexity, uncertainty, and hidden information, the authors proposed enriching MCTS with heuristic guidance and a database of decks to enhance performance. The approach was empirically validated against vanilla MCTS and state-of-the-art AI, showing significant performance gains. The integration of expert knowledge through heuristics and a deck database addressed two major challenges in Hearthstone: hidden information and a large search space. By incorporating a heuristic function into the selection and simulation steps, the algorithm could more effectively navigate the game's complex state space, leading to improved decision-making and performance. The use of a deck database allowed the MCTS algorithm to predict the opponent's cards more accurately, enhancing the quality of simulations and overall strategy. The empirical results demonstrated that the enhanced MCTS approach significantly outperformed vanilla MCTS and was competitive with existing AI players, showcasing its potential for complex, strategic games like Hearthstone. However, the reliance on pre-constructed heuristics and deck databases introduced additional complexity and potential limitations. The effectiveness of the heuristic function was highly dependent on its design and tuning, which might vary across different game scenarios and decks. Similarly, the deck database's accuracy and comprehensiveness were crucial for predicting opponent strategies, which might be challenging to maintain as new cards and strategies emerged. The additional computational overhead associated with managing and updating these enhancements could impact real-time performance, particularly in time-constrained gameplay environments. While the approach showed strong performance in controlled experiments, further validation in diverse and dynamic real-world scenarios was necessary to fully assess its robustness and adaptability.

Information Set MCTS (ISMCTS), an extension of MCTS designed for games with hidden information and uncertainty, was introduced in [98]. ISMCTS algorithms searched trees of information

ARTICLE IN PRESS

sets rather than game states, providing a more accurate representation of games with imperfect information. The approach was tested across three domains with different characteristics. The ISMCTS algorithms excelled in efficiently managing hidden information by constructing trees where nodes represented information sets instead of individual states. This method mitigated the strategy fusion problem seen in determinization approaches, where different states in an information set were treated independently. By unifying statistics about moves within a single tree, ISMCTS made better use of the computational budget, leading to more informed decision-making. The empirical results demonstrated that ISMCTS outperformed traditional determinization-based methods in games with significant hidden information and partially observable moves. For instance, in Lord of the Rings: The Confrontation and the Phantom, ISMCTS achieved superior performance by effectively leveraging the structure of the game and reducing the impact of strategy fusion. However, the ISMCTS approach introduced additional complexity in maintaining and updating information sets, which could lead to increased computational overhead. The scalability of the method in environments with extensive hidden information and large state spaces remained a challenge, as the branching factor in information set trees could become substantial. While ISMCTS showed promising results in the tested domains, further validation in more diverse and dynamic scenarios was necessary to fully assess its robustness and general applicability. The reliance on accurate modeling of information sets and the necessity for domain-specific adaptations could limit the ease of implementation and the algorithm's flexibility across different types of games.

Authors in [99] investigated the application of MCTS to Kriegspiel, a variant of chess characterized by hidden information and dynamic uncertainty. The authors explored three MCTS-based methods, incrementally refining the approach to handle the complexities of Kriegspiel. They compared these methods to a strong minimax-based Kriegspiel program, demonstrating the effectiveness of MCTS in this challenging environment. The authors' incremental refinement of MCTS methods for Kriegspiel effectively addressed the game's dynamic uncertainty and large state space. By leveraging a probabilistic model of the opponent's pieces and incorporating domain-specific heuristics, the refined MCTS algorithms significantly improved performance compared to the initial naive implementation. The experimental results showed that the final MCTS approach outperformed the minimax-based program, achieving better strategic planning and decision-making. The innovative use of a three-tiered game tree representation and opponent modeling techniques demonstrated the adaptability and robustness of MCTS in handling partial information games. This study provided valuable insights into the application of MCTS in environments with incomplete information, highlighting its potential for broader applications in similar domains. Having said that, the reliance on extensive probabilistic modeling and heuristic adjustments introduced additional complexity, which could be computationally intensive and challenging to maintain. The performance improvements were heavily dependent on the accuracy and relevance of the probabilistic models, which might vary across different game scenarios and opponents. While the approach showed strong performance in the controlled environment of Kriegspiel, its scalability and robustness in more diverse and dynamic real-world scenarios remained to be fully validated. The necessity for domain-specific knowledge to construct effective heuristics and models limited the generalizability of the method to other games with different characteristics. Further research was needed to explore the method's applicability and efficiency in a wider range of partial information games and real-world decision-making tasks.

Two extensions of MCTS tailored for Asymmetric Trees (MCTS-T) and environments with loops were introduced in [37]. The first algorithm, MCTS-T, addressed the challenges posed by asymmetric tree structures by incorporating tree uncertainty into the UCB formula, thus improving exploration. The second algorithm, MCTS-T+, further extended this approach to handle loops by detecting and appropriately managing repeated states within a single trace. The effectiveness of these algorithms was demonstrated through benchmarks on OpenAI Gym and Atari 2600 games. The key strength of MCTS-T lies in its ability to efficiently manage asymmetric trees by backing up the uncertainty related to the tree structure. This approach ensured that the algorithm could prioritize exploration in less-explored subtrees, significantly improving the learning efficiency in environments with uneven tree depths. MCTS-T+ further enhanced this capability by addressing loops, preventing redundant expansions of the same state, and thus saving computational resources. The empirical results on benchmark tasks demonstrated that both MCTS-T and MCTS-T+ consistently outperformed standard MCTS, particularly in environments with strong asymmetry and loops. These improvements highlighted the potential of the proposed methods to extend the applicability of MCTS to more complex domains, such as robotic control and single-player video games, where asymmetry and looping were common. However, the reliance on non-stochastic environments and full state observability

ARTICLE IN PRESS

limited the generalizability of MCTS-T and MCTS-T+. These assumptions might not hold in many real-world scenarios, such as partially observable or highly stochastic environments, reducing the algorithms' applicability. The computational overhead associated with maintaining and updating the tree uncertainty and detecting loops could be substantial, particularly in large and dynamic state spaces. While the proposed methods showed promising results in controlled experiments, further validation in diverse real-world applications was necessary to fully assess their scalability and robustness. The complexity of tuning additional parameters, such as the uncertainty thresholds and loop detection mechanisms, posed additional challenges for practical implementation.

Prioritized Sweeping

Prioritized sweeping, introduced by [100], improves model-based RL by concentrating updates on the most important state-action pairs. This prioritization enhances both the speed of learning and convergence. As described in Alg. 4, the core mechanisms of prioritized sweeping include model learning, priority queue management, and backward sweeping. In model learning, the algorithm builds a model of the environment, capturing state transition probabilities and rewards (lines 3-6). A priority queue is used, where state-action pairs are prioritized based on the magnitude of their expected updates (error) (lines 7-8). Backward sweeping ensures that significant changes are promptly addressed by propagating their effects backward to earlier states (lines 9-17). The process starts with experience collection (lines 3-5), where the agent interacts with the environment to gather transitions (state, action, reward, next state). The model is then updated with these transitions (line 6). For each affected state-action pair, the algorithm calculates its priority (lines 7-8), reflecting the expected magnitude of the value update. The priorities are managed in a queue, and the algorithm updates the values in order of priority until the queue is empty or a convergence criterion is met (lines 9-17) [1, 100, 101].

Prioritized sweeping refines the Bellman equation by updating states in a priority-driven manner as follows:

$$P(s,a) = \left| R(s,a) + \gamma \left(\max_{a'} Q(s',a') \right) - Q(s,a) \right| \tag{9}$$

where $P(s,a)$ is the priority of the state-action pair (s,a). By reviewing several studies in the following paragraphs, the key advantage of prioritized sweeping becomes clear: its efficiency. Concentrating updates on high-priority areas minimizes unnecessary computations and accelerates the learning process. This prioritization also results in faster convergence compared to traditional DP methods. However, the algorithm introduces additional complexity by maintaining a priority queue and performing backward sweeps, which adds computational overhead. Moreover, the efficiency improvements are dependent on the accuracy of the model, which may not always be perfect in every scenario [1, 100].

A prioritized sweeping algorithm was introduced in [101] to improve learning efficiency in stochastic Markov systems. The method combined the rapid performance of incremental learning methods like TD and Q-learning with the accuracy of DP by prioritizing updates. By maintaining a priority queue, the algorithm processed the most impactful updates first, leading to faster convergence in large state-space environments. Although the algorithm showed superior performance in simulations, it involved significant computational overhead due to the priority queue maintenance and state tracking. The complexity of setting up and tuning the priority mechanism posed challenges, particularly in dynamic environments with unpredictable state transitions.

An enhancement to the Deep Deterministic Policy Gradient (DDPG) algorithm by incorporating prioritized sweeping for morphing UAVs is introduced in [102]. The objective is to improve the efficiency and effectiveness of morphing strategy decisions by avoiding the random selection of state-action pairs and instead focusing on those with a significant impact on learning outcomes. The integration of prioritized sweeping with the DDPG framework notably enhances learning efficiency by prioritizing state-action pairs that are most influential in updating the policy. This targeted approach accelerates convergence and improves decision-making accuracy, which is crucial for the dynamic and complex task of UAV morphing. The method effectively combines the strengths of value-based and Policy Gradient-based RL, leveraging Deep Neural Networks (DNNs) for state evaluation and policy improvement. The simulation results demonstrate that the improved algorithm achieves faster learning and higher total rewards compared to DDPG, validating its superiority in handling complex morphing scenarios. Despite its advantages, the reliance on accurate prioritization mechanisms introduces additional complexity in maintaining and updating the priority queue, which could impact performance in highly dynamic environments where state changes are rapid and unpredictable. The

ARTICLE IN PRESS

Algorithm 4 Prioritized Sweeping [1]

Initialize $Q(s,a)$, Model(s,a), for all s,a, and $PQueue$ to empty
while true **do** ▷ Do forever
  $S \leftarrow$ current (non-terminal) state
  $A \leftarrow$ policy(S,Q)
  Execute action A; observe resultant reward, R, and state, S'
  Model$(S,A) \leftarrow (R,S')$
  $P \leftarrow |R + \gamma \max_a Q(S',a) - Q(S,A)|$
  if $P > \theta$ **then**
    Insert (S,A) into $PQueue$ with priority P
  end if
  for $i = 1$ to n **do**
    if $PQueue$ is not empty **then**
      $S,A \leftarrow$ first$(PQueue)$
      $R,S' \leftarrow$ Model(S,A)
      $Q(S,A) \leftarrow Q(S,A)$
      $+ \alpha[R + \gamma \max_a Q(S',a) - Q(S,A)]$
      for each $(\tilde{S},\tilde{A})$ predicted to lead to S **do**
        $\tilde{R} \leftarrow$ predicted reward for $(\tilde{S},\tilde{A},S)$
        $P \leftarrow |\tilde{R} + \gamma \max_a Q(S,a) - Q(\tilde{S},\tilde{A})|$
        if $P > \theta$ **then**
          Insert $(\tilde{S},\tilde{A})$ into $PQueue$ with
          priority P
        end if
      end for
    end if
  end for
end while

assumption that changes in sweep angle do not significantly affect flight status within short time frames may oversimplify real-world conditions, potentially limiting the model's accuracy. While the simulation results are promising, further validation in real-world applications is necessary to fully assess the method's robustness and scalability. The initial phase of training and data generation still requires significant computational resources, which could be a bottleneck in practical deployments.

Authors in [103] introduced modifications to Dyna-learning and prioritized sweeping algorithms, incorporating an epoch-incremental approach using Breadth-first Search (BFS). The combination of incremental and epoch-based updates improved learning efficiency, leading to faster convergence in dynamic environments like grid worlds. The use of BFS after episodes provided a more comprehensive understanding of the state space. However, managing dual modes of policy updates and accurate BFS calculations introduced complexity, potentially increasing computational overhead. The method showed strong simulation results, but real-world validation was necessary to understand its scalability and practical implications.

Authors in [104] introduced an innovative extension to the prioritized sweeping algorithm by employing small backups instead of full backups. The primary aim is to enhance the computational efficiency of model-based RL by reducing the complexity of value updates, making it more suitable for environments with a large number of successor states. The most notable advantage of this approach is its ability to perform value updates using only the current value of a single successor state, significantly reducing the computation time. By utilizing small backups, the algorithm allows for finer control over the planning process, leading to more efficient update strategies. The empirical results demonstrate that the small backup implementation of prioritized sweeping achieves substantial performance improvements over traditional methods, particularly in environments where full backups are computationally prohibitive. The theoretical foundation provided in the paper supports the robustness of the small backup approach, ensuring that it maintains the accuracy of value updates while enhancing computational efficiency. Additionally, the parameter-free nature of small backups eliminates the need for tuning step-size parameters, which is a common challenge in traditional sample-based methods. On the other hand, the approach's reliance on maintaining and

ARTICLE IN PRESS

updating a priority queue introduces additional memory and computational overhead, particularly in environments with many state-action pairs. While the method improves computational efficiency, it requires careful management of memory resources to store component values associated with small backups. The simulations primarily focus on deterministic environments, leaving the performance of the small backup approach in highly stochastic or dynamic settings less explored. Further validation in real-world applications is necessary to fully assess the scalability and practicality of the method. Additionally, the initial phase of model construction and priority queue management could introduce latency, impacting the algorithm's real-time performance in complex environments.

Authors in [105] introduced cooperative prioritized sweeping to efficiently handle multi-agent MDPs using factored Q-functions and Dynamic Decision Networks (DDNs). CPS managed large state-action spaces effectively, leading to faster convergence in multi-agent environments. However, the reliance on accurately specified DDN structures and the batch update mechanism introduced complexity, increasing computational overhead in dynamic environments. While simulations showed cooperative prioritized sweeping's potential, further real-world validation was needed to assess its scalability and robustness.

A prioritized sweeping approach combined with Confidence-based Dual RL (CB-DuRL) for routing in Mobile Ad Hoc Networks (MANETs) is investigated by researchers in [41]. The proposed method dynamically selects routes based on real-time traffic conditions, aiming to minimize delivery time and congestion. The key strength of this approach is its dynamic adaptability to real-time traffic conditions, addressing the limitations of traditional shortest-path routing methods. By leveraging prioritized sweeping, the algorithm prioritizes updates to the most critical state-action pairs, enhancing learning efficiency and ensuring optimal path selection under varying network loads. The inclusion of CB-DuRL refines routing decisions by considering the reliability of Q-values, thus improving the robustness and reliability of the routing protocol. Empirical results from simulations on a 50-node MANET demonstrate that the proposed method significantly outperforms traditional routing protocols in terms of packet delivery ratio, dropping ratio, and delay, showcasing its effectiveness in handling high traffic conditions and reducing network congestion. However, the approach's dependence on accurate estimation of traffic conditions and Q-values may limit its adaptability in highly dynamic or unpredictable environments where traffic patterns change rapidly. The initial phase of gathering sufficient data to populate the Q-tables and confidence values can introduce latency and computational overhead, potentially impacting the algorithm's performance in real-time applications. While the simulations provide strong evidence of the method's efficacy, further validation in larger and more diverse network topologies is necessary to fully assess its scalability and robustness. Managing priority queues and updating Q-values in a distributed manner across multiple nodes can also pose challenges in maintaining synchronization and consistency in real-world deployments.

A structured version of prioritized sweeping to enhance RL efficiency in large state spaces by leveraging Dynamic Bayesian Networks (DBNs) was introduced in [106]. The method accelerated learning by grouping states with similar values, reducing updates needed for convergence. DBNs provided a compact environment representation, further improving computational efficiency. However, the reliance on predefined DBN structures limited applicability in dynamic environments, and maintaining these structures added computational overhead. While promising in simulations, the method required further validation in diverse real-world scenarios to fully assess its scalability and robustness.

Dyna-Q

There are two variations of Dyna-Q that are commonly explored in RL: one using a tabular representation and the other utilizing function approximation. In this section, we analyze both variations of the Dyna-Q algorithm.

The Dyna-Q algorithm, introduced in [80], integrates traditional RL with planning, offering a novel approach to enhance the efficiency and adaptability of learning agents. This section provides an overview of the core concepts and mechanisms that underlie Dyna-Q. The algorithm combines learning from actual experiences with planning, utilizing a learned model of the environment. This integration enables the agent to improve its policy more quickly and efficiently by drawing from both real and simulated experiences. Dyna-Q leverages Q-learning [107], which combines immediate rewards with the discounted value of the next state, guiding the agent to take actions that maximize long-term rewards. Additionally, the Dyna-Q architecture includes a model of the environment that predicts the next state and reward given the current state and action. This model allows the agent to simulate hypothetical experiences, enabling it to plan future actions without having to interact continuously with the real environment. The agent then uses the learned model to generate simulated

ARTICLE IN PRESS

Algorithm 5 Dyna-Q [80]

1: Initialize $Q(s,a)$ and Model(s,a) for all $s \in S$ and $a \in A(s)$
2: **repeat**
3: $S \leftarrow$ current (non-terminal) state
4: $A \leftarrow \epsilon$-greedy(S,Q)
5: Execute action A; observe resultant reward, R, and state, S'
6: $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max_a Q(S',a) - Q(S,A)]$
7: Model$(S,A) \leftarrow R, S'$ (assuming deterministic environment)
8: **for** $i = 1$ to n **do**
9: $S \leftarrow$ random previously observed state
10: $A \leftarrow$ random action previously taken in S
11: $R, S' \leftarrow$ Model(S,A)
12: $Q(S,A) \leftarrow Q(S,A) +$
$\alpha[R + \gamma \max_a Q(S',a) - Q(S,A)]$
13: **end for**
14: **until** convergence or a stopping criterion is met

experiences, treating them like real experiences to update the $Q(s,a)$ values and refine the agent's policy. This process accelerates learning by allowing the agent to practice and plan multiple scenarios internally.

The Dyna-Q planning process is incremental, meaning it can be paused and resumed at any time. This feature makes the algorithm highly adaptable to dynamic environments where the agent must continuously learn and adjust its policy based on new information. The algorithm operates through a series of steps, as shown in Alg. 5. First, the agent interacts with the environment, observes the current state, selects an action, receives a reward, and transitions to the next state (lines 3-5). Based on this experience, the $Q(s,a)$ value is updated (line 6). The environment model is then updated using the observed transition (s,a,r,s') (line 7), enhancing its predictions for future planning. The agent then uses this model to generate hypothetical experiences, selecting states and actions, and predicting their outcomes (lines 8-14). These simulated experiences are used to further update the $Q(s,a)$ values, enabling the agent to refine its policy without additional real-world interactions.

This paper [108] explored the development and application of the Dyna-H algorithm, a strategy that incorporated heuristic planning for optimizing decision-making in Role-playing Games (RPGs). The Dyna-H algorithm aimed to enhance path-finding efficiency by blending heuristic search with the model-free characteristics of RL, specifically within the context of the Dyna architecture. The primary advantage of the Dyna-H algorithm lies in its innovative combination of heuristic search and effectively addressing the challenges posed by large and complex search spaces typical in RPG scenarios. This synergy allowed the algorithm to focus on promising solution branches, significantly improving learning speed and policy quality compared to traditional methods like Q-learning and Dyna-Q. Additionally, the use of heuristic planning enabled the Dyna-H algorithm to perform efficiently without requiring a complete model of the environment beforehand, which was particularly beneficial for real-time applications. However, one drawback of the approach was the reliance on predefined heuristic functions, which might have limited its adaptability to various game environments where such heuristics were not readily available or accurate. The paper's evaluation, while thorough, focused primarily on deterministic grid-like environments, potentially overlooking the complexities and stochastic nature of more dynamic and unpredictable game settings. This narrow evaluation scope might have raised questions about the generalizability of the algorithm's performance in broader contexts.

In [109], a multi-objective Dyna-Q based Routing (MODQR) algorithm designed to optimize both delay and energy efficiency in wireless mesh networks was introduced. By leveraging the Dyna-Q technique, the algorithm aimed to address dynamic network conditions, ensuring that selected paths exhibited minimal delay and optimal energy usage. The key strength of the MODQR approach was its innovative integration of Dyna-Q with multi-objective optimization, effectively balancing delay and energy efficiency in real time. The use of Dyna-Q enhanced the convergence speed, making the algorithm well-suited for dynamic and complex network environments where conditions frequently change. Additionally, the consideration of both forward and reverse link conditions in determining path quality provided a more accurate assessment of route reliability. The inclusion of dynamic learning and exploration rates further refined the algorithm's adaptability, ensuring continuous improvement and

ARTICLE IN PRESS

optimization as the network evolved. However, the approach's reliance on predefined model parameters and assumptions, such as the gray physical interference model, might have limited its flexibility across different network topologies and conditions. While the Dyna-Q algorithm's fast convergence was advantageous, its performance in highly volatile environments with extreme fluctuations might not have been as robust. Moreover, the evaluation primarily focused on simulation results, which, while promising, might not have fully captured the complexities and unpredictable behaviors encountered in real-world deployments. The lack of comprehensive real-world testing left some uncertainty regarding the algorithm's practical effectiveness and scalability.

Authors in [110] presented a novel approach for managing Type-1 diabetes Mellitus through a Glycemic control system using Dyna-Q. The system aimed to automate insulin infusion based on Continuous Glucose Monitoring (CGM) data, eliminating the need for manual carbohydrate input from patients. The primary strength of this approach lies in its innovative use of the Dyna-Q algorithm, which combines model-based and model-free RL to enhance learning efficiency and control accuracy. By leveraging past CGM and insulin data, the system could predict future glucose levels and optimize insulin dosage in real time, resulting in improved Glycemic control. The algorithm's ability to operate effectively without explicit carbohydrate information significantly reduced the cognitive load on patients, aligning well with the goals of developing a fully automated artificial pancreas. Furthermore, the incorporation of a precision medicine approach tailored the model to individual patients, enhancing the adaptability and accuracy of the Glycemic predictions. On the other hand, the reliance on precise CGM data and insulin records for model training might have posed challenges in real-world scenarios where data quality could vary. While the algorithm showed promise in simulations and preliminary tests with real patients, its generalizability across diverse patient populations and long-term robustness remained areas of concern. The study's limited real-world testing meant that further extensive clinical trials were necessary to fully validate the system's effectiveness and safety in everyday use.

Dyna-T, which integrated Dyna-Q with Upper Confidence Bounds applied to Trees (UCT) for enhanced planning efficiency, was developed in [111]. The method aimed to improve action selection by using UCT to explore simulated experiences generated by the Dyna-Q model, demonstrating its effectiveness in several OpenAI Gym environments. The primary strength of Dyna-T was its ability to combine the model-based learning of Dyna-Q with the robust exploration capabilities of UCT. This combination allowed for a more directed exploration strategy, which enhanced the agent's ability to find optimal actions in complex and stochastic environments. The algorithm's use of UCT ensured that the most promising action paths were prioritized, significantly improving learning efficiency and convergence speed. The empirical results indicated that Dyna-T outperformed traditional RL methods, especially in environments with high variability and sparse rewards, showcasing its potential for broader applications in RL tasks. Despite its advantages, Dyna-T exhibited some limitations, particularly in deterministic environments where the added computational complexity of UCT might not have yielded significant benefits. The initial overhead of constructing and maintaining the UCT structure could be costly, especially in simpler tasks where traditional Dyna-Q or Q-learning might have sufficed. Moreover, while Dyna-T showed promise in simulated environments, its performance in real-world scenarios with continuous state and action spaces remained to be fully validated. The reliance on the exploration parameter 'c' and its impact on performance also warranted further investigation to ensure robust adaptability across different problem domains.

A novel multi-path load-balancing routing algorithm for Wireless Sensor Networks (WSNs) leveraging the Dyna-Q algorithm was proposed in [112]. The proposed method, termed ELMRRL (Energy-efficient Load-balancing multi-path Routing with RL), aimed to minimize energy consumption and enhance network lifetime by selecting optimal routing paths based on residual energy, hop count, and energy consumption of nodes. The main advantage of the ELMRRL algorithm was its effective integration of Dyna-Q RL, which combined real-time learning with planning to adaptively select optimal routing paths. This dynamic adjustment ensured that the algorithm could respond to changes in network conditions, thus extending the network lifetime. The use of RL enabled each sensor node to act as an agent, making independent decisions based on local information, which was crucial for distributed environments like WSNs. Furthermore, the algorithm's focus on both immediate and long-term cumulative rewards led to more balanced energy consumption across the network, preventing premature node failures and improving overall network resilience. On the other hand, the reliance on local information for decision-making might have limited the algorithm's effectiveness in scenarios where global network knowledge could provide more optimal solutions. The initial learning phase, where nodes gathered and processed data to update their Q-values, could have introduced

ARTICLE IN PRESS

latency and computational overhead, potentially affecting the algorithm's performance in highly dynamic environments. Additionally, the approach's dependence on the correct parameterization of the reward function and learning rates could have been challenging, as these parameters significantly impacted the algorithm's efficiency and convergence speed. The paper primarily validated the algorithm through simulations, so its real-world applicability in diverse WSN deployments remained to be fully explored.

Researchers in [34] introduced a new motion control method for path planning in unfamiliar environments using the Dyna-Q algorithm. The goal of the proposed method was to improve the efficiency of motion control by combining direct RL with model learning, enabling agents to effectively navigate both dynamic and static obstacles. The main benefit of this method was its use of the Dyna-Q algorithm, which merged model-based and model-free RL techniques, facilitating concurrent learning and planning. This integration notably enhanced convergence speed and adaptability to dynamic environments, as demonstrated by faster path optimization compared to traditional Q-learning. The algorithm's capacity to simulate experiences allowed the agent to plan more efficiently, resulting in improved decision-making in complex situations. Furthermore, employing an ϵ-greedy policy for action selection ensured a balanced exploration-exploitation trade-off, which was essential for finding optimal paths in unknown environments. Nonetheless, the method's dependence on accurate environment models for effective planning might have restricted its performance in highly unpredictable settings where models might not accurately reflect real-world dynamics. The computational burden of maintaining and updating the Q-table and environment model could also have been significant, especially in large state-action spaces. Additionally, while the method demonstrated promising results in simulation environments, its scalability and robustness in real-world applications with continuous state spaces and real-time constraints needed further validation. The reliance on predefined parameters, such as learning rates and discount factors, could also have impacted the algorithm's efficiency and effectiveness, requiring careful tuning for different scenarios.

Researchers in [113] introduced an improved Dyna-Q algorithm for mobile robot path planning in unknown, dynamic environments. The proposed method integrated heuristic search strategies, Simulated Annealing (SA), and a new action-selection strategy to enhance learning efficiency and path optimization. The enhanced Dyna-Q algorithm effectively merged heuristic search and SA, significantly improving the exploration-exploitation balance. By incorporating heuristic rewards and actions, the algorithm ensured efficient navigation through complex environments, avoiding local minima and achieving faster convergence. The novel SA-ϵ-greedy policy dynamically adjusted exploration rates, optimizing the learning process. Empirical results from simulations and practical experiments showed that the improved algorithm outperformed Q-learning and Dyna-Q methods, demonstrating superior global search capabilities, enhanced learning efficiency, and robust convergence properties in both static and dynamic obstacle scenarios. Despite the performance improvements from integrating heuristic search and SA, the approach's reliance on predefined heuristic functions might have limited its adaptability to diverse environments. The initial training phase required extensive exploration, potentially leading to higher computational overhead and longer training times. Additionally, the dependence on grid-based environment representation might have restricted the algorithm's scalability to continuous state spaces. The focus on simulated and controlled real-world environments raised concerns about the algorithm's robustness and generalizability in more complex and unpredictable real-world applications. Further studies were necessary to validate the method's effectiveness in larger, unstructured, and more dynamic environments.

An innovative approach for scheduling the charging of Plug-in Electric Vehicles (PEVs) using the Dyna-Q algorithm was developed in [114]. The primary goal was to minimize long-term charging costs while considering the stochastic nature of driving behavior, traffic conditions, energy usage, and fluctuating energy prices. The method formulated the problem as an MDP and employed DRL techniques for solution optimization. The key strength of this approach lies in its effective combination of model-based and model-free RL, which enhances learning speed and efficiency. By continuously updating the model with real experiences and generating synthetic experiences, the Dyna-Q algorithm ensured rapid convergence to an optimal charging policy. This dual approach allowed for robust decision-making even in the face of uncertain and dynamic real-world conditions, such as varying energy prices and unpredictable driving patterns. Additionally, the integration of a deep-Q network for value approximation facilitated handling the vast state space inherent in PEV charging scenarios, ensuring the method's scalability and applicability to real-world settings. However, the reliance on accurate initial parameter values and predefined models for the user's driving behavior might have limited the algorithm's adaptability during the initial learning phase. The system's performance

ARTICLE IN PRESS

heavily depended on the quality and accuracy of these models, which might have varied across different users and environments. Moreover, while the approach demonstrated significant improvements in simulations, its effectiveness in diverse and more complex real-world scenarios required further validation. The potential computational overhead associated with maintaining and updating the deep-Q network and model could have posed challenges for real-time applications, especially in large-scale deployments with numerous PEVs.

Authors in [115] presented an enhanced Dyna-Q algorithm tailored for Automated Guided Vehicles (AGVs) navigating complex, dynamic environments. The key improvements included a global path guidance mechanism based on heuristic graphs and a dynamic reward function designed to address issues of sparse rewards and slow convergence in large state spaces. The improved Dyna-Q algorithm stood out by integrating heuristic graphs that provided a global perspective on path planning, significantly reducing the search space and enhancing efficiency. This method enabled AGVs to quickly orient towards their goals by leveraging precomputed shortest path information, thus mitigating the problem of sparse rewards commonly encountered in extensive environments. Additionally, the dynamic reward function intensified feedback, guiding AGVs more effectively through complex terrains and around obstacles. The experimental results in various scenarios with static and dynamic obstacles demonstrated superior convergence speed and learning efficiency compared to traditional Q-learning and standard Dyna-Q algorithms, highlighting its robustness and effectiveness in dynamic settings. However, the dependency on heuristic graphs, which required prior computation, might have limited the algorithm's adaptability in environments where real-time updates were necessary or in scenarios with unpredictable changes. The initial setup phase, involving the creation of the heuristic graph, could have introduced overheads that might not have been feasible for all applications. Furthermore, while the dynamic reward function enhanced learning efficiency, its design relied heavily on accurate modeling of the environment, which could have been challenging in highly variable or noisy conditions. The paper's focus on simulated environments left room for further validation in real-world applications, where additional factors such as sensor noise and real-time constraints could have impacted performance.

A Dyna-Q-based anti-jamming algorithm designed to enhance the efficiency of path selection in wireless communication networks subject to malicious jamming was introduced in [116]. By leveraging both model-based and model-free RL techniques, the algorithm aimed to optimize multi-hop path selection, reducing packet loss and improving transmission reliability in hostile environments. The application of the Dyna-Q algorithm in this context was innovative, combining direct learning with simulated experiences to accelerate the convergence of the Q-values. This dual approach allowed the system to adapt quickly to dynamic jamming conditions, ensuring more reliable path selection and communication efficiency. The inclusion of a reward function that considered various modulation modes based on the signal-to-jamming noise ratio enhanced the robustness of the algorithm. Simulation results demonstrated that the Dyna-Q algorithm significantly outperformed traditional Q-learning and multi-armed bandit models, achieving faster convergence to optimal paths and better handling of interference, thus showcasing its potential for real-time applications in complex electromagnetic environments. Nevertheless, the method's reliance on pre-established environmental models might have limited its effectiveness in highly unpredictable or rapidly changing conditions, where the initial models might not accurately capture real-time dynamics. The need for accurate initial state representations and model updates introduced additional computational overhead, which could have impacted performance in larger or more complex networks. Furthermore, while the algorithm showed promise in simulated environments, its scalability and adaptability in real-world applications with varying node densities and jamming strategies required further validation. The focus on hop count minimization might also have overlooked other critical factors, such as energy consumption and latency, which were essential for comprehensive network performance assessment.

Researchers in [117] introduced a model-based RL method using Dyna-Q tailored for multi-agent systems. It emphasized efficient environmental modeling through a tree structure and proposed methods for model sharing among agents to enhance learning speed and reduce computational costs. The approach leveraged the concept of knowledge sharing, where agents with more experience assisted others by disseminating valuable information. The integration of a tree-based model for environmental learning within the Dyna-Q framework significantly enhanced the efficiency of model construction and memory usage. This method allowed agents to generate virtual experiences, thus accelerating the learning process. The innovative model sharing techniques proposed in the paper, such as grafting partial branches and resampling, enabled agents to build more accurate models collaboratively. By reducing redundant data transfer and focusing on useful experiences, these sharing methods improved

ARTICLE IN PRESS

sample efficiency and learning speed in complex environments. The simulation results demonstrated the effectiveness of these techniques in multi-agent cooperation scenarios, highlighting their potential to optimize learning in large, continuous state spaces. Despite its advantages, the reliance on accurate decision tree models for sharing experiences might have limited the approach's flexibility in highly dynamic or heterogeneous environments. The effectiveness of the sharing methods depended on the quality and relevance of the shared models, which might have varied across different agents and scenarios. Additionally, the initial phase of building accurate tree models could have been computationally intensive, particularly in environments with high variability. While the proposed methods showed promising results in simulations, further validation in diverse real-world applications was needed to fully assess their scalability and robustness. The paper also assumed a certain level of homogeneity among agents, which might not always have been applicable in more varied multi-agent systems.

The application of the Dyna-Q algorithm for path planning and obstacle avoidance in Unmanned Ground Vehicles (UGVs) and UAVs within complex urban environments was explored in [118]. The study focused on utilizing a vector field-based approach for effective navigation and air-ground collaboration tasks. The integration of the Dyna-Q algorithm with a vector field method significantly enhanced the efficiency and accuracy of path planning in dynamic urban settings. The approach leveraged both real and simulated experiences to adaptively update the agent's policy, ensuring rapid convergence to optimal paths. By simplifying the urban environment into a grid world, the method allowed for precise waypoint calculation, facilitating smooth navigation and effective obstacle avoidance. The use of PID controllers for UAV and UGV coordination further improved the stability and responsiveness of the system, enabling robust air-ground collaboration. Simulation results demonstrated that the proposed method effectively handled dynamic obstacles and complex path scenarios, showcasing its potential for real-world applications in urban environments. On the other hand, the reliance on grid-based environment representation might have limited the algorithm's scalability and adaptability to continuous state spaces found in more diverse and unstructured urban areas. The initial phase of creating the vector field and the grid map could have introduced computational overheads, which might have impacted real-time performance. While the paper focused on simulated environments, further validation in real-world scenarios was necessary to assess the approach's robustness and effectiveness under varying conditions. Additionally, the method's dependency on accurate dynamic models for both UGV and UAV could have posed challenges, as any discrepancies between the model and the real environment might have affected the overall performance.

This paper [119] evaluated the performance of the Dyna-Q algorithm in robot navigation within partially known environments containing static obstacles. The study extended the Dyna-Q algorithm to multi-robot systems and conducted extensive simulations to assess its efficiency and effectiveness. The primary strength of this study lies in its thorough analysis of the Dyna-Q algorithm in both single and multi-agent contexts. By integrating planning and model-based learning, the Dyna-Q algorithm sped up the learning process, enabling robots to navigate efficiently even with limited prior knowledge of the environment. The use of simulations with the robot motion toolbox allowed for a comprehensive evaluation of the algorithm's performance across various parameters, providing valuable insights into the optimal settings for different scenarios. Extending the Dyna-Q algorithm to multi-robot systems showcased its adaptability and potential for complex task coordination, where agents could share knowledge to enhance overall system performance. However, the paper's focus on static obstacles and deterministic environments might have limited the applicability of the findings to more dynamic and stochastic settings. The initial need for environment discretization and model construction introduced additional computational overhead, which could have been a bottleneck in real-time applications. While the simulations offered a robust analysis, the lack of real-world validation left some uncertainty about the algorithm's practical effectiveness in unpredictable and continuously changing environments. Furthermore, the performance degradation observed in multi-agent scenarios indicated that further refinement was needed to improve coordination and reduce inter-agent interference.

The main advantage of Dyna-Q is its ability to accelerate learning by combining both real and simulated experiences. This dual approach reduces the reliance on extensive real-world interactions, making it ideal for applications where such interactions are costly or limited. Additionally, Dyna-Q is naturally adaptable to changing environments. The combination of continuous learning and planning allows the agent to dynamically update its policy in response to new information or environmental changes [1, 80]. Table 3 summarizes the studies that have applied model-based planning algorithms.

ARTICLE IN PRESS

Table 3: Model-based Planning Paper Review by Domain

Application Domain	References
Theoretical Research (Convergence, stability)	[101], [104]
Multi-agent Systems and Autonomous Behaviors	[49], [105] [117], [114], [115], [118]
Games and Simulations	[88], [90], [92], [94], [96], [97], [98], [99], [37] [108], [113]
Energy and Power Management (IoT Networks, Smart Energy Systems)	[112], [110]
Transportation and Routing Optimization	[41]
Network Resilience and Optimization	[116], [112]
Hybrid RL Algorithms	[91], [93], [95], [102] [111]
Dynamic Bayesian networks	[106]
Dynamic Environments	[109], [34], [113], [115]
Robotics	[119]

3.1.3 Algorithmic Rationale and Selection Criteria: Model-Based Planning

The integration of planning and learning allows agents to simulate experiences, drastically reducing the number of real-world interactions required. The choice among model-based algorithms depends on the environment's complexity, the availability of a transition model, and the computational budget.

- **Monte Carlo Tree Search (MCTS):** Practitioners select MCTS when dealing with massive state spaces where traditional dynamic programming is computationally intractable, but a perfect forward model (or ruleset) exists. Its ability to balance exploration and exploitation asymmetrically makes it the gold standard for discrete, adversarial environments like board games (e.g., Go, Chess) and complex multi-agent path-planning [49, 88].
- **Prioritized Sweeping:** This algorithm is chosen when learning efficiency must be maximized in environments with sparse but highly impactful rewards. By focusing computational resources on state-action pairs with the largest Bellman errors, prioritized sweeping accelerates convergence. It is highly favored in dynamic routing networks or multi-agent environments where rapid propagation of significant updates is required to maintain system stability [41, 105].
- **Dyna-Q:** Dyna-Q bridges the gap between model-free learning and model-based planning. It is uniquely selected for environments where real-world sampling is expensive, slow, or risky (e.g., medical dosage planning or EV charging schedules [110, 114]). By allowing the agent to "hallucinate" or simulate experiences using a learned model, Dyna-Q significantly improves sample efficiency while remaining adaptable to environment changes.

3.2 Tabular Model-free Algorithms

Tabular model-free algorithms are designed for problems with discrete state and action spaces that are sufficiently small to be represented in tabular form. Unlike model-based approaches, which rely on an explicit model of the environment's dynamics, model-free algorithms learn directly from interactions with the environment without requiring knowledge of its internal structure. In this context, we examine various methods classified as tabular model-free algorithms, beginning with MC methods.

3.2.1 Monte Carlo Methods

When complete knowledge of the environment is either unavailable or impractical, MC methods provide an effective alternative. These methods learn from experience by utilizing sampled sequences of states, actions, and rewards obtained through real or simulated interactions with the environment [1]. A key advantage of learning from real experience is that MC does not require prior knowledge of the environment's dynamics while still being capable of achieving optimal behavior. Similarly, learning from simulated experience is advantageous, as it allows for effective policy learning with a model that only needs to generate sample transitions rather than the full probability distributions of all possible transitions, as required in DP. To ensure well-defined returns, MC methods are primarily

ARTICLE IN PRESS

applied to episodic tasks, where experiences are divided into episodes that always reach a terminal state regardless of the chosen actions. Value estimates and policies are updated only at the end of an episode based on sample returns. Consequently, MC methods operate incrementally on an episode-by-episode basis rather than in a step-by-step (online) manner. Although the term "Monte Carlo" is often broadly associated with estimation techniques involving significant randomness, in this context, it specifically refers to methods that compute averages over complete returns [1] (see Figure 4).

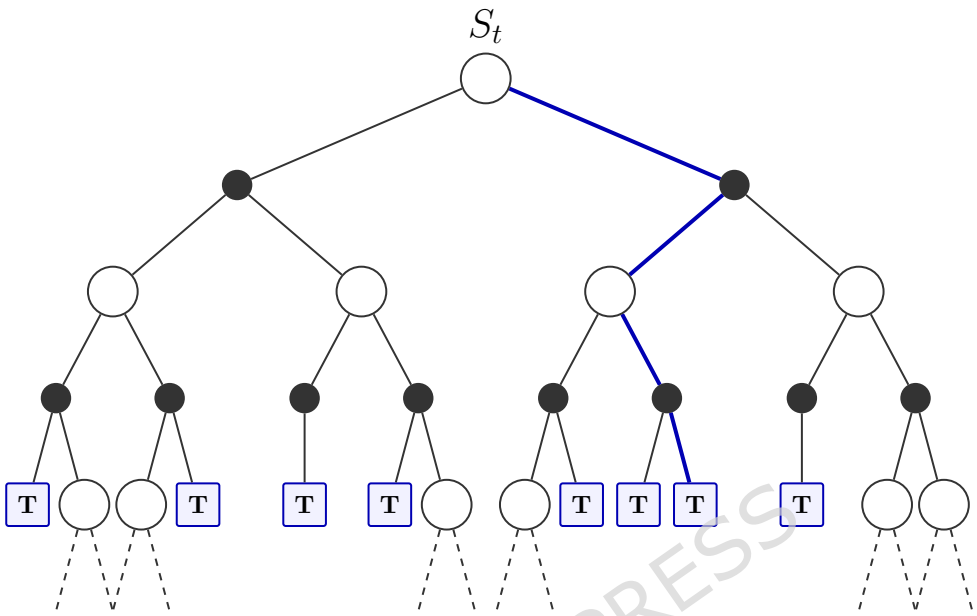


Figure 4: An illustration of the Monte Carlo (MC) backup approach. The update relies on a full sample trajectory extending all the way to a terminal state (T), representing a deep but narrow backup.

ARTICLE IN PRESS

Next, we describe how MC methods are utilized to learn the state-value function for a given policy π, outlining the various approaches available for this purpose.

Monte Carlo Estimation of State Values

A fundamental approach to estimating the value of a state from experience is to compute the average of its observed returns after each visit. The value of a state is defined as the expected cumulative discounted reward in the future. By the law of large numbers, this average converges to the expected value as more returns are observed, forming the foundation of MC methods. The value function $v_\pi(s)$ for a state s under policy π is estimated using a set of episodes generated by following π and passing through s. Each occurrence of s within an episode is referred to as a visit. If s appears multiple times in a single episode, the first occurrence is termed the first visit. The **first-visit MC method** estimates $v_\pi(s)$ by averaging returns only from first visits to the state, whereas the **every-visit MC method** averages returns from all visits to s.

In MC, both the first-visit and every-visit methods converge to $v_\pi(s)$ as the number of visits approaches infinity [1]. Algorithm 6 outlines the first-visit MC approach for estimating V_π. The only distinction between the two methods, as noted earlier, is in line 8, where every-visit MC updates the value function based on all occurrences of state s rather than only the first visit.

MC Estimation of Action Values (with Exploration Starts)

In the absence of an environment model, estimating action values (values associated with state-action pairs) rather than state values becomes particularly advantageous. While state values are sufficient for deriving a policy by evaluating one step ahead and selecting the action that maximizes the expected reward and next-state value [1], they alone are inadequate without a model. An explicit estimate of each action's value is necessary to provide meaningful policy guidance. A deterministic policy π poses a major challenge by restricting exploration to a limited subset of state-action pairs, as only one action is observed per state. Consequently, MC estimates for unvisited actions do not

ARTICLE IN PRESS

Algorithm 6 First-visit MC [1]

```
Initialize:
π ← policy to be evaluated
V ← an arbitrary state-value function
Returns(s) ← an empty list, for all s ∈ S
repeat
    Generate an episode using π
    for each state s appearing in the episode do
        G ← return following the first occurrence of s
        Append G to Returns(s)
        V(s) ← average(Returns(s))
    end for
until forever
```

improve with experience, leading to a critical learning limitation. By estimating action values, an agent can evaluate all available actions in each state, facilitating more informed decision-making. It is essential to estimate the value of all actions from each state rather than only the currently favored one to ensure meaningful comparisons.

While exploring starts can be useful in some cases, they are not always feasible, particularly when learning directly from the environment, where initial conditions may not be favorable. As an alternative, stochastic policies that assign a non-zero probability to all actions in each state can ensure that all state-action pairs are sufficiently explored. To introduce an MC-based adaptation of classical policy iteration, we begin by iteratively evaluating and improving policies. This approach starts with an arbitrary policy π_0 and progressively refines it until both the optimal policy and the optimal action-value function are obtained.

$$\pi_0 \xrightarrow{\text{Evaluation}} q_{\pi_0} \xrightarrow{\text{Improvement}} \pi_1 \xrightarrow{\text{Evaluation}} q_{\pi_1} \xrightarrow{\text{Improvement}} \pi_2 \xrightarrow{\text{Evaluation}} \cdots \xrightarrow{\text{Improvement}} \pi_* \xrightarrow{\text{Improvement}} q_{\pi_*} \tag{10}$$

As the approximate action-value function asymptotically converges to the true function, numerous episodes occur. For now, we assume an infinite number of observed episodes, all generated with exploration starts. Exploration starts refer to the assumption that all states have a nonzero probability of being the starting state. While this assumption promotes exploration, it is not always practical, as it does not hold in many real-world applications. Under these assumptions, for any arbitrary policy π_k, MC methods can accurately estimate q_{π_k}.

To improve the policy, it must be made greedy with respect to the current value function. This is achieved by utilizing the action-value function to construct a greedy policy without requiring a model. Given an action-value function q, a greedy policy selects the action that maximizes the action-value for each state $s \in S$.

$$\pi(s) \equiv \arg\max_a q(s,a) \tag{11}$$

Policy improvement is achieved by defining each π_{k+1} as the greedy policy with respect to q_{π_k}. The Policy Improvement Theorem, previously discussed, applies to π_k and π_{k+1}, since for all $s \in S$, the following holds:

$$q_{\pi_k}(s, \pi_{k+1}(s)) = q_{\pi_k}(s, \arg\max_a q_{\pi_k}(s,a)) = \max_a q_{\pi_k}(s,a) \geq q_{\pi_k}(s, \pi_k(s)) = v_{\pi_k}(s) \tag{12}$$

According to the General Policy Theorem [1], each π_{k+1} is either strictly superior to or equally optimal as π_k, ensuring that both policies converge to the optimal solution. Consequently, this process leads to an optimal value function and policy. Using MC methods, optimal policies can be derived exclusively from sample episodes without requiring explicit knowledge of the environment's dynamics.

Although MC simulations guarantee convergence, two key assumptions must be addressed for practical implementation: the requirement of exploration starts in episodes and the necessity of an infinite number of episodes for policy evaluation. Our focus is on eliminating the assumption of infinite episodes. This can be accomplished by approximating q_{π_k} during each evaluation, leveraging empirical measurements and assumptions to minimize error. While this approach theoretically guarantees

ARTICLE IN PRESS

Table 4: MC Paper Review by Domain

Application Domain	References
Train Control and Energy Efficiency	[27]
Algorithmic RL (Renewal Theory, Rough Set Theory)	[123], [122]
Robotics and Trajectory Control	[23]
Bayesian RL	[124], [125]
Game Strategy and Card Games	[126]
Signal Processing and Parameter Estimation	[43]
Network Reliability and Blocking Probabilities	[30]
Simulation Efficiency in Rare-Event Phenomena	[25]
Rough Set Theory and Approximation Spaces	[127]

correct convergence, it often necessitates an impractically large number of episodes, particularly in complex environments. An alternative is to avoid full policy evaluation before updating the policy. Instead, the value function is incrementally adjusted toward q_{π_k} over multiple steps, as observed in generalized policy iteration. A similar approach is seen in value iteration, where policy improvement occurs after just one policy evaluation step.

MC policy iteration inherently alternates between evaluation and improvement at the end of each episode. The returns observed within an episode are utilized for evaluation, followed by policy improvement for all visited states. While the assumption of exploration simplifies estimation by ensuring that all state-action pairs are encountered, it is often impractical. By relaxing this assumption, an on-policy MC approach can be employed, where action values are estimated directly from trajectories generated by the policy itself. In the following discussion, we examine this alternative approach, which does not rely on exploration starts.

On-policy MC (MC without Exploration Starts)

MC with on-policy starts, also referred to as MC without exploration starts, is a learning approach where the agent derives knowledge from episodes generated by following its current policy. Unlike methods that employ exploratory starts, allowing the agent to begin from any state-action pair, on-policy MC relies solely on actual experiences encountered while executing the policy in the environment. Consequently, the agent estimates the value of the policy by averaging cumulative rewards from episodes that begin and terminate according to the policy. Ensuring sufficient exploration is crucial for obtaining accurate value estimates, as the agent's experiences are restricted to those dictated by its current policy. This can be achieved by employing stochastic policies or incorporating soft exploration strategies. The approach is particularly useful in scenarios where exploration starts are impractical or where the policy must be learned through direct environmental interaction [1, 120].

We highlight two studies in this section, recognizing that numerous other contributions exist in the literature.

In [121], researchers introduced an off-policy MC learning method that incorporated approximation spaces and rough set theory. This approach refined the RL process in dynamic environments by employing weighted sampling based on observed behavior patterns, enabling a more precise estimation of action values and enhancing learning efficiency and adaptability. The method's success depended on accurately identifying and leveraging behavior patterns, particularly in complex environments. A more extensive comparison with other advanced RL techniques could have further clarified its effectiveness and scalability across various applications.

In [122], the authors developed an approach that advanced RL by utilizing rough set theory and approximation spaces to evaluate state values within MC without exploration starts. Tested in a simulation modeling zebra danio fish behavior, this method integrated observed patterns to refine action-value estimations, demonstrating the potential for modeling complex systems. While the technique was innovative, it introduced computational challenges, underscoring the trade-off between methodological advancement and practical feasibility. The research highlighted the potential of pattern-based evaluation while emphasizing the importance of scalability and broader implementation.

Table 4 gives a summary of articles that utilized MC methods. Over the next subsection, we analyze the integration of importance sampling [128], a well-known concept in statistics, with MC methods.

ARTICLE IN PRESS

Algorithm 7 Off-policy Prediction (via Importance Sampling) [1]

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:
  $Q(s,a) \in \mathbb{R}$ (arbitrarily)
  $C(s,a) \leftarrow 0$
  $\pi(s) \leftarrow \arg\max_a Q(s,a)$ (ties broken)
repeat
  $b \leftarrow$ any soft policy
  Generate an episode using b: $(S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T)$
  $G \leftarrow 0$
  $W \leftarrow 1$
  for each step of episode, $t = T-1, \ldots, 0$: **do**
    $G \leftarrow \gamma G + R_{t+1}$
    $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
    $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)}\left[G - Q(S_t, A_t)\right]$
    $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$(with ties broken consistently)
    if $A_t \neq \pi(S_t)$ **then**
      break ▷ Proceed to next episode
    end if
    $W \leftarrow W \cdot \frac{1}{b(A_t|S_t)}$
  end for
until forever

Off-policy Prediction via Importance Sampling

The use of RL for control presents a fundamental challenge: the need to learn action values based on the assumption of subsequent optimal behavior, while also requiring non-optimal behavior to explore all possible actions in search of the optimal one. This raises the question of how to learn optimal policies while following exploratory policies. A policy-based approach provides a compromise by learning action values for a policy that, while not necessarily optimal, remains close to optimal and includes exploration mechanisms. However, due to its exploratory nature, this approach does not directly resolve the problem of learning the optimal action values.

Off-policy learning addresses this challenge by utilizing two distinct policies: a target policy (π), which aims to become the optimal policy, and a behavior policy (b), which generates behavior. This dual-policy framework enables exploration to be conducted separately from the process of learning the optimal policy, allowing the agent to learn from data generated outside the target policy. Off-policy methods are more flexible and powerful than on-policy approaches, with on-policy methods being a special case where the target and behavior policies are identical. However, off-policy methods introduce additional complexity, necessitating more advanced concepts and notation. Since off-policy learning relies on data from a different policy, it tends to have higher variance and slower convergence compared to on-policy learning. While on-policy methods offer simplicity and allow direct learning from the agent's own exploratory actions, off-policy methods provide a more robust framework for learning optimal policies indirectly through a separate behavior policy. Ultimately, the distinction between on-policy and off-policy learning reflects the exploration-exploitation trade-off inherent in RL, enabling agents to effectively learn and adapt in complex environments [3]. Alg. 7 provides a general overview of this algorithm.

In the following paragraphs, we examine selected research studies that have applied MC methods and their variations, evaluating their underlying rationale and addressing the challenges they encountered. By conducting an in-depth analysis of these studies, we aim to highlight the versatility and effectiveness of MC methods in solving diverse problems while emphasizing the decision-making processes that led to their adoption.

Using MC with historical data, [27] proposed an intelligent train control approach aimed at enhancing energy efficiency and punctuality. This model-free method achieved a 6.31% reduction in energy consumption while improving schedule adherence. However, its effectiveness was highly dependent on the availability and quality of historical data, and its scalability to larger, more complex railway networks remained an open question.

ARTICLE IN PRESS

In [123], the authors introduced a Renewal MC (RMC) algorithm designed to reduce variance, minimize update delays, and accelerate convergence to locally optimal policies. The approach demonstrated strong performance in continuous state and action spaces and was applicable to various fields, including robotics and game theory. Additionally, an approximate version was proposed to improve convergence speed while maintaining bounded errors. However, the performance of RMC was influenced by the choice and size of the renewal set.

The study by [122] introduced an MC off-policy strategy enhanced with rough set theory, presenting a novel methodology in the field. While this integration provided new insights, its complexity posed challenges for broader applicability. Furthermore, the research primarily focused on theoretical formulations, requiring additional empirical validation.

Authors in [23] developed a Bayesian model-free Markov chain MC algorithm for policy search, applied specifically to a 2-DoF robotic manipulator. The algorithm proved to be both practical and effective in real-world implementations, utilizing a gradient-free strategy that simplified optimization and excelled in controlling complex trajectories with relatively few iterations. However, its applicability might be restricted to specific domains, and the estimator's high variance introduced challenges.

The work by [124] introduced an innovative MC Bayesian RL approach that streamlined Bayesian RL (BRL). By sampling a limited set of hypotheses, the method constructed a discrete Partially Observable Markov Decision Process (POMDP), eliminating the reliance on conjugate distributions and enabling the use of point-based approximation algorithms. This framework was adaptable to both fully and partially observable environments, demonstrating robust performance across various domains. However, its effectiveness was contingent on the choice of prior distributions, and insufficient sample sizes could degrade performance.

In [125], the authors proposed a factored MC BRL method to address BRL problems in an online setting. By leveraging factored representations, the approach reduced the complexity of learning parameters and employed partially observable MC planning as an online solver. This significantly enhanced scalability and efficiency, particularly in large-scale domains.

Researchers in [126] explored the development of self-learning agents for the Batak card game using MC methods for state-value estimation combined with Artificial Neural Networks (ANNs) for function approximation. This approach effectively managed the game's large state space, enabling agents to progressively improve their gameplay. However, the study's focus on a specific card game might limit the broader applicability of its findings.

Next, we explore **MC with importance sampling**, an off-policy learning algorithm.

MC with Importance Sampling

MC importance sampling is a technique designed to enhance the efficiency of MC simulations when estimating expected values. Instead of sampling directly from the original distribution, it draws samples from a proposal distribution. As illustrated in Alg. 8, the samples are re-weighted based on the ratio of the original distribution to the proposal distribution. This re-weighting process corrects the bias introduced by sampling from an alternative distribution, enabling more accurate and efficient estimations [129].

Next, we examine several studies that have applied MC importance sampling.

In [43], the authors proposed a non-iterative method for estimating the parameters of superimposed chirp signals in noisy environments using MC importance sampling. The approach employed maximum likelihood estimation to optimize the process efficiently, offering an alternative to traditional grid-search methods. The method focused on estimating chirp rates and frequencies, providing a computationally efficient solution for multidimensional problems. By eliminating iterative steps, it reduced computational complexity while maintaining high accuracy, even in challenging conditions. Additionally, the technique was scalable to multiple signal scenarios without significantly increasing computational demands. However, performance could degrade below certain signal-to-noise ratio thresholds, and problem-specific tuning of MC samples and parameters was necessary.

In [30], a simulation-based method was introduced for estimating blocking probabilities in multicast loss systems, enhancing static MC methods with importance sampling. The approach decomposed the complex problem into simpler subproblems by isolating the contributions of individual network links to the blocking probability. A tailored distribution was used for each link's blocking states, and an inverse convolution method for sample generation, combined with a dynamic control algorithm, significantly reduced variance. While the method effectively allocated samples and

ARTICLE IN PRESS

Algorithm 8 MC with Importance Sampling [1]

Initialize, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$:
  $Q(s,a) \leftarrow$ arbitrary
  $C(s,a) \leftarrow 0$
  $\mu(a|s) \leftarrow$ an arbitrary soft behavior policy
  $\pi(a|s) \leftarrow$ an arbitrary target policy
repeat ▷ For ever
  Generate an episode using μ: $(S_0, A_0, R_1, \ldots, S_T, A_T, R_T, S_T)$
  $G \leftarrow 0$
  $W \leftarrow 1$
  for $t = T-1, T-2, \ldots, 0$ down to 0 **do**
    $G \leftarrow \gamma G + R_{t+1}$
    $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
    $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)}\left[G - Q(S_t, A_t)\right]$
    $W \leftarrow W \cdot \frac{\pi(A_t|S_t)}{\mu(A_t|S_t)}$
    if $W = 0$ **then**
      break ▷ Exit inner loop
    end if
  end for
until convergence or a stopping criterion is met

improved variance reduction, its complexity and setup requirements could present barriers compared to more straightforward approaches.

The study by [25] developed an important sampling technique to improve simulation efficiency for rare-event phenomena. Initially designed for Markovian random walks, the method was later extended to accommodate non-Markovian processes and molecular dynamics simulations. By optimizing the sampling of successful transition paths, the approach enhanced computational efficiency. Additionally, a strategy for identifying an optimal importance function was introduced. However, optimizing this function remained challenging, especially in systems with extensive state spaces or those deviating from Markovian assumptions.

In [127], the authors presented an advanced MC importance sampling approach for evaluating the reliability of stochastic flow networks. The method utilized a recursive state-space decomposition to efficiently target critical network segments, reducing computational costs and improving reliability estimate precision. However, its applicability was potentially limited in very large or highly complex networks due to its dependence on specific network attributes.

3.2.2 Temporal Difference (TD) Learning

TD methods are regarded as a hybrid of MC methods and DP. Like MC approaches, TD learning enables agents to learn directly from raw experience without requiring a model of the environment's dynamics. At the same time, TD algorithms share similarities with DP, as they refine predictions using previously learned estimates rather than waiting for a final outcome to update values (a process known as bootstrapping). Figure 5 illustrates the general working mechanism of TD methods. The following discussion introduces and analyzes various studies on TD-based approaches to provide a broad understanding of these algorithms before examining their specifics. The discussion begins with TD(0) and progresses to N-step SARSA, which will be covered in detail later in this survey.

TD(0)

Alg. 9 describes tabular TD(0) for estimating V_π. While MC methods require waiting until the conclusion of an episode to calculate the update to $V(S_t)$ as G_t, TD methods only need to wait until the subsequent timestep. At time $t+1$, TD methods swiftly establish a target and perform an effective update using the observed reward R_{t+1} and the estimated value $V(S_{t+1})$. It is important to recognize that the expression within brackets in the TD(0) update represents a type of error. This error measures the discrepancy between the estimated value of S_t and a more refined estimate, $R_{t+1} + \gamma V(S_{t+1})$. This discrepancy is known as the TD error. The most basic form of the TD method executes this update as follows (line 7 in Alg. 9):

ARTICLE IN PRESS

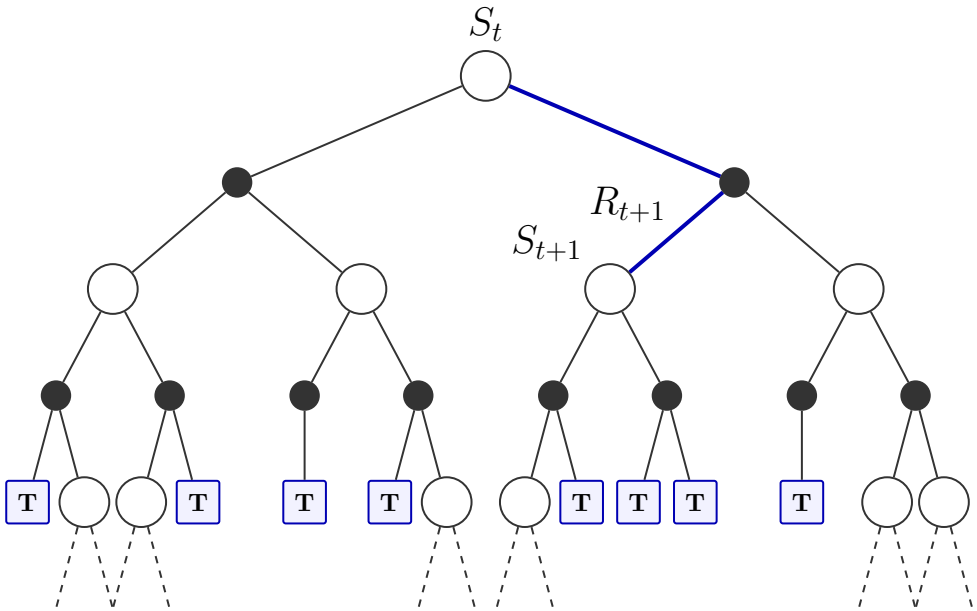


Figure 5: An overview of the Temporal Difference (TD) backup approach. The update bootstraps from a single, 1-step sample transition, representing a narrow and shallow backup.

Algorithm 9 Tabular TD(0) [1]

Initialize the value function $V(s)$ arbitrarily (e.g., $V(s) = 0$ for all $s \in S^+$)
repeat ▷ For each episode
  Initialize state S
  repeat ▷ For each step of the episode
    $A \leftarrow$ action given by policy π for S
    Take action A; observe reward R and next state S'
    $V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$
    $S \leftarrow S'$
  until S is terminal
until convergence or for a specified number of episodes

ARTICLE IN PRESS

$$V(S_t) \leftarrow V(S_t) + \alpha \left[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\right] \tag{13}$$

Upon transitioning to S_{t+1} and receiving R_{t+1}, TD methods implement the update immediately. The target for an MC update is G_t, while the target for the TD update is $R_{t+1} + \gamma V(S_{t+1})$. This TD method is called TD(0), or one-step TD. TD(0) is a bootstrapping method, like DP, as its update is in part on an existing estimate [1].

TD(0) serves as a foundational concept for understanding other TD learning algorithms, including Q-learning, SARSA, and double Q-learning, which are examined in detail in the following subsections.

TD(0)-Replay

TD(0)-replay algorithm (Alg. 10), introduced in [130], enhances policy learning efficiency by leveraging full experience replay. Utilizing the agent's entire history of interactions allows for comprehensive updates to the value function and policy at each step. δ_t refers to the TD error, R_{t+1} is the reward signal, γ is a discount factor, ϕ_t^T is the transpose of feature vector ϕ_t obtained through current state S_t, θ_t^T is the transpose of the weight vector θ_t, and α_t is a learning step; all varies according to time step t.

This approach accelerates convergence to optimal policies, particularly in environments where acquiring new experiences is costly or challenging. By efficiently replaying past experiences, TD(0)-replay proves invaluable in domains where re-experiencing events is infeasible or expensive. The method facilitates rapid optimization of the agent's value function, leading to faster learning and adaptation in dynamic environments, thereby offering significant advantages over approaches that

ARTICLE IN PRESS

Algorithm 10 TD(0)-Replay [130]

1: **Input:** $\alpha, \gamma, \theta_{\text{init}}$
2: $\theta \leftarrow \theta_{\text{init}}$
3: **loop** ▷ Over episodes
4: Obtain initial state S, features ϕ
5: $\hat{e} \leftarrow I_{n\times n}$, $e \leftarrow \mathbf{0}_{n\times 1}$
6: **while** (terminal state has not been reached) **do**
7: Act according to the policy
8: Observe next reward $R = R_{t+1}$, next state $\hat{S} = S_{t+1}$, and its features $\hat{\phi} = \phi_{t+1}$
9: $\alpha \leftarrow \ell(\alpha)$
10: $e \leftarrow e + \alpha\phi \left(\left(\gamma\hat{\phi} - \phi \right)^{\top} e + R \right)$
11: $\hat{e} \leftarrow \hat{e} + \alpha\phi \left[\left(\gamma\hat{\phi} - \phi \right)^{\top} \hat{e} \right]$
12: $\theta \leftarrow \theta + \hat{e} + e$
13: $\phi \leftarrow \hat{\phi}$
14: **end while**
15: **end loop**

lack replay mechanisms. The algorithm's adaptability is further demonstrated through dynamic updates of weight parameters based on historical data, ensuring effectiveness as environmental conditions evolve [130]. Its broad applicability across various domains highlights its versatility, making it suitable for both theoretical research and practical implementations.

TD(λ)

TD(λ) is a powerful and flexible algorithm introduced in [4], extending the simpler TD(0) method by incorporating a parameter λ that controls the weighting of n-step returns. By blending one-step updates with MC methods, TD(λ) enables the algorithm to account for entire trajectories of experiences rather than only the immediate next step. A key feature of TD(λ) is the eligibility trace, which maintains a record of visited states and their relevance to learning updates. This mechanism addresses the credit assignment problem, helping determine how many past or future steps should contribute to an update. Algorithm 11 provides a comprehensive representation of the TD(λ) algorithm. The eligibility trace (lines 9-11) acts as a temporary record of events, such as state visits, and decays over time, governed by the parameter λ. This decay mechanism enables the algorithm to assign credit to prior states in a temporally distributed manner, effectively capturing delayed reward dependencies.

The value update in TD(λ) is given by (line 13):

$$V(s) \leftarrow V(s) + \alpha\delta e(s) \tag{14}$$

In TD(λ), α represents the learning rate, δ denotes the TD error defined as $\delta = R + \gamma V(s') - V(s)$, and $e(s)$ is the eligibility trace associated with state s. TD(λ) establishes a continuum between TD(0) (pure TD updates) and MC (full return) methods, often achieving greater efficiency by balancing the bias-variance trade-off. By enabling the rapid propagation of value estimates across the state space, TD(λ) enhances learning efficiency and accelerates convergence [1, 4].

Next, we examine a selection of studies that have applied TD(λ) variants to explore the practical applications of this algorithm.

In [131], the authors introduced KnightCap, a chess program that learned its evaluation function using TDLeaf(λ), a variation of TD(λ) integrated with game-tree search. By playing on the Free Internet Chess Server, KnightCap improved its rating from 1650 to 2150 within 308 games over three days. This integration of TDLeaf(λ) with game-tree search marked a significant advancement in combining RL with traditional AI techniques in chess. The approach effectively leveraged deep evaluation through game-tree search while utilizing TD learning to refine its evaluation function dynamically. KnightCap's rapid improvement demonstrated the benefits of online learning against diverse opponents, emphasizing the role of varied strategic encounters. The adaptation of TD(λ) for minimax search allowed iterative improvement based on real-game outcomes, showcasing the algorithm's potential in interactive environments. However, while the results were promising, further

ARTICLE IN PRESS

Algorithm 11 TD(λ) [1]

Initialize $V(s)$ arbitrarily (but set to 0 if s is terminal)
repeat ▷ For each episode
  Initialize $E(s) = 0$, for all $s \in S$
  Initialize S
  repeat ▷ For each step of episode
    $A \leftarrow$ action given by π for S
    Take action A, observe reward R, and next state S'
    $\delta \leftarrow R + \gamma V(S') - V(S)$
    $E(S) \leftarrow E(S) + 1$ ▷ Accumulating traces
    or $E(S) \leftarrow (1 - \alpha)E(S) + 1$ ▷ Dutch traces
    or $E(S) \leftarrow 1$ ▷ Replacing traces
    for all $s \in S$ **do**
      $V(s) \leftarrow V(s) + \alpha\delta E(s)$
      $E(s) \leftarrow \gamma\lambda E(s)$
    end for
    $S \leftarrow S'$
  until S is terminal
until convergence or a stopping criterion is met

validation across varied game scenarios and against stronger AI opponents would provide a more comprehensive evaluation.

In [132], the authors extended the convergence theorem for TD learning to handle arbitrary time steps, which was critical for predicting future outcomes from past states. This theoretical advancement strengthened the foundation of TD(λ), ensuring convergence even in the presence of linearly dependent state representations. The work bridged the gap between TD learning and DP, reinforcing the strong convergence properties of TD methods. While primarily theoretical, the study provided strong guarantees regarding the stability and reliability of TD learning. Additional empirical validation in dynamic environments would further substantiate the proposed methods.

Building on prior theoretical work, [133] established a rigorous proof that TD(λ) algorithms converge with probability 1 under specific conditions. Expanding upon the earlier contributions of Sutton and Watkins, the authors formulated a general proof, ensuring reliable and consistent predictions. This theoretical advancement was instrumental in fostering broader adoption of TD learning, as it provided strong assurances regarding stability and reliability. Although the paper remained theoretical, its detailed analysis significantly contributed to a deeper understanding of TD learning dynamics, informing future research directions.

In [134], the authors introduced QV(λ)-learning and the Actor-Critic Learning Automaton (ACLA), both extensions of TD(λ). QV(λ)-learning combined value function estimation with aspects of Q-learning, while ACLA applied a learning automaton-inspired update rule to the actor component. These algorithms were tested in various environments, demonstrating robustness and generalizability. QV(λ)-learning effectively merged the strengths of traditional Q-learning and actor-critic frameworks, resulting in improved stability and efficiency. ACLA introduced a flexible mechanism for adjusting action preferences, making it adaptable to diverse environments. Empirical results showed that these methods outperformed Q(λ)-learning, SARSA(λ), and traditional actor-critic approaches, particularly in learning speed and final performance. However, further validation in dynamic real-world applications would be necessary to assess their full potential in complex and partially observable domains.

N-step Bootstrapping

Having discussed TD(0)-replay and TD(λ), we now have a strong foundation for exploring bootstrapping methods. N-step bootstrapping extends the concept of value function updates by incorporating multiple steps rather than relying solely on immediate rewards or the value of the next state. This method strikes a balance between MC approaches, which defer updates until the end of an episode, and one-step TD methods, which update values immediately based on the next state. In N-step bootstrapping, the value function update is determined by *n subsequent rewards* and the estimated value of the state following these n steps. A key advantage of this approach is its potential to accelerate

ARTICLE IN PRESS

learning while reducing variance in updates compared to one-step methods [1, 4, 33]. The general update rule for the N-step method is given as:

$$V(S_t) \leftarrow V(S_t) + \alpha \left[\sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n V(S_{t+n}) - V(S_t) \right] \tag{15}$$

where R_{t+k+1} are the rewards received after taking action A_t from state S_t, γ is the discount factor, and α is the learning rate.

N-step TD Prediction

As previously discussed, N-step bootstrapping serves as the foundation for various adaptations, including N-step TD and N-step SARSA. N-step methods ($n \neq 1$) differ by incorporating multiple future steps, allowing for flexibility based on specific requirements. This approach contrasts with MC methods, which update state values based on the full sequence of observed rewards until the episode concludes, and one-step TD methods, which rely solely on the next reward and use the value of the state one step ahead as an estimate for future returns [135]. This distinction can be bridged by updating values based on an intermediate number of rewards—more than one, but fewer than the total number of future rewards in the episode. For instance, a two-step update incorporates the first two rewards along with the estimated value of the state two steps ahead. This concept extends naturally to three-step, four-step, and higher-order updates, progressively incorporating additional rewards and future state values.

Alg. 12 gives a general overview of the N-step TD prediction algorithm. For a given state S_t at time t, the update rule for the value function V in N-step TD prediction is (line 21):

$$V(S_t) \leftarrow V(S_t) + \alpha(G_t^{(n)} - V(S_t)) \tag{16}$$

where α is the learning rate, and $G_t^{(n)}$ is derived as follows (line 18):

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}) \tag{17}$$

Next, we review research studies that have implemented these methods, evaluating their advantages and limitations.

In [35], the authors introduced an advanced generalization of TD learning by incorporating networks of interrelated predictions. This method extended traditional TD techniques by linking multiple predictions over time, enabling a more structured approach to learning and representation. TD networks facilitated the learning of interconnected predictions, significantly expanding the range of representable and learnable functions. This advancement proved especially beneficial in solving problems that standard TD methods struggled with, as demonstrated through experiments on the random-walk problem and predictive state representations. Experimental results indicated that TD networks learned complex prediction tasks more efficiently than MC methods, particularly in terms of data efficiency and learning speed. The practical advantages of TD networks were evident in environments requiring specific sequences of actions. While the study focused on small-scale experiments, the results suggested that TD networks had considerable potential for broader applications.

In [33], the authors introduced an advanced multi-step TD learning technique that utilized per-decision control variates to reduce variance in updates. This approach significantly improved the performance of multi-step TD algorithms, particularly in off-policy learning settings, by enhancing stability and convergence speed. Empirical results from tasks such as 5x5 Grid World and Mountain Car demonstrated that N-step SARSA outperformed N-step Expected SARSA by reducing root-mean-square error and improving learning efficiency. The method was also compatible with function approximation, reinforcing its practicality in complex environments.

Researchers in [136] developed a unified framework to analyze the finite-sample convergence guarantees of various value-based asynchronous RL algorithms. By reformulating these algorithms as Markovian stochastic approximation methods and applying Lyapunov analysis, the authors derived mean-square error bounds for their convergence. This framework systematically addressed convergence properties in algorithms such as Q-learning, N-step TD, TD(λ), and off-policy TD algorithms like V-trace. The study effectively tackled challenges related to asynchronous updates and provided robust convergence guarantees using detailed finite-sample mean-square bounds.

ARTICLE IN PRESS

Algorithm 12 N-step TD for Estimating $V \approx v_\pi$ [1]

Input: a policy π
Algorithm parameters: step size $\alpha \in (0, 1]$, a positive integer n
Initialize $V(s)$ arbitrarily for all $s \in S$
All store and access operations (for S_t and R_t) use index mod $n + 1$
repeat ▷ Loop for each episode
  Initialize and store S_0 such that $S_0 \neq$ terminal
  $T \leftarrow \infty$
  for $t = 0, 1, 2, \ldots$ **do**
    if $t < T$ **then**
      Take an action according to $\pi(\cdot|S_t)$
      Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
      if S_{t+1} is terminal **then**
        $T \leftarrow t + 1$
      end if
    end if
    $\tau \leftarrow t - n + 1$ ▷ τ is the time whose state's estimate is being updated
    if $\tau \geq 0$ **then**
      $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
      if $\tau + n < T$ **then**
        $G \leftarrow G + \gamma^n V(S_{\tau+n})$
      end if
      $V(S_\tau) \leftarrow V(S_\tau) + \alpha\,[G - V(S_\tau)]$
    end if
  end for
until $\tau = T - 1$

In [137], the authors investigated the deadly triad in RL—off-policy learning, bootstrapping, and function approximation—through a rigorous theoretical study of multi-step TD learning. Their work established a solid theoretical foundation for understanding these algorithms in off-policy settings with linear function approximation. A key contribution was the introduction of model-based deterministic counterparts to multi-step TD learning algorithms, improving the robustness of the analysis. The findings demonstrated that multi-step TD learning algorithms could converge to meaningful solutions when the sampling horizon n was sufficiently large, mitigating divergence issues observed in certain conditions.

In [138], the authors explored multiple facets of TD learning, particularly focusing on multi-step methods. The study introduced the use of control variates in multi-step TD learning to reduce variance in return estimates, thereby improving both learning speed and accuracy. The work also presented a unified framework for multi-step TD methods, extending the N-step Q(σ) algorithm and proposing novel variations, including N-step CV Q(σ) and Q(σ, λ). This unification clarified the relationships between different multi-step TD algorithms and their variants. Additionally, the study extended predictive knowledge representation into the frequency domain, allowing TD learning agents to detect periodic structures in return sequences, leading to a richer understanding of the environment.

In [36], the authors introduced Undelayed N-step TD prediction (TD-P), which integrated eligibility traces, value function approximators, and environmental models. This method employed Neural Networks (NNs) to predict future steps within a learning episode, combining RL techniques to enhance learning efficiency and performance. By incorporating a forward-looking mechanism, the TD-P method aimed to gather additional information that backward-looking eligibility traces might overlook, leading to more precise value function updates and improved decision-making. The TD-P method was particularly well-suited for partially observable environments, where it leveraged NNs to handle continuous and complex state-action spaces effectively.

N-step Off-policy Learning

N-step off-policy learning is an advanced technique that integrates N-step returns with off-policy updates. This method enhances learning stability and performance by utilizing multi-step returns while allowing data collection from a behavior policy b that differs from the target policy π, which

ARTICLE IN PRESS

Table 5: TD Papers Review

Application Domain	References
General RL (Policy learning, raw experience)	[130], [4]
Games (Chess)	[131]
Theoretical Research (Convergence, stability)	[132], [133], [136], [137], [138], [139]
Dynamic Environments (Mazes, Mountain Car, Atari)	[33]
Partially Observable Environments (Predictions)	[134],[35], [36]
Benchmark Tasks (Mountain Car, Acrobot, GCRL)	[107], [44]

is being improved [1]. Unlike one-step methods, multi-step returns aggregate rewards over multiple steps, providing a broader perspective on future rewards and yielding more precise value estimates. Importance sampling is employed to correct discrepancies between the behavior policy and the target policy. By adjusting updates with importance sampling ratios, the method accounts for differences in action probabilities between the two policies, ensuring accurate value estimation in off-policy settings.

The N-step off-policy return, $G_t^{(n)}$, can be calculated in the following way:

$$G_t^{(n)} = R_{t+1} + \sum_{k=1}^{n-1} \gamma^k \left(\prod_{i=0}^{k-1} \rho_{t+i} \right) R_{t+k+1} + \left(\prod_{i=0}^{n-1} \rho_{t+i} \right) \gamma^n V(S_{t+n}) \tag{18}$$

By explicitly separating the immediate reward term (R_{t+1}) for $k = 0$, we avoid the notational ambiguity of an empty product, ensuring the formulation is mathematically rigorous and straightforward to interpret. Here, $\rho_{t+i} = \frac{\pi(A_{t+i}|S_{t+i})}{b(A_{t+i}|S_{t+i})}$ is the importance sampling ratio, with π as the target policy and b the behavior policy [3].

With a solid foundation in place, we now conduct a detailed analysis of research papers to examine N-step off-policy methods.

In [139], the authors introduced the Greedy Multi-step Value Iteration (GM-VI) algorithm, which approximates the optimal value function using a novel multi-step bootstrapping approach. The method dynamically adjusts the step size along each trajectory based on a greedy principle, effectively balancing information propagation and estimation accuracy. The adaptive step-size mechanism modifies step sizes according to the quality of trajectory data, improving learning efficiency and robustness. Unlike traditional off-policy methods, GM-VI learns safely from arbitrary behavior policies without requiring importance sampling corrections, simplifying the algorithm and reducing variance. Theoretical analysis demonstrated that GM-VI converged more rapidly to the optimal value function compared to one-step Bellman optimality operators. Empirical results on standard RL benchmarks, such as Mountain Car and Acrobot, confirmed that GM-VI achieved state-of-the-art sample efficiency and superior reward performance.

The core contribution of [140] was its elimination of importance sampling ratios in multi-step TD learning. By removing these ratios, the method reduced estimation variance, leading to improved stability and efficiency in off-policy learning. The introduction of action-dependent bootstrapping parameters further enhanced adaptability across different state-action pairs, refining update precision and reducing variance. Empirical evaluations on complex off-policy tasks demonstrated the algorithm's stability and superior performance relative to state-of-the-art methods, highlighting its practical applicability in real-world learning environments.

In [44], the authors proposed bias management techniques for multi-step Goal-Conditioned RL (GCRL), categorizing and mitigating shooting and shifting biases. This strategy enabled the use of larger step sizes, improving learning efficiency and overall performance. The proposed approach was validated across various tasks, consistently outperforming baseline multi-step GCRL benchmarks. The integration of quantile regression for bias handling further demonstrated the practical effectiveness of the approach. Additionally, the introduction of resilient bias-management strategies ensured robust learning improvements, with empirical results confirming its success across diverse environments.

Table 5 provides an overview of the TD-based research studies.

In the following discussion, we provide a detailed analysis of Q-learning, one of the most widely utilized algorithms in RL.

ARTICLE IN PRESS

Algorithm 13 Q-learning [1]

1: Initialize $Q(s,a)$, $\forall s \in S, a \in A(s)$, arbitrarily, and $Q(\text{terminal-state},\cdot) = 0$
2: **repeat** ▷ For each episode
3: Initialize S
4: **repeat** ▷ For each step of episode
5: Choose A from S using policy derived from Q (e.g., ϵ-greedy)
6: Take action A, observe R, S'
7: $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$
8: $S \leftarrow S'$
9: **until** S is terminal
10: **until** convergence or a stopping criterion is met

Q-learning

Building upon TD(0), a significant breakthrough in RL was introduced by [5] with Q-learning, a model-free algorithm classified as off-policy TD control. Q-learning allows an agent to estimate the value of an action in a given state through direct experience, without requiring a model of the environment. It is based on learning an action-value function, which represents the expected utility of selecting a particular action in each state and subsequently following a fixed policy. A general overview of the Q-learning algorithm is presented in Alg. 13. At its core, Q-learning updates Q-values (state-action value pairs), where the learned function Q approximates q_*, the optimal action-value function, independently of the policy being followed. This independence simplifies the algorithm's analysis and has facilitated early convergence proofs. However, the policy still influences the learning process by determining which state-action pairs are visited and subsequently updated (lines 4-9).

$$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \alpha\big[R_{t+1} + \gamma \max_a Q(S_{t+1},a) - Q(S_t,A_t)\big] \tag{19}$$

With a foundational understanding of Q-learning established, we now turn our attention to examining research papers that have applied this algorithm in various contexts. Q-learning, being fundamental and relatively straightforward, has been extensively applied across numerous studies. Here, we briefly touch upon a variety of notable studies.

Researchers in [141] investigated the performance differences between deterministic and stochastic policies within a grid-world problem using Q-learning. The authors developed a flexible agent capable of operating under both policy types to determine which parameters maximized cumulative reward. Their results indicated the superiority of deterministic policies in achieving higher rewards in a structured task environment. The study's strength lies in its clear methodological execution, systematically exploring the impact of policy variations on learning outcomes. However, the study was confined to a simulated grid world, which may not fully capture the complexities of real-world environments, potentially reducing the generalizability of the findings. Additionally, the paper focused on policy optimization without significant consideration of the computational costs associated with each policy type.

In [142], the authors explored the development of a hardware architecture optimized for Q-learning, focusing on real-time applications. Key innovations included low power usage, high throughput, and minimal use of hardware resources. The implementation, tested on an evaluation kit, demonstrated improved performance metrics such as speed, power consumption, and hardware resource use compared to existing Q-learning hardware accelerators. While the study presented a comprehensive approach to optimizing Q-learning for hardware implementation, making it suitable for real-time and Internet of Things (IoT) applications, it was limited by the specific hardware used for testing and the types of environments evaluated. The generalizability of the findings to other hardware or more complex real-world applications was not fully explored.

The application of Q-learning to improve power allocation in Wireless Body Area Networks (WBANs) was explored in [38]. The focus was on enhancing energy efficiency while maintaining effective communication within the network, particularly through controlling transmission power, reducing interference, and optimizing routing paths. The paper addressed critical aspects of WBANs that impact their practical deployment, especially in healthcare settings. While it presented a robust approach to managing interference and power consumption, the paper did not discuss the computational overhead introduced by the Q-learning algorithm and game theory applications, which was crucial for feasibility in devices with limited computational capabilities.

ARTICLE IN PRESS

In [143], authors explored improvements to the Q-learning algorithm's efficiency using massively parallel computing techniques, such as multi-threading and GPU computing. The paper addressed the issue of slow convergence in traditional Q-learning, particularly in complex, real-world system control problems that demanded quick adaptation to dynamic environments. By leveraging GPUs and multi-threading, the authors significantly decreased convergence time, which was vital for real-time applications like robotics and industrial control. However, the paper did not thoroughly examine how parallel processing affected the Q-learning algorithm's integrity and reliability, particularly under different computational loads.

Researchers in [40] investigated the use of Q-learning to automate portfolio rebalancing. The research applied basic Q-learning agents to discern trading patterns across 15 Indian financial assets using technical indicators. The paper innovatively harnessed Q-learning to improve portfolio rebalancing, a domain where traditional rule-based systems might not perform optimally. However, the simplicity of the Q-learning models used might not have fully captured the intricate dependencies and dynamics of highly fluctuating markets.

Authors in [144] extended traditional Q-learning to handle POMDPs by incorporating L_0 regularization to manage the complexity of the state representation derived from the agent's history. This innovative approach transformed non-Markov and partially observable environments into a history-based RL framework, allowing Q-learning to be applicable in more complex scenarios. While the method showed improved computational and memory efficiency, the reliance on L_0 regularization could have made the learning process highly dependent on the initial choice of features, potentially limiting adaptability in dynamic environments.

An innovative approach to enhancing the performance of Proportional Integral Derivative (PID) controllers for magnetic levitation train systems through the integration of Q-learning was introduced in [145]. This method adapts PID parameters in real time, maintaining optimal levitation control despite non-linearities and uncertainties. While it showed improved performance over traditional PID controllers, the requirement for continuous learning and adjustment could be computationally intensive.

A novel application of Deep Q-learning (discussed in subsection 3.3.1) to optimize parameter settings in Hadoop, improving its efficiency by iteratively adjusting configurations based on feedback from performance metrics introduced in [146]. This approach reduced the manual effort and expertise required for parameter tuning and demonstrated marked improvements in processing speeds. However, the model's simplification of the parameter space might have overlooked interactions that could achieve further optimization, and the scalability and adaptability in varying operational environments remained somewhat uncertain.

Researchers in [45] presented a novel approach to enhancing the accuracy of nuclei segmentation in pathological images using Q-learning and Deep Q-Network (DQN) algorithms. The study reported improvements in Intersection over Union for segmentation, critical in cancer diagnosis. While the approach adapted the segmentation threshold dynamically, leading to better outcomes, the reliance on RL introduced computational complexity and required significant training data.

The sampling efficiency of Q-learning was discussed in [147], exploring the feasibility of making model-free algorithms as sample-efficient as model-based counterparts. The research introduced a variant of Q-learning that incorporated UCB exploration, achieving a regret bound that approached the best possible by model-based methods. This paper marked a significant theoretical advance by providing rigorous proof that Q-learning could achieve sublinear regret in episodic MDPs without requiring a simulator. However, the analysis relied on assumptions such as an episodic structure and precise model knowledge, which might not have translated well to more uncertain environments.

Researchers in [148] examined the deployment of Q-learning on Field Programmable Gate Arrays (FPGAs) to enhance processing speed by leveraging parallel computing capabilities. The study demonstrated substantial acceleration of the Q-learning process, making it suitable for real-time scenarios. However, the focus on a particular FPGA model might have restricted the broader applicability of the findings to different hardware platforms.

The challenge of managing frequent handovers in high-speed railway systems using a Q-learning-based approach was addressed in [149]. The authors proposed a scheme that minimized unnecessary handovers and enhanced network performance by dynamically adjusting to changes in the network environment. While the approach showed promise, its complexity, computational demands, and need for real-time processing presented challenges for practical implementation.

ARTICLE IN PRESS

Table 6: Q-learning Paper Review by Domain

Application Domain	References
General RL (Policy learning, raw experience)	[141], [147]
Games and Simulations (Chess, StarCraft)	[144], [155]
Theoretical Research (Convergence, Stability)	[132], [5]
Dynamic and Complex Environments (Mazes, Mountain Car, Atari)	[47], [134]
Partially Observable Environments (Predictions, POMDPs)	[156]
Real-time Systems and Hardware Implementations (FPGA, Real-time applications)	[142], [38], [148]
Energy Efficiency and Power Management (IoT, WBAN, PID controllers)	[153]
Financial Applications (Portfolio Rebalancing)	[40]
Data Management and Processing (Hadoop, Pathological Images)	[146], [45]
Network Optimization (Optical Transport Networks, Fog RAN)	[151], [154]
Transportation Systems (Railway, Taxi, Electric Vehicles)	[145], [149]
Autonomous Systems and Robotics (Learning, Routing)	[150], [152]

In [150] authors explored optimizing taxi dispatching and routing using Q-learning and MDPs to enhance taxi service efficiency in urban environments. The method enabled dynamic adjustments based on real-time conditions, potentially alleviating urban traffic congestion. However, the model's effectiveness heavily depended on the accuracy and comprehensiveness of input data, and the computational complexity might have posed challenges for real-time deployment.

The study [151] addressed the optimization of network routing within optical transport networks using a Q-learning-based algorithm under the software-defined networking framework. The approach improved network capacity and efficiency by managing routes based on learned network conditions. While the method outperformed traditional routing strategies, its scalability and computational complexity in real-world scenarios remained areas for further exploration.

In [152], the authors proposed a Q-learning algorithm motivated by internal stimuli, specifically visual novelty, to enhance autonomous learning in robots. The method blended Q-learning with a cognitive developmental approach, making the robot's learning process more dynamic and responsive to new stimuli. While the approach reduced computational costs and enhanced adaptability, its scalability in complex environments was not thoroughly explored.

The use of Q-learning and double Q-learning to manage the duty cycles of IoT devices in environmental monitoring was analyzed in [153]. The study demonstrated significant advancements in optimizing IoT device operations, enhancing energy efficiency, and operational effectiveness. However, the performance heavily depended on environmental parameters, and the increased memory requirements for double Q-learning might have posed challenges for memory-constrained IoT devices.

The study [154] presented a robust implementation of RL to address resource allocation challenges in Fog Radio Access Networks (Fog RAN) for IoTs. The use of Q-learning is effectively adapted to changes in network conditions, improving network performance and reducing latency. While the approach showed promise, the reliance on Q-learning might not have fully captured the complexities of larger-scale IoT networks, and the exploration-exploitation trade-off could have led to suboptimal performance in highly dynamic environments.

The overview of the reviewed papers categorized by their respective domains is presented in Table 6.

Double Q-learning

Double Q-learning, introduced by [157], enhances traditional Q-learning by mitigating overestimation bias, which can be problematic in environments with stochastic rewards. The overestimation arises because Q-learning uses the maximum action value as an approximation for the maximum expected action value [158]. To address this, as illustrated in Alg. 14, double Q-learning maintains two separate estimators (Q-tables), denoted as Q_A and Q_B (line 1). Each estimator is updated independently using the maximum value from the other estimator, thereby reducing the overestimation bias that standard Q-learning exhibits (lines 3-10).

The core update rule for double Q-learning involves: Selecting an action a based on the average of Q_A and Q_B. Updating one of the Q-functions at random with equal probability (0.5), for instance, updating Q_A using (line 7):

ARTICLE IN PRESS

Algorithm 14 Double Q-learning [157]

1: Initialize Q^A, Q^B, s
2: **repeat**
3: Choose a based on $Q^A(s,\cdot)$ and $Q^B(s,\cdot)$, observe r, s'
4: Choose (e.g., random) either UPDATE(A) or UPDATE(B)
5: **if** UPDATE(A) **then**
6: Define $a^* = \arg\max_a Q^A(s',a)$
7: $Q^A(s,a) \leftarrow Q^A(s,a) + \alpha(s,a)\left(r + \gamma Q^B(s',a^*) - Q^A(s,a)\right)$
8: **else if** UPDATE(B) **then**
9: Define $b^* = \arg\max_a Q^B(s',a)$
10: $Q^B(s,a) \leftarrow Q^B(s,a) + \alpha(s,a)\left(r + \gamma Q^A(s',b^*) - Q^B(s,a)\right)$
11: **end if**
12: $s \leftarrow s'$
13: **until** end

$$Q_A(s,a) \leftarrow Q_A(s,a) + \alpha\Big(r + \gamma Q_B\left(s', \arg\max_a Q_A(s',a)\right) - Q_A(s,a)\Big) \tag{20}$$

where s' is the next state in the environment. Alternatively, we shall update Q_B similarly using (line 10):

$$Q_B(s,a) \leftarrow Q_B(s,a) + \alpha\Big(r + \gamma Q_A\left(s', \arg\max_a Q_B(s',a)\right) - Q_B(s,a)\Big) \tag{21}$$

Essentially, this approach alternates updates between Q_A and Q_B, using the maximum action value from the other table. This strategy is considered to provide a less biased estimate of the underlying value function [158]. By occasionally underestimating the maximum expected values, this method mitigates the positive bias inherent in Q-learning, aiming for a balance that better approximates true expectations.

In the following paragraphs, we explore a selection of research papers that have applied double Q-learning.

The focus of [159] was on the FPGA-based implementation of the double Q-learning algorithm, emphasizing its efficiency in stochastic environments and its superiority over standard Q-learning due to reduced overestimation biases. This implementation of FPGA allowed for the parallel processing of actions, significantly speeding up the learning process. This was particularly beneficial for applications requiring real-time decision-making. Additionally, the use of asynchronous reading and synchronous writing memory architectures optimized data exchange and reduced the hardware footprint, which was a critical advancement in hardware implementations of RL algorithms. However, while the hardware implementation was efficient for the stated range of states (from 8 to 256 states), the paper did not extensively discuss scalability beyond this range, which might be a limitation for environments with a much larger state space. The paper focused on the hardware aspect without a detailed discussion of how the implementation could be adapted to different RL scenarios or environments, which may have limited its applicability without additional modifications or tuning.

The integration of A* pathfinding with double Q-learning to optimize route planning in autonomous driving systems was investigated in [160]. This innovative approach aimed to enhance route efficiency and safety by minimizing the common problem of action value overestimations found in standard Q-learning. The paper introduced a novel combination of A* pathfinding with RL, providing a dual strategy that leveraged the deterministic benefits of A* for initial path planning and the adaptive strengths of the proposed method for real-time adjustments to dynamic conditions, such as unexpected obstacles. This synergy allowed for a balanced approach to navigating real-world driving scenarios efficiently. By employing double Q-learning, the system addressed and mitigated the issue of action value overestimation, which was prevalent in RL. This enhancement was crucial for autonomous driving applications where decisions had to be both accurate and dependable to ensure safety and operational reliability. On the other hand, this combination, while robust, introduced a significant computational demand that might have impacted the system's performance in real-time scenarios. Quick decision-making was essential in dynamic driving environments, and the increased computational load could have hindered the system's ability to respond promptly. Additionally, the promising method introduced lacked a detailed exploration of how this hybrid model

ARTICLE IN PRESS

performed across different environmental conditions or traffic scenarios. This limitation might have affected the model's effectiveness in diverse settings without further adaptation or refinement.

Researchers in [161] delved into optimizing IoT devices' power management by employing a double Q-learning-based controller. This Double Data-Driven Self-Learning (DDDSL) controller dynamically adjusted operational duty cycles, leveraging predictive data analytics to enhance power efficiency significantly. A notable strength of the paper was the improved operational efficiency introduced by the double Q-learning, which effectively handled the overestimation issues found in standard Q-learning within stochastic environments. This led to more precise power management decisions, crucial for prolonging battery life and minimizing energy usage in IoT devices. Furthermore, the DDDSL controller showed a marked performance enhancement, outperforming traditional fixed duty cycle controllers by 42–50%, and the previous data-driven self-learning model by 2–12%. However, while the performance improvements were compelling, they were obtained under specific conditions, which might have limited the broader applicability of the findings without further adaptations or validations for different operational environments or IoT device configurations.

A double Q-learning-based routing protocol for optimizing maritime network routing, crucial for effective communication in maritime search and rescue operations, was developed in [162]. The use of double Q-learning in this context aimed to tackle the overestimation problems inherent in Q-learning protocols, which was a significant improvement in maintaining stability in the model's predictions and actions. This approach not only enhanced the routing efficiency but also incorporated a trust management system to ensure the reliability of data transfers and safeguard against packet-dropping attacks. The protocol demonstrated robust performance in various simulated attack scenarios with efficient energy consumption and minimal resource footprint, crucial for the resource-constrained environments in which maritime operations occurred. While the proposed method showed promising results in simulations, the complexity of real-world application scenarios could have posed challenges. Maritime environments were highly dynamic with numerous unpredictable elements, which might have affected the consistency of the performance gains observed in controlled simulations. Additionally, the scalability of this approach when applied to very large-scale networks or under extreme conditions typical of maritime emergencies could have required further validation. The integration of such sophisticated systems also raised concerns about the computational overhead and the practical deployment in existing maritime communication infrastructures.

Authors in [163] explored the application of double Q-learning to manage adaptive wavelet compression in environmental sensor networks. This method aimed to optimize data transmission efficiency by dynamically adjusting compression levels based on real-time communication bandwidth availability. A key advantage of this approach was its high adaptability, which ensured efficient bandwidth utilization and minimized data loss even under fluctuating network conditions, critical for remote environmental monitoring stations where connectivity might have been inconsistent. Furthermore, this integration helped significantly reduce the risk of overestimating action values, a common problem in Q-learning, which could have led to suboptimal compression settings. However, the complexity of this implementation posed a notable challenge, particularly in environments where computational resources were limited. The necessity for managing two separate Q-values for each action increased the computational demand, potentially impacting the system's real-time response capabilities. Additionally, the performance of the algorithm heavily depended on the accuracy of the network condition assessments, with inaccuracies potentially leading to inefficient compression and data transmission.

In [164], authors explored the application of double Q-learning to enhance dynamic voltage and frequency scaling in multi-core real-time systems. Their approach addressed the inherent overestimation bias found in Q-learning, aiming to provide more accurate and reliable power management. A major strength of their work lies in the innovative use of double Q-learning to mitigate the overestimation issue common in single-estimator Q-learning methods. By utilizing two estimators, the system could potentially make more informed decisions that better balance power consumption against system performance needs. Additionally, their thorough simulation-based evaluation indicated that this method could outperform traditional methods, offering substantial energy savings across various system conditions. Nevertheless, managing two Q-value estimators increased the computational load, which could have challenged the limited resources available in real-time systems where rapid processing was paramount. Furthermore, while the simulation results were decent, translating these outcomes to real-world scenarios required additional adjustments to accommodate the diverse nature and unpredictability of real system workloads.

Table 7 summarizes the examined papers by their domain.

ARTICLE IN PRESS

Table 7: Double Q-learning Papers Review

Application Domain	References
Hardware Implementations (FPGA, Real-Time Systems)	[159], [164]
IoT and Power Management (IoT Devices, Maritime Network Routing)	[161], [162]
Data Transmission and Compression (Environmental Monitoring)	[163]

Algorithm 15 SARSA [1]

1: Initialize $Q(s, a)$, $\forall s \in S, a \in A(s)$, arbitrarily, and $Q(\text{terminal-state}, \cdot) = 0$
2: **repeat** ▷ For each episode
3: Initialize S
4: Choose A from S using policy derived from Q (e.g., ϵ-greedy)
5: **repeat** ▷ (for each step of the episode)
6: Take action A, observe R, S'
7: Choose A' from S' using policy derived from Q (e.g., ϵ-greedy)
8: $Q(S, A) \leftarrow Q(S, A) + \alpha\left[R + \gamma Q(S', A') - Q(S, A)\right]$
9: $S \leftarrow S'$; $A \leftarrow A'$
10: **until** S is terminal
11: **until** convergence or a stopping criterion is met

Having examined the off-policy TD algorithm Q-learning and its variant, double Q-learning, which employs two Q functions, we now shift our focus to another TD-based algorithm, SARSA, in the next subsection.

State-Action-Reward-State-Action (SARSA)

The SARSA algorithm is an on-policy method that updates the action-value function (Q-function) incrementally and in real time. Introduced in [66], SARSA differs from other approaches by learning Q-values directly from the policy being executed. Unlike off-policy methods such as Q-learning, which can learn the optimal policy regardless of the agent's actions, SARSA updates values based on the policy currently being followed. The core mechanism of SARSA lies in its Q-value update rule, which follows the equation defined in [1] and is implemented in Algorithm 15, line 8:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)] \tag{22}$$

Both SARSA and Q-learning update their estimates of the action-value function based on the agent's actions. However, the key distinction lies in how they update these values. Q-learning updates based on the maximum estimated future reward, using the highest Q-value of the next state, irrespective of the action taken. In contrast, SARSA updates its Q-values using the actual action taken in the next state, rather than selecting the best possible action. In other words, SARSA does not wait to compute the maximum Q-value of the next action but directly incorporates the next available action into its update process.

Next, we examine several research studies that have applied the SARSA algorithm.

The application of the SARSA algorithm in a simulated shepherding scenario where a dog herds sheep towards a target was investigated in [48]. This RL model incorporated a discretized state and action space and designed a specific reward system to facilitate learning. The application of SARSA to a complex, dynamic task like shepherding demonstrated the algorithm's versatility and its potential in environments involving multiple agents and stochastic elements. The model successfully taught a dog to herd sheep by learning to reach sub-goals, which simplified the learning process and improved the manageability of the task. However, the discretization of the state and action spaces might have limited the dog's movement and decision-making capabilities, potentially leading to less optimal performance in more realistic or varied environments. The stochastic nature of sheep movement and the complexity of the shepherding task led to a significant learning time, with notable success only after many episodes, indicating potential efficiency issues in more demanding or time-sensitive applications.

Authors in [155] primarily focused on the implementation of Q-learning and SARSA algorithms, incorporating eligibility traces to improve handling delayed rewards. The research demonstrated the

ARTICLE IN PRESS

effectiveness of RL algorithms in navigating dynamic tasks like micro-managing combat units in real-time strategy games. Adding eligibility traces significantly boosted the algorithms' learning from sequences of interdependent actions, essential in fast-paced, chaotic environments. Using a commercial game like StarCraft as a testbed introduced real-world complexity often absent in simulated settings. This method confirmed the practicality of RL algorithms in real scenarios and highlighted their potential to adapt to commercial applications. The algorithms showed promise in small-scale combat, but scaling to larger, more complex battles in StarCraft or similar games remained uncertain. Concerns arose about computational demands and the efficiency of learning optimal strategies without extensive prior training. While effective within StarCraft, their broader applicability to other real-time strategy games or applications remained untested. The specialized design of state and action spaces for StarCraft could have hindered transferring these methods to different domains without significant modifications.

In [28], the authors studied the enhancement of energy efficiency in IoT networks through strategic power and channel resource allocation using a SARSA-based algorithm. This approach addressed the unpredictable nature of energy from renewable sources and the variability of wireless channels in real-time settings. A major contribution of the study lies in its innovative use of a model-free, on-policy RL method to manage energy distribution across IoT nodes. This method efficiently handled the stochastic nature of energy harvesting and channel conditions, optimizing network performance in terms of energy efficiency and longevity. Integrating SARSA with linear function approximation helped refine solutions to continuous state and action spaces, enhancing the practicality in real-world IoT applications. However, relying on linear function approximation introduced limitations in capturing the full complexity of interactions in dynamic, multi-dimensional state spaces typical of IoT environments. While the proposed SARSA algorithm demonstrated network efficiency improvements, implementing such an RL system in real IoT networks posed challenges. These included the need for continual learning and adaptation to changing conditions, impacting the deployment's feasibility and scalability.

Authors in [165] explored applying SARSA to optimize peer-to-peer (P2P) electricity transactions among small-scale users in smart energy systems. This research aimed to enhance economic efficiency and reliability in decentralized energy markets. A significant strength lies in its innovative application of SARSA to a complex, dynamic energy trading system. Researchers modeled P2P electricity transactions as an MDP, allowing the system to handle uncertainties in small-scale energy trading, like fluctuating prices and varying demands. This modeling enabled the system to learn optimal transaction strategies over time, potentially enhancing both efficiency and profitability in energy trading. However, the approach had limitations. SARSA, while effective in learning optimal policies through trial and error, required extensive interaction with the environment to achieve satisfactory performance. This data requirement could have been a drawback in real-world applications where immediate decisions were necessary, and historical transaction data was limited. Moreover, implementing such a system in a live environment, where real-time decision-making was crucial, posed additional challenges, including the need for robust computational resources to handle continuous state and action space calculations.

In [47], they examined integrating SARSA(0) with Fully Homomorphic Encryption (FHE) for cloud-based control systems to ensure data confidentiality while performing RL computations on encrypted data. A significant strength was preserving privacy in cloud environments where sensitive control data could have been vulnerable to breaches. Using FHE allowed the RL algorithm to execute without decrypting the data, providing a robust method for maintaining confidentiality. The paper successfully demonstrated this method on a classical pole-balancing problem, showing it was theoretically sound and practically feasible. However, implementing SARSA(0) over FHE introduced challenges related to computational overhead and latency due to encryption operations. These factors could have impacted the efficiency and scalability of the RL system, especially in environments requiring real-time decision-making. Additionally, encryption-induced delays and managing encrypted computations might have limited this method's application to scenarios where control tasks could tolerate such delays.

Authors in [156] presented a novel application of SARSA within a swarm RL framework to solve optimization problems more efficiently, particularly those involving large negative rewards. The authors incorporated individual learning and cooperative information exchange among multiple agents, aiming to speed up learning and enhance decision-making efficiency. This approach significantly leveraged swarm intelligence and RL, particularly SARSA's ability to handle tasks with substantial negative rewards. By enabling agents to learn from individual experiences and shared

ARTICLE IN PRESS

Algorithm 16 Expected SARSA [167]

Initialize $Q(s, a)$ arbitrarily for all s, a
repeat ▷ over episodes
  Initialize s
  repeat ▷ for each step in the episode
    choose a from s using policy π derived from Q
    take action a, observe r and s'
    $V_{s'} = \sum_{a'} \pi(a'|s') \cdot Q(s', a')$
    $Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma V_{s'} - Q(s, a) \right]$
    $s \leftarrow s'$
  until s is terminal
until end of episodes

insights, the system could converge to optimal policies more swiftly than traditional single-agent or non-cooperative multi-agent systems. Implementing the shortest path problem demonstrated the method's practicality and effectiveness, showcasing improved learning speeds and robustness against pitfalls with significant penalties. Nonetheless, managing multiple agents and their interactions increased the computational overhead and complexity of the learning process. Moreover, the approach heavily relied on designing the information-sharing protocol among agents, which, if not optimized, could have led to inefficiencies or suboptimal learning outcomes. The generalized application of this method across different environments or RL tasks remains to be thoroughly tested, suggesting potential limitations in adaptability.

In [166], the authors explored the application of SARSA to optimize routing for Electric Vehicles (EVs) to minimize energy consumption. This approach is adapted to real-time driving conditions to reduce on-road energy needs by selecting routes with lower energy requirements. The research's strength lies in its real-time application and use of SARSA to learn and predict the most energy-efficient routes under various conditions, like traffic and road type. By utilizing Markov chain models to estimate energy requirements based on actual driving data, the framework aimed to extend the driving range of EVs, crucial for reducing range anxiety. However, the reliance on real-time data and the inherent variability of driving conditions presented challenges. The accuracy of the SARSA model's predictions heavily depended on the data quality and immediacy, which could have been compromised by limitations in data transmission or processing delays. Implementing this system in a real-world environment could have posed scalability and adaptability challenges, particularly concerning integration with existing vehicle navigation systems and the computational demands on the vehicle's hardware.

A variation of SARSA that considers expectations over all possible next actions rather than relying solely on the sampled next action is introduced in the next subsection.

Expected SARSA

The Expected SARSA algorithm, introduced by [167], extends the classic SARSA algorithm by incorporating expectations over all possible next actions rather than relying solely on the sampled next action. This modification reduces the variance in the update rule, potentially leading to faster and more stable convergence in learning tasks. The key idea behind Expected SARSA is that averaging over all possible next actions, weighted by their probability under the current policy, provides a more stable estimate of state-action values. Expected SARSA ensures convergence under conditions similar to those required for SARSA, such as all state-action pairs being visited infinitely often. [167] formally proved that Expected SARSA converges to the optimal action-value function, assuming finite state and action spaces and a policy that becomes greedy in the limit with infinite exploration. Empirical results demonstrated that Expected SARSA outperformed both SARSA and Q-learning in various tasks, particularly those where certain actions could result in significant negative consequences. By integrating the expectation over all possible next actions, the algorithm smooths out learning updates, which is particularly advantageous in stochastic environments where certain actions or state transitions exhibit high variability in rewards [1, 167]. In summary, Expected SARSA provides a robust approach to RL by reducing the variance in state-action value updates, leading to more reliable learning performance, especially in complex and uncertain environments.

The expected SARSA update rule is given as follows:

ARTICLE IN PRESS

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \sum_a \pi(a|s_{t+1}) Q(s_{t+1}, a) - Q(s_t, a_t) \right] \tag{23}$$

Before reviewing studies that have applied Expected SARSA, a detailed overview of the algorithm is provided in Alg. 16.

Authors in [168] designed a comprehensive exploration of combining Model Predictive Control (MPC) with the Expected SARSA algorithm to tune MPC models' parameters. This integration aimed to enhance the robustness and efficiency of control systems, particularly in applications where the system model's parameters were not fully known or subject to change. A key advantage was the innovative approach to speeding up learning by directly integrating RL with MPC, reducing the episodes needed for effective training. This efficiency was achieved using the Expected SARSA algorithm, which offered smoother convergence and better performance due to its average-based update rule compared to the more common Q-learning method, which focused on maximum expected rewards and might lead to higher variance in updates. However, the approach's complexity and the need for precise tuning of MPC model parameters represented significant challenges. The computational demands increased due to the dual needs of continuous adaptation by the RL algorithm and the rigorous constraints enforced by MPC. While the framework showed potential in simulations, its real-world applicability, especially in highly dynamic and unpredictable environments, required further validation.

Authors in [169] focused on improving Deterministic and Synchronous Multichannel Extension (DSME) networks' resilience against WiFi interference using the Expected SARSA algorithm. It evaluated channel adaptation and hopping strategies to mitigate interference in industrial environments, providing a detailed analysis with Expected SARSA. The study stood out for its rigorous simulation-based evaluation of interference mitigation strategies in a controlled DSME network. By using Expected SARSA, the research effectively reduced uncertainty in channel quality assessment, leading to more reliable network performance under interference conditions, crucial for industrial applications where reliable data transmission was vital for operational efficiency and safety. However, the complexity of RL implementation and its dependency on accurate real-time data posed challenges. The computational demands of running Expected SARSA in real-time environments could limit the practical deployment of this strategy in resource-constrained settings.

In [170], researchers investigated using the Expected SARSA learning algorithm to manage Load Frequency Control in multi-area power systems. The study focused on enhancing power systems' stability integrated with distributed feed-in generation-based wind power, using a model-free RL approach to adjust to variable power supply conditions without a predefined system model. The primary strength lies in its innovative approach to addressing challenges in integrating renewable energy sources into the power grid. Expected SARSA, an on-policy RL algorithm, demonstrated how to manage and stabilize frequency variations due to unpredictable renewable energy outputs and fluctuating demand adaptively. This was crucial for maintaining reliability and efficiency in power grids, which are increasingly incorporating variable renewable energy sources. However, the approach's downside was related to the RL algorithm's computational demands and the need for extensive simulation and testing to fine-tune system parameters. Implementing such a system in a real-world setting could be constrained by these factors, particularly in terms of real-time computation capabilities and the scalability of the solution to larger, more complex grid systems.

Authors in [31] utilized the Expected SARSA algorithm to optimize Energy Storage Systems (ESS) operation in managing uncertainties in wind power generation forecasts. The study modeled the problem as an MDP where the state and action spaces were defined by the ESS's operational constraints. Expected SARSA's primary result in this context was its superior performance compared to conventional Q-learning-based methods. The algorithm effectively handled the wide variance in wind power forecasts, crucial for optimizing ESS's charging and discharging actions to reduce forecast errors. The strategy's effectiveness was underscored by its near-optimal performance, closely approximating the optimal solution with complete future information. Simulation results demonstrated that the Expected SARSA-based strategy could manage wind power forecast uncertainty more effectively by adapting to varying conditions. This adaptability was enhanced by including frequency-domain data clustering, which refined the learning process and reduced input data variability, further improving the RL model's performance. Lastly, we examine the last variant of SARSA, namely N-step SARSA.

ARTICLE IN PRESS

N-step SARSA

With foundational concepts of n-step TD and bootstrapping established, we can expand on these ideas and explore research papers utilizing them. First, we explore N-step SARSA, an enhancement of the conventional one-step SARSA within the on-policy learning framework. In N-step SARSA, as shown in Alg. 17, the update to the value (or action-value) is not limited to just the next state and action, as seen in one-step SARSA, but instead incorporates a sequence of n actions and rewards. The value function in the equation is replaced with $Q(S_{t+n}, A_{t+n})$:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[\sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n Q(S_{t+n}, A_{t+n}) - Q(S_t, A_t) \right] \tag{24}$$

N-step SARSA retains the on-policy nature of SARSA, allowing it to effectively combine the advantages of TD learning with the broader horizon of MC methods. By adjusting the number of steps n ahead, it balances the bias-variance trade-off. N-step methods, including N-step SARSA, improve learning by incorporating multiple future outcomes rather than depending solely on the immediate next state and action. This leads to faster learning and enhances policy performance, particularly in complex environments where future rewards depend heavily on actions taken over multiple steps [1, 135].

Algorithm 17 N-step SARSA [1]

Initialize $Q(s, a)$ arbitrarily, for all $s \in S, a \in A$
Initialize π to be ϵ-greedy with respect to Q, or to a fixed given policy
Algorithm parameters: step size $\alpha \in (0, 1]$, small $\epsilon > 0$, a positive integer n
All store and access operations (for S_t, A_t, R_t) can take their index mod $n + 1$
repeat ▷ For each episode
    Initialize and store $S_0 \neq$ terminal
    Select and store an action $A_0 \sim \pi(\cdot|S_0)$
    $T \leftarrow \infty$
    for $t = 0, 1, 2, \ldots$ **do**
        if $t < T$: **then**
            Take action A_t
            Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
            if S_{t+1} is terminal **then**
                $T \leftarrow t + 1$
            else
                Select and store an action $A_{t+1} \sim \pi(\cdot|S_{t+1})$
            end if
        end if
        $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)
        if $\tau \geq 0$: **then**
            $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
            if $\tau + n < T$: **then**
                $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ ($G_{\tau:\tau+n}$)
            end if
            $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha [G - Q(S_\tau, A_\tau)]$
            if π is being learned **then**
                Ensure that $\pi(\cdot|S_\tau)$ is ϵ-greedy with respect to Q
            end if
        end if
    end for
until $\tau = T - 1$

In [171], the N-step SARSA algorithm was applied to optimize traffic signal control to dynamically adjust signals based on real-time conditions to reduce congestion more effectively than traditional

ARTICLE IN PRESS

methods like static signaling and the longest queue first. The approach incorporated multiple future steps for more strategic planning, tested using the SUMO simulator with real-world traffic data from Texas. The study showed potential improvements in traffic flow management and scalability by using a centralized control agent to manage state-action space growth. However, challenges for real-world implementation include high computational demands, reliance on real-time data processing, and communication infrastructure. Additionally, the study's focus on a specific urban environment and the risk of overfitting to simulation conditions may affect the algorithm's generalizability across different traffic patterns or during events like accidents or road closures. Following, we examine the practical applications of this algorithm across different domains.

In [172], authors introduced a hybrid method that melded on-policy traits of SARSA with off-policy features of Q-learning, integrating an N-step look-ahead capability to enhance foresight in decision-making. The algorithm's flexibility to adjust the number of look-ahead steps (N) dynamically allowed it to suit different learning stages, providing an effective balance between exploration and exploitation. This adaptability could facilitate more efficient learning than traditional RL methods. By combining on-policy and off-policy updates, the algorithm capitalized on the strengths of both SARSA and Q-learning, potentially decreasing the overestimation bias seen in Q-learning while still targeting optimal policies. The approach's ability to adjust the N parameter based on learning phases or specific conditions suggested customization for a broad array of applications, from simple gaming scenarios to intricate decision-making tasks in real-world contexts. However, the algorithm's requirement to finely adjust various hyperparameters, like maximum and minimum values of N and the breakpoint for decrementing N, added complexity to its configuration and optimization. This could pose a challenge, particularly for users with limited RL experience. Additionally, maintaining and updating a combination of policies over multiple steps before reaching a decision resulted in higher computational demands, potentially restricting the algorithm's applicability in settings with limited resources or where rapid response times were essential.

The categorization of the examined papers by their domain is given in Table 8.

3.2.3 Summary of Tabular Model-free Methods

In this section, we examined tabular model-free algorithms, an important category in RL. After reviewing an overview of each algorithm and the studies that applied them, we now present a comprehensive summary in Table 9. Figure 6 provides a broad overview of tabular model-free methods.

It is necessary to explain what scalability, sample efficiency, bootstrapping, eligibility traces, and experience replay mean.

- **Scalability**: Refers to the algorithm's ability to handle varying sizes of environments or state spaces. The levels of scalability are:
  - **High scalability**: The algorithm remains efficient even in larger environments or state spaces.
  - **Moderate scalability**: The algorithm can handle medium-sized problems effectively.

Table 8: SARSA Paper Review by Domain

Application Domain	References
Multi-agent Systems and Autonomous Behaviors (Shepherding, Virtual Agents)	[48]
Games and Simulations (Real-Time Strategy Games)	[155]
Energy and Power Management (IoT Networks, Smart Energy Systems)	[28], [165]
Cloud-based Control and Encryption Systems	[47]
Swarm Intelligence and Optimization Problems	[156]
Transportation and Routing Optimization (EVs)	[166]
MPC Tuning	[168]
Network Resilience and Optimization	[169], [31]
Power Systems and Energy Management	[170]
Network Optimization (Optical Transport Networks, Fog RAN)	[154]
Intelligent Traffic Signal Control	[171]
Hybrid RL Algorithms	[172]

ARTICLE IN PRESS

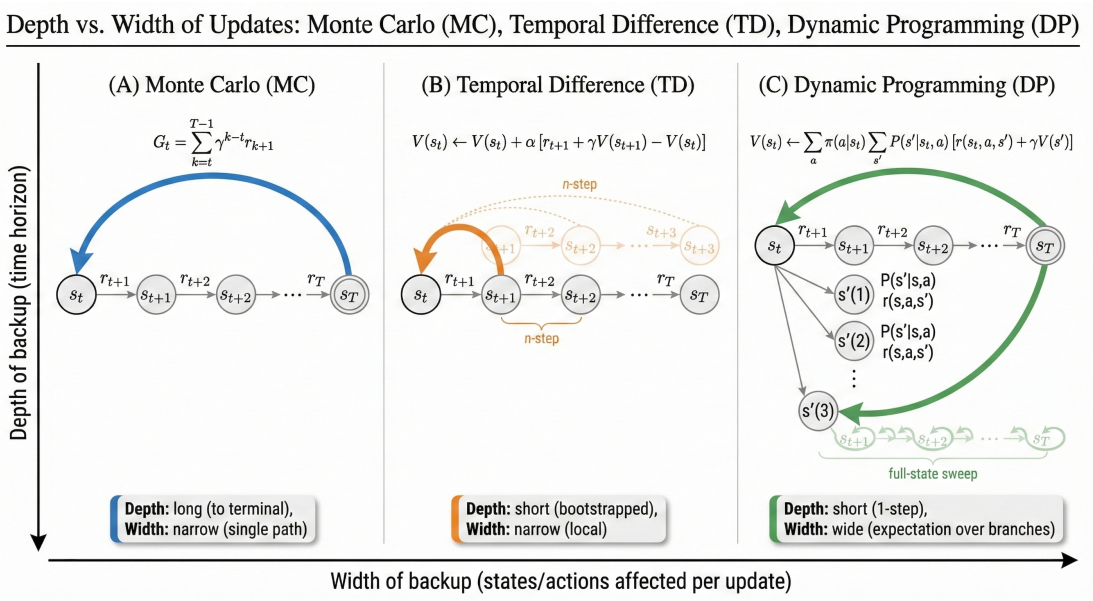


Figure 6: An illustration of the depth and width of updates across MC, TD, and DP methods.

Table 9: Comparison of Tabular Model-free Algorithms

Algorithm	On-policy/ Off-policy	Scalability	Sample-Efficiency	Additional Information
TD Learning	On-policy	High	Moderate	Online learning, bootstrapping
TD(0)-Replay Algorithm	Off-policy	High	Moderate	Uses experience replay
TD(λ)	On-policy	High	Moderate	Trace decay parameter λ
N-step Bootstrapping	On-policy	High	Moderate	Generalization of TD
N-step TD Prediction	On-policy	High	Moderate	Predictive, bootstrapping
N-step Off-policy Learning	Off-policy	High	Moderate	Uses off-policy returns
Q-learning	Off-policy	High	Moderate	Finds optimal policy, bootstrapping
Double Q-learning	Off-policy	High	Moderate	Reduces overestimation bias
SARSA	On-policy	High	Moderate	Learns the action-value function
Expected SARSA	On-policy	High	Moderate	Uses expected value of next state
N-step SARSA	On-policy	High	Moderate	Generalization of SARSA
MC Methods	On-policy	Moderate	Low	Does not bootstrap, needs full episodes
On-policy MC	On-policy	Moderate	Low	Requires exploration starts
MC Importance Sampling	Off-policy	Moderate	Low	Corrects for different policies

  - **Low scalability**: The algorithm struggles with large state spaces, often due to computational complexity or memory requirements.

- **Sample-efficiency**: Reflects how effectively the algorithm learns from the available data. The levels of sample efficiency are:
  - **High sample-efficiency**: The algorithm can learn effectively from fewer samples.
  - **Moderate sample-efficiency**: The algorithm requires a moderate number of samples to learn effectively.
  - **Low sample-efficiency**: The algorithm needs a large number of samples to learn effectively.
- **Bootstrapping**: Involves using estimates to update values, which accelerates the learning process.
- **Eligibility Traces**: Track state-action pairs to improve learning efficiency by integrating information over multiple steps.
- **Experience Replay**: Stores past experiences, which can be reused during learning to improve performance.

ARTICLE IN PRESS

- **Exploration Starts**: Ensure that all states are visited by starting from random states, promoting thorough exploration of the environment.

3.2.4 Algorithmic Rationale and Selection Criteria: Tabular Model-Free Methods

While various sections highlight diverse application domains—ranging from train control to IoT power management—the decision to employ a specific tabular model-free algorithm depends heavily on the operational constraints of the environment. Practitioners must weigh the trade-offs between variance, bias, and the necessity of online learning.

- **Monte Carlo (MC) vs. Temporal Difference (TD):** MC methods are strictly episodic, meaning they require the agent to complete a full trajectory before updating values. This makes them suitable for domains where episodic simulation is cheap and unbiased estimates are prioritized over learning speed (e.g., game strategy and offline signal processing, as seen in [43, 126]). Conversely, TD methods bootstrap, allowing for step-by-step, online updates. This is critical in continuous, non-episodic tasks or environments requiring immediate adaptation, such as real-time routing or dynamic IoT power management.
- **Q-Learning (Off-Policy) vs. SARSA (On-Policy):** The choice between Q-learning and SARSA hinges on the cost of exploration. Q-learning aggressively targets the optimal policy by assuming greedy behavior, which is ideal for environments where exploratory mistakes are not fatal (e.g., simulated network optimization or data processing [146, 151]). However, in domains where safety during learning is critical (e.g., autonomous driving or physical robotic control [166]), SARSA is preferred. SARSA accounts for the agent's actual exploratory actions, leading to a safer, albeit more conservative, learning path.
- **Mitigating Bias with Double Q-Learning and Expected SARSA:** In highly stochastic environments, traditional Q-learning is prone to overestimation bias due to its max operator. Practitioners shift to Double Q-learning when state-action value precision is critical to prevent catastrophic failures, such as in autonomous driving systems [160] or sensitive hardware implementations [159]. Expected SARSA is chosen when variance reduction is paramount, smoothing out updates by considering the probability distribution of all possible next actions, making it highly effective in fluctuating environments like smart grids [170].

3.3 Approximation Model-free Algorithms

Approximation model-free algorithms focus on learning policies and value functions exclusively through interaction with an environment, without requiring an explicit model of the environment's dynamics. These algorithms typically employ function approximators, such as neural networks, to generalize from observed state-action pairs to those not directly observed. As a result, they are highly effective at managing large or continuous state and action spaces [59, 173, 174].

Key characteristics of approximation model-free algorithms are:

- These algorithms directly learn the policy by optimizing the expected reward without needing to explicitly construct a model of the environment.
- In value-function estimation, the value function is approximated using function approximators, which predict expected rewards for states or state-action pairs.
- The solutions are scalable, making them suitable for problems with large or continuous state and action spaces because they can generalize from limited data.

A well-known example is DQN, which applies function approximation to Q-learning algorithms. Therefore, approximation model-free algorithms are crucial for practical applications in RL, especially when creating an explicit model of the environment is infeasible or computationally expensive.

3.3.1 Deep Q-Networks (DQN)

The **DQN** algorithm integrates Q-learning [107] with NNs (see Figure 7 for a depiction of a fully-connected neural network) to learn control policies directly from raw pixel inputs, as shown in Figure 8. It employs Convolutional Neural Networks (CNNs) to process these inputs, along with an experience replay mechanism that helps stabilize learning by breaking correlations between consecutive experiences. The target network, which is updated less frequently, further stabilizes the training process. DQN demonstrated state-of-the-art performance on various Atari 2600 games, outperforming

ARTICLE IN PRESS

previous methods and, in some cases, human experts, all while using a consistent network architecture and hyperparameters across games [175].

DQN integrates the Bellman equation with deep learning methods, including the loss function and gradient descent, to optimize and determine the optimal policy, as illustrated below:

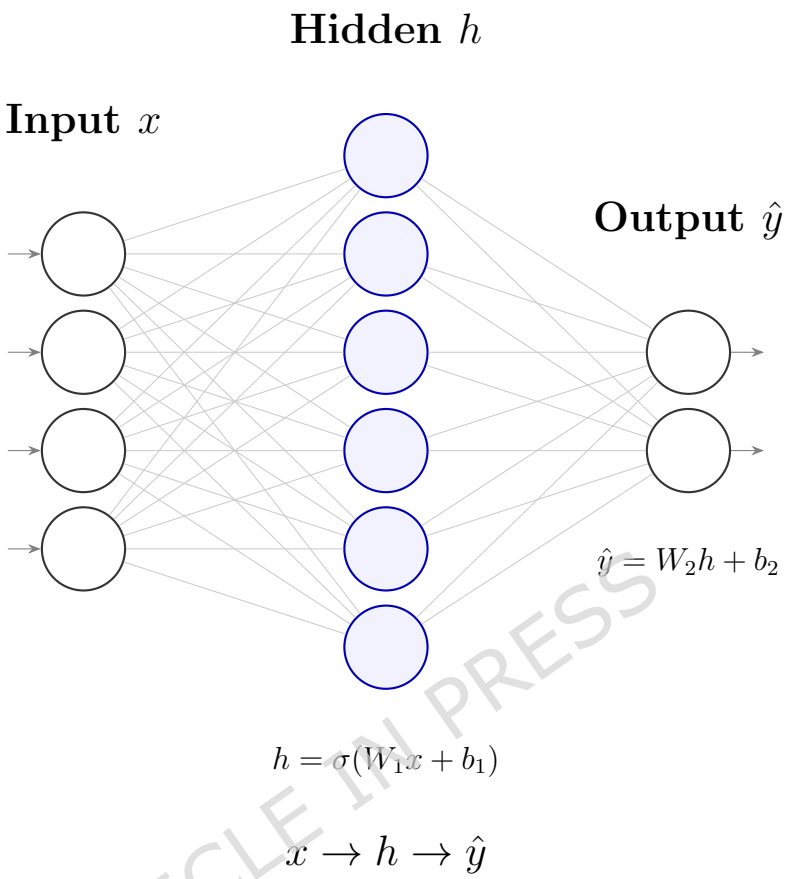


Figure 7: A high-level abstract illustration of a simple fully-connected neural network.

ARTICLE IN PRESS

Loss Function

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s')\sim D}\left[\left(y_i - Q(s,a;\theta_i)\right)^2\right] \tag{25}$$

where

$$y_i = r + \gamma \max_{a'} Q(s', a'; \theta^-) \tag{26}$$

Gradient Descent Step

$$\nabla_{\theta_i} L_i(\theta_i) = \mathbb{E}_{(s,a,r,s')\sim D}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s,a;\theta_i)\right)\nabla_{\theta_i} Q(s,a;\theta_i)\right] \tag{27}$$

Alg. 18 details the complete Deep Q-learning algorithm, regardless of using CNNs, as per the first paper. This algorithm is a generalization to have a general overview of the algorithm.

Having established a solid foundation of the algorithm, we now turn to an analysis of the papers in the literature. In [176], authors explored the use of DQN for controlling Heating, Ventilation, and Air Conditioning (HVAC) systems in an energy-efficient manner without relying on simulation models. The authors applied DQN to a reference office building, simulating its performance with an EnergyPlus model, aiming to minimize energy use while maintaining indoor CO2 concentrations below 1,000 ppm. One of the key strengths of this study was its demonstration of how DQN could improve HVAC control by learning from previous actions, states, and rewards, thus offering a model-free optimization approach. The paper reported a significant reduction in total energy usage (15.7%) compared to baseline operations, highlighting the potential of DQN to enhance energy efficiency in buildings. The research also addressed the complexity and interconnectivity of HVAC systems,

ARTICLE IN PRESS

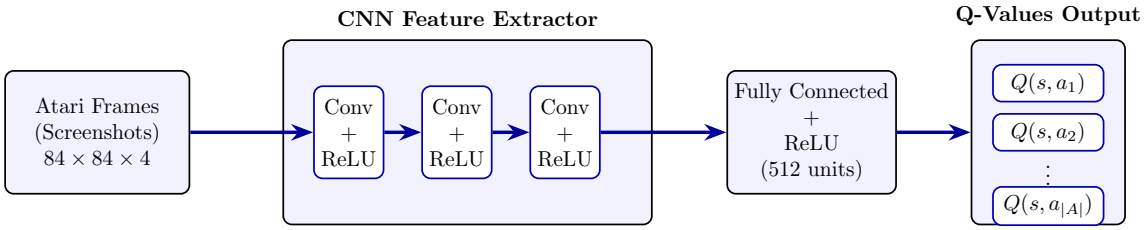


Figure 8: The architecture of the Deep Q-Network (DQN), employing a deep CNN to approximate the value function directly from game screenshots.

Algorithm 18 Deep Q-learning [175]

Initialize replay memory $\mathcal{D}$ to capacity N and action-value function Q with random weights
for episode = 1 to M **do**
  Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$
  for $t = 1$ to T **do**
    With probability ϵ, select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    $$y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$
  end for
end for

providing a practical solution to a traditionally challenging control problem. However, the paper also had some limitations. The reliance on simulation data for training and testing the DQN might not have fully captured the intricacies and variations of real-world scenarios. Additionally, the study focused on a specific type of building and HVAC setup, which might have limited the generalizability of the results to other building types or climates. Furthermore, the study did not delve deeply into the potential challenges of implementing such a system in practice, such as the need for extensive data collection and processing capabilities.

An innovative application of DQN to localize brain tumors in MRI images was introduced in [177]. The key strength of this study lies in its ability to generalize tumor localization with limited training data, addressing significant limitations of supervised DL, which often require large annotated datasets and struggle with generalization. The authors demonstrated that the DQN approach achieved 70% accuracy on a testing set with just 30 training images, significantly outperforming the supervised DL method that showed only 11% accuracy due to over-fitting. This showcased the robustness of RL in handling small datasets and its potential for broader application in medical imaging. The use of a grid-world environment to define state-action spaces and a well-designed reward system further strengthened the methodology. A notable weakness, however, was the limitation to two-dimensional image slices, which might not have fully captured the complexities of three-dimensional medical imaging. Future work should have addressed this by extending the approach to 3D volumes and exploring more sophisticated techniques to improve stability and accuracy.

Authors in [178] presented a routing algorithm for enhancing the sustainability of Rechargeable Wireless Sensor Networks. The authors proposed an adaptive dual-mode routing approach that integrated multi-hop routing with direct upload routing, optimized through DQN. The strength of this paper lies in its innovative use of RL to dynamically adjust the routing mode based on the life expectancy of nodes, significantly improving the network's energy efficiency and lifespan. The simulation results demonstrated that the proposed algorithm achieved a correct routing mode selection rate of 95% with limited network state information, showcasing its practical applicability and robustness. However, the paper did not fully address potential real-world challenges such as the computational overhead of implementing DQN in resource-constrained sensor nodes and the impact

ARTICLE IN PRESS

of network topology changes over time. Additionally, while the simulations showed promising results, practical deployment and testing in diverse environments would have been necessary to validate the generalizability of the approach.

Researchers in [179] explored the application of DQN optimized using Particle Swarm Optimization (PSO) for resource allocation in fog computing environments, specifically tailored for healthcare applications. A notable strength of this study was its innovative combination of DQN and PSO, which effectively balanced resource allocation by reducing makespan and improving both average resource utilization and load balancing levels. The methodology leveraged real-time data to dynamically adjust resources, showcasing a practical application of DQN in a critical domain. However, the paper could have benefited from a more detailed discussion on the scalability of the proposed system and its performance under varying network conditions. Additionally, while the results were promising, they were primarily based on simulations, which might not have fully captured the complexities and unpredictability of real-world fog environments. Further validation in practical deployments would have been necessary to fully ascertain the efficacy of the proposed approach.

An adaptive power management strategy for Parallel Plug-in Hybrid EVs (PHEVs) using DQN was investigated in [180]. The strength of this paper lies in its practical application of DQN for real-time power distribution in PHEVs. The approach considered continuous state variables such as battery State of Charge (SOC), required power, vehicle speed, and remaining distance ratio, which enhanced the model's adaptability and precision in dynamic driving conditions. The DQN model successfully minimized fuel consumption while maintaining battery SOC within desired limits, showing a 6% increase in fuel consumption compared to DP, which was globally optimal but computationally impractical for real-time applications. However, the research was primarily based on simulations using the FTP-72 driving cycle, which might not have fully captured the variability of real-world driving conditions. Additionally, the study focused on a specific type of PHEV and driving scenario, which might have limited the generalizability of the results. Further real-world testing and validation across different vehicle models and driving conditions were necessary to establish the robustness and practical applicability of the proposed strategy.

Authors in [181] explored the application of DQN to create an AI for a visual fighting game. A significant strength of this study was its innovative reduction of the action space from 41 to 11 actions, which simplified the training process and enhanced the model's performance. The DQN architecture included convolutional and fully connected layers optimized for handling sequential frame inputs, effectively learning to perform complex combinations in a dynamic, competitive environment. However, a notable limitation was the reliance on a static opponent (None agent) during training, which might not have fully captured the complexities and adaptive behaviors of actual gameplay against diverse opponents. Additionally, the experiments were conducted in a controlled environment with specific hardware, potentially limiting the generalizability of the results to other setups or real-world gaming scenarios. Further work should have focused on testing against more dynamic and varied opponents to evaluate the robustness and adaptability of the AI.

Authors in [182] presented a novel path-planning approach using an enhanced DQN combined with dense network structures. The strength of this work was its innovative policy of leveraging both depth and breadth of experience during different learning stages, which significantly accelerated the learning process. The introduction of a value evaluation network helped the model quickly grasp environmental rules, while the parallel exploration structure improved the accuracy by expanding the experience pool. The use of dense connections further enhanced feature propagation and reuse, contributing to improved learning efficiency and path planning success. However, the primary limitation was that the experiments were conducted in a controlled grid environment with specific sizes (5x5 and 8x8), which might not have fully represented the complexities of real-world scenarios. Additionally, the reliance on a fixed maximum number of steps could potentially have led to suboptimal policy evaluations in dynamic and larger environments. Future work should have focused on validating this approach in more diverse and scalable settings to assess its generalizability and robustness.

The use of DQN for the real-time control of ESS co-located with renewable energy generators was discussed in [183]. A key strength of this work was its model-free approach, which did not rely on distributional assumptions for renewable energy generation or real-time prices. This flexibility allowed the DQN to learn optimal policies directly from interaction with the environment. The simulation results demonstrated that the DQN-based control policy achieved near-optimal performance, effectively balancing energy storage management tasks like charging and discharging without violating operational constraints. However, the primary limitation was the reliance on simulated data for both training and evaluation, which might not have captured all the complexities of real-world energy

ARTICLE IN PRESS

Table 10: DQN Papers Review

Application Domain	References
General RL (Policy learning, raw experience)	[177]
Network Optimization	[178]
Swarm Intelligence and Optimization Problems	[179],
Network Optimization (Optical Transport Networks, Fog)	[180]
Games and Simulations	[181], [182]
Security Games and Strategy Optimization	[184]

systems. Additionally, the approach assumed the availability of significant computational resources, which might not have been feasible for all consumers. Future work should have focused on real-world implementations and considered the computational constraints of practical deployment environments.

Study [184] presented a novel approach to countering intelligent Unmanned Aerial Vehicle (UAV) jamming attacks using a Stackelberg dynamic game framework. The UAV jammer, acting as the leader, used Deep Recurrent Q-Networks (DRQN) to optimize its jamming trajectory, while ground users, as followers, employed DQN to find optimal communication trajectories to evade the jamming. The strength of this paper was its comprehensive modeling of the UAV jamming problem using DRQN and DQN, which effectively handled the dynamic and partially observable nature of the environment. The approach proved effective in simulations, showing that both the UAV jammer and ground users could achieve optimal trajectories that maximized their respective long-term cumulative rewards. The Stackelberg equilibrium ensured that the proposed strategies were stable and effective in a competitive environment. However, the primary limitation was the complexity and computational demands of implementing DRQN and DQN in real-time scenarios, which might have been challenging in practical deployments. Additionally, the simulations were based on specific scenarios and parameters, which might not have fully captured the variability of real-world environments. Further validation through real-world experiments and a more extensive range of scenarios would have been necessary to confirm the robustness and scalability of the proposed approach.

Table 10 provides an overview of the examined papers in DQN and their applications across different domains.

3.3.2 Double Deep Q-Networks (DDQN)

The **DDQN** algorithm is an extension of DQN, specifically designed to mitigate the overestimation bias found in Q-learning. It does so by separating the process of action selection from the evaluation of the Q-value, resulting in a more accurate estimation of action values [185, 186].

The key contributions of **DDQN** can be summarized as follows:

- **Overestimation Reduction**: By employing two distinct networks to select and evaluate actions, **DDQN** reduces the overestimation bias that is common in standard DQN.
- **Improved Stability and Performance**: The decoupling mechanism enhances both the stability and performance of the learning process, particularly in more complex environments.

The double Q-learning update rule is:

$$y = r + \gamma Q(s', \arg\max_{a'} Q(s', a'; \theta); \theta^-) \tag{28}$$

where θ are the parameters of the online network and θ^- are the parameters of the target network. Alg. 19, provides a general overview of the DDQN algorithm.

Now, let us examine the papers in the literature that have utilized DDQN.

Authors in [188] explored the application of DDQN for optimizing energy efficiency in Cloud Radio Access Networks (CRAN). A key strength of this work was its innovative use of DDQN to address the Q-value overestimation issue inherent in traditional DQN methods. By separating the action selection and evaluation processes, the DDQN framework achieved more accurate and stable policy learning. The simulation results showed that DDQN significantly enhanced energy efficiency compared to baseline solutions and standard DQN approaches, demonstrating up to 22% power savings and a 20% improvement in EE. However, the study's reliance on simulated environments and predefined parameters might not have fully reflected the complexities of real-world CRAN scenarios. The

ARTICLE IN PRESS

Algorithm 19 DDQN [187]

Initialize: Online network parameters θ, Target network parameters θ^-, Replay buffer $\mathcal{D}$, Exploration rate ϵ, and Discount factor γ
for each episode **do**
  while (not done) **do**
    Observe state s
    Select action a based on ϵ-greedy policy:
    $$a \leftarrow \begin{cases} \text{random action} & \text{probability } \epsilon \\ \arg\max_a Q(s,a;\theta) & \text{probability } 1-\epsilon \end{cases}$$
    Execute a, observe r and s'
    Store transition (s,a,r,s') in replay buffer $\mathcal{D}$
    Sample a mini-batch of transitions from $\mathcal{D}$
    Compute target:
    $$y = r + \gamma Q(s', \arg\max_{a'} Q(s',a';\theta);\theta^-)$$
    Update online network by minimizing loss:
    $$L(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[(y - Q(s,a;\theta))^2\right]$$
    if Every C steps **then**
      Update the target network:
      $$\theta^- \leftarrow \theta$$
    end if
  end while
end for

scalability and adaptability of the proposed method to diverse network conditions and dynamic user demands need further validation through practical implementations. Additionally, while the study provided a robust theoretical framework, the practical deployment challenges, and computational overheads associated with DDQN in live networks required more in-depth exploration.

The application of DDQN to object detection tasks was investigated in [189]. One of the key strengths of this work was its innovative use of DDQN to enhance the accuracy and efficiency of object detection. By decoupling action selection and evaluation using two separate Q-networks, the DDQN approach addressed the overestimation problem associated with traditional DQN methods. This led to higher precision and recall rates, as demonstrated in their experiments. The method proved efficient, requiring fewer steps to detect objects and showing strong adaptability to different environments, including person detection scenarios. However, the study relied heavily on specific datasets and controlled environments, which might have limited the generalizability of the results. Real-world applications could have presented more complex scenarios that might have challenged the robustness of the proposed method. Further validation in diverse and uncontrolled settings would have helped ascertain the practical applicability and scalability of the DDQN-based object detection framework.

Authors in [190] presented an enhanced path-tracking approach for robotic vehicles using DDQN. A key strength of this work was its innovative application of DDQN for both path smoothing and tracking, which significantly reduced overshoot and settling time compared to traditional methods like pure pursuit control. The proposed method demonstrated superior performance in navigating paths with sharp turns, making it particularly suitable for agricultural applications where precise path following was critical. The use of a simulation environment for training and subsequent testing on a real rover enhanced the robustness of the developed algorithm. However, the reliance on a specific rover model and controlled environments might have limited the generalizability of the results. The study could have benefited from further validation in diverse real-world conditions and with different types of robotic vehicles to fully establish the robustness and adaptability of the proposed method. Additionally, the computational demands of implementing DDQN in real-time applications might have posed practical challenges that needed addressing.

A novel method for improving tactical decision-making in autonomous driving using DDQN enhanced with spatial and channel attention mechanisms was introduced in [191]. The strength of this paper lies in its innovative use of a hierarchical control structure that integrated spatial

ARTICLE IN PRESS

and channel attention modules to better encode the relative importance of different surrounding vehicles and their features. This approach allowed for more accurate and efficient decision-making, as evidenced by the significant improvement in safety rates (54%) and average exploration distance (30%) in simulated environments. The combination of the algorithm with double attention enhanced the agent's ability to make intelligent and safe tactical decisions, outperforming baseline models in terms of both reward and capability metrics. However, the reliance on simulation-based testing limited the assessment of the model's performance in real-world driving scenarios, which might have presented additional complexities and unpredictabilities. The computational overhead associated with the attention modules and the utilized algorithm could also have posed challenges for real-time implementation in autonomous vehicles. Further validation in diverse and realistic environments would have been essential to confirm the robustness and scalability of this approach.

Study [26] explored a novel approach to solving the distributed heterogeneous hybrid flow-shop scheduling problem with multiple priorities of jobs using a DDQN-based co-evolutionary algorithm. A key strength of this work was its comprehensive framework that integrated global and local searches to balance computational resources effectively. The proposed DDQN-based co-evolutionary algorithm showed significant improvements in minimizing total weighted tardiness and total energy consumption by efficiently selecting operators and accelerating convergence. The numerical experiments and comparisons with state-of-the-art algorithms demonstrated the superior performance of the proposed method, particularly in handling real-world scenarios and large-scale instances. However, the study primarily relied on simulations and controlled experiments, which might not have fully captured the complexities and variability of actual manufacturing environments. Additionally, the computational overhead associated with training the DDQN model could have posed challenges for real-time applications. Further validation through real-world implementations and exploring dynamic events such as new job inserts and due date changes would have been necessary to fully establish the robustness and practical applicability of the proposed method.

A method for autonomous mobile robot navigation and collision avoidance using DDQN was developed in [192]. A significant strength of this work was its innovative use of DDQN to reduce reaction delay and improve training efficiency. The proposed method demonstrated superior performance in navigating robots to target positions without collisions, even in multi-obstacle scenarios. By employing a Kinect2 depth camera for obstacle detection and leveraging a well-designed reward function, the method achieved quick convergence and effective path planning, as evidenced by both simulation and real-world experiments on the Qbot2 robot platform. However, the reliance on a controlled laboratory environment and specific hardware (e.g., Optitrack system for positioning) might have limited the generalizability of the results. Real-world applications could have introduced additional complexities and variabilities that the study did not address. Further validation in diverse and uncontrolled environments, along with a discussion on the computational requirements and scalability of the approach, would have been necessary to fully establish its practical applicability.

Authors in [193] implemented an RL-based method for autonomous vehicle decision-making in overtaking scenarios with oncoming traffic. The study leveraged DDQN to manage both longitudinal speed and lane-changing decisions. A notable strength of this research was the use of DDQN with prioritized experience replay, which accelerated policy convergence and enhanced decision-making precision. The simulation results in SUMO showed that the proposed method improved average speed, reduced time spent in the opposite lane, and lowered the overall overtaking duration compared to traditional methods, such as SUMO's default lane-change model. The RL-based approach demonstrated a high collision-free rate (98.5%) and effectively mimicked human decision-making behavior, showcasing significant improvements in safety and efficiency. However, the reliance on simulated environments and specific scenarios might have limited the generalizability of the results to real-world applications. The study assumed complete observability of the state space, which might not have been realistic in actual driving conditions where sensor imperfections and uncertainties were common. Future work should have focused on addressing these limitations by exploring POMDPs and validating the approach in diverse, real-world scenarios.

A DDQN-based control method for obstacle avoidance in agricultural robots was introduced in[24]. A key strength of this work was its innovative use of DDQN to handle the complexities of dynamic obstacle avoidance in a structured farmland environment. The proposed method effectively integrated real-time data from sensors and used a neural network to decide the optimal actions, leading to significant improvements in space utilization and time efficiency compared to traditional risk index-based methods. The study reported high success rates in obstacle avoidance (98-99%) and demonstrated the model's robustness through extensive simulations and field experiments. However,

ARTICLE IN PRESS

the reliance on predefined paths and the assumption of only dynamic obstacles appearing on these paths might have limited the approach's flexibility in more complex or unstructured environments. Additionally, the computational demands of the DDQN framework might have posed challenges for deployment in resource-constrained settings typical of agricultural machinery. Further validation in diverse real-world scenarios and exploration of more scalable solutions could have enhanced the practical applicability of this method.

A DDQN-based algorithm for global path planning of amphibious Unmanned Surface Vehicles (USVs) was studied in [194]. A major strength of this paper was its innovative use of DDQN for handling the complex path-planning requirements of amphibious USVs, which must navigate both water and air environments. The integration of electronic nautical charts and elevation maps to build a detailed 3D simulation environment enhanced the realism and accuracy of the path planning. The proposed method effectively balanced multiple objectives such as minimizing travel time and energy consumption, making it suitable for diverse scenarios, including emergency rescue and long-distance cruising. However, the study primarily relied on simulated environments and predefined scenarios, which might not have fully captured the complexities of real-world applications. The computational demands of the DDQN framework could also have posed challenges for real-time implementation, especially in dynamic environments where quick decision-making was crucial. Further validation through practical deployments and testing in various real-world conditions would have been necessary to fully assess the robustness and scalability of the proposed approach.

Authors in [195] presented a Hand-Over (HO) strategy for 5G networks. The proposed Intelligent Dual Active Protocol Stack (I-DAPS) HO used DDQN to enhance the reliability and throughput of handovers by predicting and avoiding radio link failures. A key strength of this paper was its innovative approach to leveraging DDQN for dynamic and proactive handover decisions in highly variable mmWave environments. The use of a learning-based framework allowed the system to make informed decisions based on historical signal data, thereby significantly reducing HO failure rates and achieving zero millisecond mobility interruption time. The simulation results demonstrated substantial improvements in throughput and reliability over conventional handover schemes like conditional HO and baseline dual active protocol stack HO. However, the study's primary reliance on simulations and specific urban scenarios might have limited the generalizability of the findings. Real-world implementations might have faced additional challenges, such as varying environmental conditions and hardware limitations, that were not fully addressed in the simulations. Further validation in diverse real-world environments would have been necessary to confirm the practical applicability and robustness of the proposed I-DAPS HO scheme.

Authors in [196] implemented an advanced vehicle speed control system using DDQN. The approach integrated high-dimensional video data and low-dimensional sensor data to construct a comprehensive driving environment, allowing the system to mimic human-like driving behaviors. A key strength of this work was its effective use of DDQN to address the instability issues found in Q-learning, resulting in more accurate value estimates and higher policy quality. The use of naturalistic driving data from the Shanghai naturalistic driving study enhanced the model's realism and applicability. The system demonstrated substantial improvements in both value accuracy and policy performance, achieving a score that was 271.73% higher than that of DQN. However, the reliance on pre-recorded driving data and controlled environments might have limited the generalizability of the results. Real-world driving conditions could have been significantly more variable, and the system's performance in such dynamic environments needed further validation. Additionally, the computational demands of DDQN might have posed challenges for real-time implementation in autonomous vehicles, necessitating further optimization for practical deployment.

Authors in [197] addressed the dynamic multiple-channel access problem in IoT networks with energy harvesting devices using the DDQN approach. The key strength of this study lies in its innovative application of DDQN to manage scheduling policies in POMDP. By converting the partial observations of scheduled nodes into belief states for all nodes, the proposed method effectively reduced energy costs and extended the network lifetime. The simulation results demonstrated that DDQN outperformed other RL algorithms, including Q-learning and DQN, in terms of average reward per time slot. However, the paper's reliance on simulations and specific parameters might have limited its applicability to real-world IoT environments. The assumptions regarding the Poisson process for energy arrival and the fixed transmission energy threshold might not have fully captured the complexities of practical deployments. Further validation through real-world experiments and consideration of diverse environmental conditions would have been necessary to fully establish the robustness and scalability of the proposed scheduling approach.

ARTICLE IN PRESS

Table 11: DDQN Paper Review by Domain

Application Domain	References
Energy and Power Management (IoT Networks, Smart Energy Systems)	[188],[197]
Multi-agent Systems and Autonomous Behaviors	[191], [193], [194]
Optimization	[26]
Real-time Systems and Hardware Implementations	[195]
Vehicle Speed Control System	[196]
Robotics	[190], [24], [192]

Table 11 provides a detailed summary of the papers reviewed in this section, categorized by their respective domains within DDQN.

3.3.3 Dueling Deep Q-Networks (Dueling DQN)

The **Dueling DQN** algorithm introduces a novel NNS architecture that separates the representation of state values and advantages, enhancing both learning efficiency and performance. This structure allows the network to estimate the value of each state more accurately, leading to improved policy evaluation and greater stability. The dueling architecture divides the Q-network into two streams: one estimates the state value function, and the other estimates the advantage function, which is then combined to generate the Q-values. By separately estimating the state value and advantage, the dueling network provides more informative gradients, resulting in more efficient learning. This architecture has shown improved performance in a variety of RL tasks, especially in environments where many actions have similar values [198].

The dueling network update rule is given as:

$$Q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + \left(A(s,a;\theta,\alpha) - \frac{1}{|A|}\sum_{a'} A(s,a';\theta,\alpha) \right) \tag{29}$$

where θ are the parameters of the shared network, α are the parameters of the advantage stream, and β are the parameters of the value stream.

By separately estimating the state value and advantage, the dueling network offers more robust estimates of state values. This decoupling mechanism improves stability and overall performance, particularly in environments where many actions have similar values. Alg. 20 provides a comprehensive overview of the Dueling DQN algorithm.

A DRL-based framework using Dueling Double Deep Q-Networks with Prioritized Replay (DDDQNPR) to solve adaptive job shop scheduling problems was developed in [199]. The primary strength of this study lies in its innovative integration of DDDQNPR with a disjunctive graph model, which transformed scheduling into a sequential decision-making process. This approach enabled the model to adapt to dynamic environments and learn optimal scheduling policies through offline training. The method outperformed traditional heuristic rules and genetic algorithms in both static and dynamic scheduling scenarios, demonstrating substantial improvements in scheduling performance and generalization. However, the reliance on predefined benchmarks and controlled experiments may have limited the real-world applicability of the results. Additionally, the complexity and computational demands of the DDDQNPR framework could present challenges for real-time implementation in large-scale production environments. Future work should focus on validating the approach in more diverse real-world settings and exploring more efficient reward functions and neural network structures to improve the scalability and robustness of the method.

Managing Device-to-Device (D2D) communication using Dueling DQN architecture was proposed by the authors in [200]. A key strength of this paper was its effective utilization of the Dueling DQN architecture to autonomously determine transmission decisions in D2D networks, without relying on centralized infrastructure. The approach leveraged easily obtainable Channel State Information (CSI), allowing each D2D transmitter to train its neural network independently. The proposed method successfully mitigated co-channel interference, demonstrating near-optimal sum rates in low signal-to-noise ratio environments and outperforming traditional schemes like no control, opportunistic, and Suboptimal in terms of efficiency and complexity reduction. However, the study's reliance

ARTICLE IN PRESS

Algorithm 20 Dueling DQN [187]

Input: Learning rate lr, batch size $batch$, discount factor γ, exploration rate ϵ, decay rate $decay$, target network update frequency $freq$, episodes
Output: Trained Dueling Q-Network
Initialize online network weights θ
Initialize target network $\theta^- = \theta$
Initialize replay memory $\mathcal{D}$
for episode $e = 1$ to $episodes$ **do**
    Initialize state s
    while state s is not terminal **do**
        Select action a_t using ϵ-greedy policy
        Execute a_t, observe r_t, s_{t+1}
        Store (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$
        Sample minibatch from $\mathcal{D}$
        Compute targets y_j
        Update θ by minimizing loss $L(\theta)$
        if update step mod $freq = 0$ **then**
            Update target network $\theta^- \leftarrow \theta$
        end if
        $s \leftarrow s_{t+1}$
        Decay ϵ
    end while
end for

on simulation data and specific channel models might have limited the generalizability of the results to real-world D2D networks. The approach assumed ideal conditions such as perfect CSI and zero delay in TDD-based full duplex communications, which might not have held in practical scenarios. Future work should have focused on validating the method in diverse real-world environments and addressing potential implementation challenges such as real-time computational requirements and varying network conditions.

Authors explored an advanced power allocation algorithm in edge computing environments using Dueling DQN, in [29]. A significant strength of this study was its innovative use of Dueling DQN, which separated the value and advantage functions within the Q-network. This architecture enhanced the accuracy and efficiency of power allocation by focusing on state-value estimation and advantage learning. The proposed algorithm outperformed several baseline methods, including traditional DQN, fractional programming, and weighted minimum mean square error, in both simulated scenarios and different user densities. The results demonstrated substantial improvements in average downlink rates and reduced computational overhead, highlighting the algorithm's robustness and efficiency in dynamic network conditions. However, the paper relied on simulations with specific parameters and controlled environments, which might have limited its applicability to real-world scenarios. The assumptions regarding perfect synchronization and the static nature of edge servers might not have fully captured the complexities of practical deployments. Further real-world testing and validation were needed to confirm the scalability and practical implementation of the proposed Dueling DQN approach in diverse and dynamic edge computing environments.

A hybrid approach combining Dueling DQN and Double Elite Co-Evolutionary Genetic Algorithm (DECGA) for parameter estimation in variogram models used in geo-statistics and environmental engineering was implemented in [201]. A notable strength of this study was its innovative integration of RL with evolutionary algorithms. The Dueling DQN enhanced the parameter setting of the genetic algorithm, improving the convergence speed and avoiding premature convergence. The DECGA was effective in finding global optimal solutions by maintaining a diverse population. This combined approach significantly improved the accuracy of parameter estimation in both single and nested variogram models, as demonstrated by the experimental results on heavy metal concentration datasets, showing lower residual sum of squares values compared to traditional methods. However, the reliance on controlled datasets and predefined parameters might have limited the method's generalizability to more complex, real-world scenarios. The computational complexity introduced by combining Dueling

ARTICLE IN PRESS

DQN with DECGA could also have posed challenges for practical implementation, especially in scenarios requiring real-time parameter estimation. Future work should have focused on validating the approach in diverse real-world environments and optimizing the computational efficiency for broader applicability.

Authors in [202] presented a method for optimizing UAV flight paths in IoT sensor networks using Dueling DQN. The goal was to balance energy consumption and data delay. A significant strength of the study was its effective use of Dueling DQN to manage UAV paths, enhancing both energy efficiency and network performance. The approach utilized a grid-based method to model network states, reducing computational complexity. Simulation results showed notable improvements in power consumption and data transmission delays compared to existing methods. However, the reliance on simulated environments might have limited the applicability of the results to real-world scenarios, where sensor data and network conditions could have been more variable. Further validation through practical implementations and testing in diverse environments would have been essential to confirm the robustness and scalability of the proposed method. Additionally, the computational demands of Dueling DQN might have posed challenges for real-time applications in resource-constrained IoT networks.

An innovative method for optimizing the Age of Information (AoI) in vehicular fog systems using Dueling DQN was presented in [203]. A major strength of this work was its holistic approach to AoI optimization, considering both the transmission time from the information source to the monitor and from the monitor to the destination. The Dueling DQN algorithm effectively minimized the end-to-end AoI by leveraging edge computing and vehicular fog nodes to handle real-time data more efficiently. The proposed method demonstrated significant improvements in system performance and information freshness compared to DQN and analytical methods. However, the reliance on simulations to validate the approach might have limited the generalizability of the findings to real-world applications. The assumptions made regarding the processing capabilities and network conditions might not have fully captured the complexities of practical implementations. Further research involving real-world testing and validation in diverse environments was necessary to fully establish the robustness and scalability of the proposed method.

The authors in [204] presented an enhanced Dueling DQN method for improving the autonomous capabilities of UAVs in terms of obstacle avoidance and target tracking. One of the key strengths of this study was the effective integration of improved Dueling DQN with a target tracking mechanism, which allowed the UAV to make more precise and timely decisions. The improved algorithm showed superior performance in simulation tests, with significant improvements in obstacle avoidance accuracy and target tracking efficiency. The authors provided a comprehensive analysis of the algorithm's ability to maintain stability and adaptability in dynamic environments, showcasing its practical potential for real-world UAV applications. However, the study's dependence on simulation environments and controlled scenarios might not have fully captured the complexities and unpredictability of real-world applications. The effectiveness of the improved Dueling DQN algorithm in diverse and unstructured environments needs further validation through real-world testing. Additionally, the computational demands of the improved algorithm could have posed challenges for real-time processing and decision-making on resource-constrained UAV platforms. Future work should have addressed these aspects to establish the practicality and scalability of the proposed approach fully.

An advanced path planning approach for Unmanned Surface Vessels (USVs) using Dueling DQN enhanced with a tree sampling mechanism was implemented in [205]. A key strength of this work was the innovative combination of Dueling DQN with a tree-based priority sampling mechanism, which significantly improved the convergence speed and efficiency of the learning process. The approach leveraged the decomposition of the value function into state-value and advantage functions, enhancing the policy evaluation and decision-making accuracy. The simulation results demonstrated that the proposed method achieved faster convergence and more effective obstacle avoidance and path planning compared to DQN algorithms. Specifically, the Dueling DQN algorithm showed improved performance in terms of the number of steps and time required to reach the target across various test environments. However, the study's reliance on simulated static environments might have limited the generalizability of the results to real-world scenarios where the USVs would have encountered dynamic and unpredictable conditions. The paper also did not address the computational complexity introduced by the tree sampling mechanism, which could have posed challenges for real-time implementation on resource-constrained USVs. Future work should have focused on validating the approach in dynamic real-world environments and exploring ways to optimize the computational efficiency for practical applications.

ARTICLE IN PRESS

Table 12: Dueling DQN Paper Review by Domain

Application Domain	References
Energy and Power Management (IoT Networks, Smart Energy Systems)	[29], [202]
Transportation and Routing Optimization	[205]
Swarm Intelligence and Optimization Problems	[199]
Network Optimization	[200], [203]
Geo-statistics and Environmental Engineering	[201]
Autonomous UAVs	[192], [204]
Path planning approach for USVs	[205]

A summary of the analyzed papers is given in Table 12.

3.3.4 Algorithmic Rationale and Selection Criteria: Deep Q-Networks and Variants

The transition from tabular methods to Deep Q-Networks is fundamentally driven by the curse of dimensionality. When state spaces become continuous or unmanageably large, tabular methods fail. Selecting within the DQN family requires careful consideration of the environment's specific complexities. Standard DQN is selected when the primary challenge is state-space representation, but the action space remains discrete and relatively low-dimensional. Its integration of experience replay and target networks stabilizes the non-linear approximations of neural networks. Practitioners use standard DQN as a robust baseline in domains like HVAC control or basic visual recognition [176, 177], where overestimation bias is present but does not critically derail the learning objective. Double DQN (DDQN) is favored when standard DQN inherently overestimates Q-values, which can lead to suboptimal policy convergence in environments with noisy or stochastic rewards. DDQN is the algorithm of choice when decision-making precision is non-negotiable. By decoupling action selection from action evaluation, DDQN is heavily favored in high-stakes or highly dynamic applications, such as autonomous vehicle tactical decision-making, amphibious path planning, and 5G network handovers [191, 194, 195], where overestimating a dangerous action's value could be catastrophic. Finally, practitioners pivot to Dueling DQN when the environment contains many states where the choice of action has minimal impact on the outcome. By separating the value of a state from the advantage of an action, Dueling DQN learns which states are inherently valuable (or dangerous) without needing to evaluate the effect of every possible action at that state. This architecture is specifically chosen for complex coordination tasks with redundant action paths, such as multi-UAV flight optimization, resource allocation in edge computing, or tactical obstacle avoidance [29, 202, 204], as it significantly accelerates convergence compared to standard DQN.

4 Policy-based Methods

Policy-based methods are another core approach in RL, focusing more on directly optimizing the policy used to select actions for an agent. Unlike value-based methods, which first estimate the value function implicit in the task and then derive an optimal policy, policy-based methods directly parameterize and optimize the policy. This approach offers significant advantages, especially in challenging environments with high-dimensional action spaces or in cases where the policies are inherently stochastic. At the core of policy-based methods is the parameterization of policies, typically represented as $\pi(a|s;\theta)$, where θ denotes the policy parameters, s represents the state, and a is the action. In essence, the goal is to find the optimal parameters θ^* that maximize the expected cumulative reward. This is typically achieved through gradient ascent techniques, specifically by Policy Gradient methods, which compute the gradient of the expected reward with respect to the policy parameters, adjusting them in the direction that increases the reward [1, 6, 15].

The gradient of the policy with respect to Q values is estimated in the following way:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a|s) Q_{\pi_\theta}(s,a)\right] \tag{30}$$

Here, $Q_{\pi_\theta}(s,a)$ represents the action-value function under the current policy. This gradient can be estimated using samples from the environment, enabling the use of gradient ascent to iteratively update the policy parameters. The advantages of policy-based methods can be summarized as follows:

ARTICLE IN PRESS

Algorithm 21 REINFORCE [1]

1: **Input:** a differentiable policy parameterization $\pi(a|s,\theta)$
2: Initialize policy parameter $\theta \in \mathbb{R}^{d'}$
3: **repeat** ▷ Forever:
4: Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot;\theta)$
5: **for** each step of the episode $t = 0, \ldots, T-1$ **do**
6: $G \leftarrow$ return from step t
7: $\theta \leftarrow \theta + \alpha\gamma^t G \nabla_\theta \ln \pi(A_t|S_t,\theta)$
8: **end for**
9: **until** convergence or a stopping criterion is met

- **Direct Optimization of the Policy:** These methods optimize the policy directly, making them more effective than value-based methods, especially in environments with continuous and high-dimensional action spaces. They are particularly well-suited for problems in robotics and control, where actions are inherently continuous.
- **Stochastic Policies:** Policy-based methods naturally handle stochastic policies, which are useful in scenarios where exploration is crucial or when the optimal policy is stochastic by nature. Stochastic policies help to balance the exploration-exploitation trade-off more efficiently.
- **Improved Convergence Properties:** Policy-based methods often exhibit smoother convergence compared to value-based methods, which can struggle with instabilities and divergence due to challenges like the Church condition for value function approximations.

It is important to note a critical nuance regarding the taxonomy of modern policy optimization algorithms. While foundational methods like REINFORCE rely purely on Monte Carlo returns (a strict policy-based approach), advanced algorithms such as TRPO and PPO frequently employ an Actor-Critic architecture in practice. Specifically, these algorithms utilize a learned value function (the critic) to compute advantage estimates, which significantly reduces the variance of the policy gradient. However, because their core mathematical derivation and primary optimization objectives are fundamentally rooted in directly updating the policy via a surrogate objective function, they are classified under Policy-based Methods in this survey. We explicitly acknowledge this architectural overlap, as the boundary between pure policy-based methods and Actor-Critic frameworks is inherently blurred in modern Deep RL implementations.

Now, we discuss the first Policy Gradient method, REINFORCE, in the next subsection.

4.1 REINFORCE

The **REINFORCE** algorithm is a foundational contribution to RL, especially within the realm of policy gradient methods. This algorithm aims to optimize the expected cumulative reward by adjusting the policy parameters in the direction of the gradient of the expected reward. As shown in Alg. 21, the REINFORCE algorithm operates within the stochastic policy framework, where the policy, parameterized by θ, defines a probability distribution over actions based on the current state. The central idea behind the REINFORCE algorithm is to use the log-likelihood gradient estimator to update the policy parameters [206]. The gradient of the expected reward with respect to the policy parameters θ is expressed as follows:

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi \left[\nabla_\theta \log \pi_\theta(a|s) G_t \right], \tag{31}$$

where $\pi_\theta(a|s)$ is the probability of taking action a in the state s under policy π parameterized by θ, and G_t is the return (cumulative future reward) following time step t. This gradient estimation forms the basis for the parameter update rule in REINFORCE (line 7):

$$\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(a|s) G_t, \tag{32}$$

where α is the learning rate.

One of the key strengths of the REINFORCE algorithm is its simplicity and generality, which makes it applicable to a wide variety of problems. However, it also encounters challenges, such as high variance in the gradient estimates and slow convergence, especially in environments with sparse or delayed rewards. By examining the applications of REINFORCE, we can gain a deeper understanding of these advantages and limitations.

ARTICLE IN PRESS

Authors in [207] investigated the global convergence rates of the REINFORCE algorithm in episodic RL settings. The authors aimed to close the gap between theoretical and practical implementations of policy gradient methods by providing new convergence results for the REINFORCE algorithm. The paper's strengths lie in its comprehensive theoretical analysis and practical relevance. The authors derived performance bounds for the REINFORCE algorithm using a fixed mini-batch size, aligning more closely with practical implementations. They provided the first set of global convergence results for the REINFORCE algorithm, including sub-linear high probability regret bounds and almost sure global convergence of average regret. The focus on the widely-used REINFORCE gradient estimation procedure rather than state-action visitation measure-based estimators addressed a significant gap between theory and practice. The authors established that the REINFORCE algorithm was sample efficient, with polynomial complexity, which was crucial for practical applications due to the high cost of obtaining samples. However, the complexity of the analysis might have challenged practitioners less familiar with the mathematical concepts. The reliance on assumptions such as the log-barrier regularization term and soft-max policy parametrization might have limited the generality of the results. The paper focused on the stationary infinite-horizon discounted setting, and a more detailed discussion on applying the results to other settings would have enhanced its relevance. The absence of empirical validation of the proposed convergence bounds and sample efficiency was another limitation. Including experimental results would have provided additional evidence of the practical utility of the findings.

A novel approach to Energy Management Strategies (EMS) in Fuel Cell Hybrid EVs (FCHEV) using the fuzzy REINFORCE algorithm was introduced in [208]. This method integrated a fuzzy inference system (FIS) with Policy Gradient RL (PGRL) to optimize energy management, achieve hydrogen savings, and maintain battery operation. One of the key strengths of the paper was the innovative combination of fuzzy logic with the REINFORCE algorithm. By employing a fuzzy inference system to approximate the policy function, the authors effectively leveraged the generalization capabilities of fuzzy logic to handle the complexity and uncertainty inherent in energy management tasks. This integration helped to address the limitations of traditional EMS methods that relied heavily on expert knowledge and static rules, thus providing a more adaptive and robust solution. The use of a fuzzy baseline function to stabilize the training process and reduce the variance in policy gradient updates was another notable advantage. This approach enhanced the convergence rate and stability of the learning process, which was particularly beneficial in real-time applications where computational efficiency and robustness were critical. The paper's demonstration of the algorithm's adaptability to changing driving conditions and system states further underscored its practical relevance and effectiveness. However, the complexity of the proposed method might have posed implementation challenges, particularly for practitioners who were less familiar with fuzzy logic. The integration of FIS and PGRL required careful tuning of parameters and membership functions, which could have been time-consuming and computationally intensive. Additionally, while the fuzzy REINFORCE algorithm showed promise in reducing the computational burden and improving convergence, the reliance on fuzzy logic introduced an additional layer of complexity that might not have been necessary for all applications. The paper also provided a comprehensive analysis of the simulation and hardware-in-loop experiments, validating the effectiveness of the proposed method in real-world scenarios. The results indicated that the fuzzy REINFORCE algorithm could achieve near-optimal performance without requiring accurate system models or extensive prior knowledge, making it a versatile and practical solution for EMS in FCHEVs.

Authors in [209] presented a study protocol for a trial aimed at improving medication adherence among patients with type 2 diabetes using an RL-based text messaging program. One of the key strengths of this study was its innovative use of RL to personalize text message interventions. By tailoring messages based on individual responses to previous messages, the approach had the potential to optimize engagement and improve adherence more effectively than generic messaging strategies. This personalized communication could have led to more significant behavior changes and better health outcomes for patients with diabetes. The study's design also enhanced its practical relevance. Conducted in a real-world setting at Brigham and Women's Hospital, it involved patients with suboptimal diabetes control, which reflected a common clinical scenario. The use of electronic pill bottles to monitor adherence provided accurate and objective data, supporting the reliability of the study outcomes. Additionally, the trial's primary outcome of average medication adherence over six months was a meaningful measure that directly related to the study's objective. However, there were some weaknesses and challenges associated with the study. The requirement for patients to use electronic pill bottles and smartphones with a data plan or WiFi might have limited the generalizability of the

ARTICLE IN PRESS

findings to populations without access to such technology. Furthermore, the study's reliance on self-reported adherence as a secondary outcome introduced the potential for reporting bias. The study also faced potential limitations related to the length of the follow-up period and the evaluation of the long-term sustainability of the intervention. While a six-month follow-up period was sufficient to assess initial adherence improvements, longer-term studies would have been necessary to determine whether the benefits of the intervention were sustained over time.

In [210], the authors presented a novel method for rate adaptation in 802.11 wireless networks leveraging the REINFORCE algorithm. The proposed approach, named ReinRate, integrated a comprehensive set of observations, including received signal strength, contention window size, current modulation and coding scheme, and throughput, to adapt dynamically to varying network conditions and optimize network throughput. One of the key strengths of this paper was its innovative application of the REINFORCE algorithm to WiFi rate adaptation. Traditional rate adaptation algorithms like Minstrel and Ideal relied on limited observations such as packet loss rate or signal-to-noise ratio, which could have been insufficient in dynamic wireless environments. In contrast, ReinRate's broader set of observations allowed for a more nuanced response to varying conditions, leading to significant improvements in network performance. The authors demonstrated that ReinRate outperformed Minstrel and Ideal algorithms by up to 102.5% and 30.6% in network scenarios without interference, and by up to 35.1% and 66.6% in scenarios with interference. Another strength was the comprehensive evaluation of ReinRate using the ns-3 network simulator and ns3-ai OpenAI Gym. The authors conducted extensive simulations under various network scenarios, both static and dynamic, with and without interference. This thorough evaluation provided strong evidence of the algorithm's effectiveness and adaptability in real-world conditions. The results indicated that ReinRate consistently achieved higher throughput compared to traditional algorithms, showcasing its ability to handle the challenges of dynamically changing wireless environments. However, the complexity of the proposed method might have posed challenges for practical implementation. The integration of multiple observations and the application of the REINFORCE algorithm required careful tuning of parameters and computational resources.

A new approach to enhance DRL for outdoor robot navigation was investigated in [211]. The key innovation was the use of a heavy-tailed policy parameterization, which induced exploration in sparse reward settings, a common challenge in outdoor navigation tasks. A significant strength of the paper lies in addressing the sparse reward issue, which was prevalent in many real-world navigation scenarios. Traditional DRL methods often relied on carefully designed dense reward functions, which could have been impractical to implement. The authors proposed HTRON, an algorithm that leveraged heavy-tailed policy parameterizations, such as the Cauchy distribution, to enhance exploration without needing complex reward shaping. This approach allowed the algorithm to learn efficient behaviors even with sparse rewards, making it more applicable to real-world scenarios. The paper's thorough experimental evaluation was another strong point. The authors tested HTRON against established algorithms like REINFORCE, Proximal Policy Optimization (PPO), and Trust Region Policy Optimization (TRPO) (explained later in the upcoming subsections) across three different outdoor scenarios: goal-reaching, obstacle avoidance, and uneven terrain navigation. HTRON outperformed these algorithms in terms of success rate, average time steps to reach the goal, and elevation cost, demonstrating its effectiveness and efficiency. The use of a realistic unity-based simulator and the deployment of the algorithm on a Clearpath Husky robot further validated the practical applicability of the proposed method. However, the complexity of the proposed algorithm and the specific choice of heavy-tailed distributions might have posed challenges. The implementation of heavy-tailed policy gradients could have introduced instability, especially in the initial learning phases. While the authors mitigated this with adaptive moment estimation and gradient clipping, these techniques required careful tuning and expertise, potentially limiting accessibility for practitioners.

Table 13 provides an overview of the studies reviewed in this section.

The next policy-based algorithm we discuss is TRPO. In the following subsection, we will cover this algorithm.

4.2 Trust Region Policy Optimization (TRPO)

TRPO, introduced by [212], represents a significant advancement in RL, particularly within policy optimization methods. The main goal of TRPO is to optimize control policies while guaranteeing monotonic improvement, addressing the weaknesses of earlier methods [213–215] that often led to unstable policy updates and poor performance in complex tasks. TRPO is specifically designed to

ARTICLE IN PRESS

Table 13: REINFORCE Papers Review

Application Domain	References
Energy and Power Management	[208]
Theoretical Research (Convergence, stability)	[207]
Network Optimization	[210]
Robotics	[211]

handle large, nonlinear policies, such as those represented by neural networks. The algorithm ensures that each policy update results in a performance improvement by keeping the updated policy within a "trust region" around the current policy. This trust region is defined through a constraint on the KL divergence between the new and old policies, effectively preventing large, destabilizing updates [212, 216]. It operates within the stochastic policy framework, where the policy π_θ is parameterized by θ and defines a probability distribution over actions given the states. The expected discounted reward for a policy π is given by:

$$J(\pi) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)\right], \tag{33}$$

where γ is the discount factor, $r(s_t, a_t)$ is the reward at time step t, and the expectation is taken over the state and action trajectories induced by the policy. To ensure that the policy update remains within a safe boundary, TRPO constrains the KL divergence between the new policy $\pi_{\theta'}$ and the old policy π_θ:

$$D_{KL}(\pi_\theta \| \pi_{\theta'}) \leq \zeta, \tag{34}$$

where ζ is a small positive constant. This constraint ensures that the new policy does not deviate too much from the old policy, thereby providing stability to the learning process. TRPO optimizes a surrogate objective function that approximates the true objective while respecting the trust region constraint. The surrogate objective $L(\theta)$ is defined as:

$$L(\theta) = \hat{\mathbb{E}}\left[\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} \hat{A}_{\theta_{\text{old}}}(s, a)\right], \tag{35}$$

where $\hat{A}_{\theta_{\text{old}}}(s, a)$ is an estimate of the advantage function, which measures the relative value of taking action a in state s under the old policy. As demonstrated in Alg. 22, the practical implementation of TRPO involves the following steps:

1. **Sample Trajectories:** Collect a set of trajectories using the current policy $\pi_{\theta_{\text{old}}}$ (line 3).
2. **Estimate Advantages:** Compute the advantage function $\hat{A}_{\theta_{\text{old}}}(s, a)$ using the collected trajectories (line 4).
3. **Optimize Surrogate Objective:** Solve the constrained optimization problem to find the new policy parameters θ' (lines 5-7):

$$\theta' = \arg\max_\theta \hat{\mathbb{E}}\left[\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} \hat{A}_{\theta_{\text{old}}}(s, a)\right], \tag{36}$$

   subject to

$$D_{KL}(\pi_{\theta_{\text{old}}} \| \pi_\theta) \leq \zeta. \tag{37}$$

4. **Update Policy:** Update the policy parameters to θ' (lines 9-11).

Let us examine several research studies that have utilized TRPO to gain a deeper understanding of its applications and effectiveness. A Monotonic Policy Optimization (MPO) algorithm was designed to address the challenges associated with high-dimensional continuous control tasks in [218]. The primary focus was on ensuring monotonic improvement in policy performance, which was crucial for stability and efficiency in RL. One of the significant strengths of this paper was its innovative approach to policy optimization. The authors derived a new lower bound on policy improvement that penalized average policy divergence on the state space, rather than the maximum divergence. This approach addressed a critical limitation of previous algorithms, such as TRPO, which could suffer from worst-case degradation in policy performance. By focusing on average divergence, the

ARTICLE IN PRESS

Algorithm 22 TRPO [217]

Input: initial policy parameters θ_0
for $k = 0, 1, 2, \ldots$ **do**
  Collect set of trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$
  Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm
  Form sample estimates for:
  policy gradient $\hat{g}_k$ (using advantage estimates)
  and KL-divergence Hessian-vector product function $f(v) = \hat{H}_k v$
  Use CG with n_{cg} iterations to obtain $x_k \approx \hat{H}_k^{-1} \hat{g}_k$
  Estimate proposed step $\Delta_k \approx \sqrt{\frac{2\delta}{x_k^T \hat{H}_k x_k}} x_k$
  Perform backtracking line search with exponential decay to obtain final update
  $\theta_{k+1} = \theta_k + \alpha^j \Delta_k$
end for

MPO algorithm ensured more consistent and reliable improvements, making it particularly suitable for high-dimensional continuous control tasks. The empirical evaluation of the MPO algorithm was another strong point. The authors conducted extensive simulations using the MuJoCo physics engine, testing the algorithm on various challenging robot locomotion tasks, including swimming, quadruped locomotion, bipedal locomotion, and more. The results demonstrated that the MPO algorithm consistently outperformed state-of-the-art methods like TRPO and Vanilla policy gradient in terms of average discounted rewards. The algorithm's performance in these high-dimensional tasks highlighted its robustness and practical applicability. However, the complexity of the proposed algorithm might have posed challenges for practical implementation. The need for computing the natural policy gradient and performing line searches to determine optimal step sizes required significant computational resources. This complexity could have limited the accessibility of the MPO algorithm for practitioners without advanced computational capabilities. Additionally, while the algorithm guaranteed monotonic improvement, this came at the cost of slower training speed due to conservative step sizes. The authors suggested combining MPO with other faster methods during the initial training phase to mitigate this issue, but this added another layer of complexity to the implementation.

[219] introduced an enhancement to the TRPO algorithm by incorporating entropy regularization. This modification, termed EnTRPO, aimed to improve exploration and generalization by encouraging more stochastic policy choices. The paper demonstrated the effectiveness of EnTRPO through experiments on the CartPole system, showcasing better performance compared to the original TRPO. One of the main strengths of the paper was its innovative use of entropy regularization to enhance the TRPO algorithm. By adding an entropy term to the advantage function, the authors effectively encouraged exploration, which was crucial to avoid premature convergence to suboptimal policies. This approach addressed a common limitation of TRPO, which could sometimes restrict exploration due to its strict KL divergence constraints between consecutive policies. The entropy regularization helped maintain a balance between exploration and exploitation, leading to more robust learning outcomes. The empirical evaluation provided in the paper was another significant strength. The authors conducted thorough experiments using the Cart-Pole system, a well-known benchmark in the field. The results showed that EnTRPO converged faster and more reliably than TRPO, particularly when the discount factor was set to 0.85. This indicated that the proposed method not only improved exploration but also enhanced the overall convergence speed and stability of the learning process. The use of a well-defined experimental setup, including details on neural network architectures and hyperparameters, added credibility to the findings. A potential limitation was the reliance on a single benchmark task for evaluation. While the Cart-Pole system was a standard benchmark, it was relatively simple compared to many real-world applications. The paper would have benefited from additional experiments on more complex tasks and environments to demonstrate the generalizability and robustness of EnTRPO. This would have provided stronger evidence of the method's effectiveness across a wider range of scenarios.

The challenge of applying trust region methods to Multi-Agent RL (MARL) was investigated in [220]. The authors introduced Heterogeneous-agent TRPO (HATRPO) and Heterogeneous-Agent Proximal Policy Optimization (HAPPO) algorithms. These methods were designed to guarantee monotonic policy improvement without requiring agents to share parameters or relying on restrictive assumptions about the decomposability of the joint value function. A strength of the paper was

ARTICLE IN PRESS

its theoretical foundation. The authors extended the theory of trust region learning to cooperative MARL by developing a multi-agent advantage decomposition lemma and a sequential policy update scheme. This theoretical advancement allowed HATRPO and HAPPO to ensure monotonic improvement in joint policy performance, a key advantage over existing MARL algorithms that did not guarantee such improvement. This theoretical guarantee was essential for stable and reliable learning in multi-agent settings, where individual policy updates could often lead to non-stationary environments and suboptimal outcomes. The empirical validation of HATRPO and HAPPO on benchmarks such as multi-Agent MuJoCo and StarCraft II demonstrated the effectiveness of these algorithms. The results showed that HATRPO and HAPPO significantly outperformed strong baselines, including Independent Proximal Policy Optimization (IPPO), MAPPO, and MADDPG, in various tasks. This performance improvement highlighted the practical applicability of the proposed methods in complex, high-dimensional environments. The thorough experimental evaluation across multiple scenarios provided strong evidence of the robustness and generalizability of HATRPO and HAPPO. However, the complexity of implementing HATRPO and HAPPO could have been a potential limitation. The algorithms required the computation of multi-agent advantage functions and sequential updates, which could have been computationally intensive and challenging to implement efficiently. This complexity might have limited the accessibility of these methods to practitioners who might not have had advanced computational resources or expertise in implementing sophisticated algorithms.

Authors in [221] presented a method to actively recognize objects by choosing a sequence of actions for an active camera. This method utilized TRPO combined with Extreme Learning Machines (ELMs) to enhance the efficiency of the optimization algorithm. One of the significant strengths of this paper was its innovative application of TRPO in conjunction with ELMs. ELMs provide a simple yet effective way to approximate policies, reducing the computational complexity compared to traditional DNNs. This resulted in an efficient optimization process, crucial for real-time applications like active object recognition. The use of ELMs allowed for faster convergence and more straightforward implementation, making the proposed method accessible for practical applications. However, the complexity of integrating TRPO with ELMs could have posed challenges for some practitioners. Although ELMs simplified the optimization process, they still required careful tuning of parameters, such as the number of hidden nodes and the distribution of random weights. This additional layer of complexity might have limited the method's accessibility for users without extensive experience in RL and NNs.

In [42], authors explored the application of the TRPO algorithm in MARL environments, specifically focusing on hide-and-seek games. The authors compared the performance of TRPO with the Vanilla Policy Gradient (VPG) algorithm to determine the most effective method for this type of game. One of the primary strengths of this paper was its focus on a well-defined, complex multi-agent environment. Hide-and-seek games inherently involve dynamic interactions between agents, making them an excellent testbed for evaluating algorithms. By using TRPO, which was designed to ensure monotonic policy improvement, the authors addressed a significant challenge in MARL: maintaining stable and consistent learning despite the presence of multiple interacting agents. The empirical results presented in the paper highlighted the strengths of TRPO, especially in scenarios where the testing environment differed from the training environment. TRPO's ability to adapt to new environments and maintain high performance was a notable advantage over the VPG algorithm, which performed better in environments identical to the training conditions but struggled when faced with variability. This adaptability was crucial for practical applications of MARL, where agents often encountered unpredictable changes in their environment. Another strength was the comprehensive experimental setup, which included various configurations and scenarios. The authors meticulously compared the performance of TRPO and VPG across different numbers of agents and types of environments (quadrant and random walls scenarios). This thorough approach provided robust evidence supporting the efficacy of TRPO in MARL settings. However, the paper also had some limitations. The complexity of implementing TRPO in a multi-agent context could have been a barrier for practitioners. TRPO required careful tuning and substantial computational resources, which might not have been readily available in all settings. Additionally, the reliance on simulation results raised questions about the real-world applicability of the findings. While the hide-and-seek game was a useful simulation environment, real-world deployments could have presented additional challenges not captured in the simulations.

The application of TRPO to improve Cross-Site Scripting (XSS) detection systems was analyzed by the authors in [222]. The authors aimed to enhance the resilience of XSS filters against adversarial attacks by using RL techniques to identify and counter malicious inputs. One of the main

ARTICLE IN PRESS

strengths of this paper was its innovative approach to applying TRPO in Cybersecurity, specifically for XSS detection. Traditional XSS detection methods often relied on static rules and signatures, which could be easily bypassed by sophisticated attackers. By leveraging TRPO, the authors introduced a dynamic and adaptive mechanism that could learn to detect and counteract adversarial attempts to exploit XSS vulnerabilities. This use of TRPO enhanced the robustness of the detection system, making it more resilient to evolving threats. A limitation of this study was the reliance on specific hyperparameters, such as the learning rate and discount factor, which could have significantly impacted the model's performance. The paper would have benefited from a more detailed discussion on how these parameters were selected and their influence on the detection model. Providing guidelines or heuristics for parameter tuning would have helped practitioners replicate and extend the study's findings.

Authors in [223] aimed to create a universal policy for a locomotion task that could adapt to various robot morphologies, using TRPO. The study investigated the use of surrogate models, specifically Polynomial Chaos Expansion (PCE) and model ensembles, to model the dynamics of the robots. One of the primary strengths of this thesis was its innovative approach to developing a universal policy. The use of TRPO ensured stability and reliable policy updates even in complex environments. The focus on creating a policy that could generalize across different robot configurations was particularly noteworthy, as it addressed the challenge of designing controllers that were not limited to a single robot morphology. The integration of surrogate models, especially the PCE, was another strong point. PCE allowed for efficient sampling and modeling of the stochastic environment, potentially reducing the number of interactions required with the real environment. This was crucial for practical applications where real-world interactions could have been costly or risky. The theoretical foundation laid for using PCE in this context was robust and showed promise for future research. However, the thesis also highlighted several challenges and limitations. The complexity of accurately modeling the dynamics with PCE was a significant hurdle. The results indicated that while PCE showed potential, it currently could not model the dynamics accurately enough to be used in combination with TRPO effectively. The computational time required for PCE was also a practical concern, limiting its immediate applicability. The model ensemble surrogate showed some promise but ultimately failed to train a successful policy. This pointed to the difficulty of creating surrogate models that could capture the complexities of robot dynamics sufficiently. The thesis suggested that using the original environment from the RoboGrammar library yielded better results, emphasizing the need for more advanced or alternative surrogate modeling techniques.

A novel approach for optimizing Home Energy Management Systems (HEMS) using Multi-agent TRPO (MA-TRPO) was investigated in [224]. This approach aimed to improve energy efficiency, cost savings, and consumer satisfaction by leveraging TRPO techniques in a multi-agent setup. One of the primary strengths of this paper was its consumer-centric approach. Traditional HEMS solutions often prioritized energy efficiency and cost savings without adequately considering consumer preferences and comfort. By incorporating a preference factor for Interruptible-Deferrable Appliances (IDAs), the proposed MA-TRPO algorithm ensured that consumer satisfaction was taken into account, leading to a more holistic and practical solution. This consumer-centric focus was crucial for the widespread adoption of HEMS in real-world settings. Another strength was the comprehensive use of real-world data for training and validation. The authors utilized five-minute retail electricity prices derived from wholesale market prices and real-world Photovoltaic (PV) generation profiles. This approach enhanced the practical relevance and robustness of the proposed method, as it demonstrated the algorithm's effectiveness under realistic conditions. Additionally, the paper provided a detailed explanation of the various components of the smart home environment, including the non-controllable base load, IDA, Battery Energy Storage System (BESS), and PV system, which added clarity and depth to the study. The use of MA-TRPO in a multi-agent setup was also a significant contribution. The proposed method modeled and trained separate agents for different components of the HEMS, such as the IDA and BESS, allowing for more specialized and effective control strategies. This multi-agent approach addressed the complexities and inter-dependencies within the home energy environment, leading to more efficient and coordinated energy management. The paper's reliance on simulation results, while comprehensive, still leaves questions about the real-world applicability of the proposed method. Although the use of real-world data enhanced relevance, further validation in actual home environments would have strengthened the case for practical deployment. Real-world testing was essential to ensure the robustness and effectiveness of the MA-TRPO algorithm in diverse and dynamic home energy scenarios. Additionally, the reliance on discrete action spaces simplified the problem but might not have fully captured the nuances of continuous control

Table 14: TRPO Papers Review

Application Domain	References
Object Recognition	[221]
Theoretical Research (Convergence, stability)	[218]
Hybrid RL Algorithms	[219]
Multi-agent Systems and Autonomous Behaviors	[220], [42]
Cybersecurity	[222]
Robotics	[223]
Energy and Power Management	[224], [188] ,[197]

in real-world applications. Future work could explore extending the algorithm to handle continuous action spaces for more precise control.

Authors in [225] investigated the application of TRPO to address the joint spectrum and power allocation problem in the Internet of Vehicles (IoV). The objective was to minimize AoI and power consumption, which were crucial for maintaining real-time communication and energy efficiency in vehicular networks. One of the key strengths of this paper was its focus on AoI, a vital metric for ensuring timely and accurate information exchange in vehicular communications. By incorporating AoI into the optimization framework, the authors addressed a significant challenge in IoV networks, where the freshness of information directly impacted road safety and traffic efficiency. The proposed TRPO-based approach effectively balanced the trade-off between minimizing AoI and reducing power consumption, showcasing its practical relevance. The paper's reliance on certain assumptions, such as the availability of CSI and the periodic reporting of CSI to the base station, might have limited its generalizability. In real-world scenarios, obtaining accurate and timely CSI could have been challenging due to various factors like signal interference and mobility. Future work could explore more practical approaches to CSI estimation and reporting to enhance the applicability of the proposed solution.

Table 14 organizes the studies reviewed in this section by their respective domains, providing a summary of the research landscape in TRPO. An advancement over TRPO was proposed as a new algorithm, PPO.

4.3 Proximal Policy Optimization (PPO)

PPO, introduced by [226], marks a significant advancement in policy gradient methods. PPO is designed to achieve reliable performance and sample efficiency, addressing the limitations of earlier policy optimization algorithms, such as VPG methods and TRPO. Using policy gradient methods, the policy parameters are optimized through stochastic gradient ascent by estimating the gradient of the policy. One of the most widely used policy gradient estimators is:

$$\hat{g} = \hat{\mathbb{E}}_t \left[\nabla_\theta \log \pi_\theta(a_t|s_t) \hat{A}_t \right], \tag{38}$$

where π_θ represents the policy parameterized by θ, and $\hat{A}_t$ is an estimator of the advantage function at time step t. This estimator helps construct an objective function whose gradient corresponds to the policy gradient estimator:

$$L_{PG}(\theta) = \hat{\mathbb{E}}_t \left[\log \pi_\theta(a_t|s_t) \hat{A}_t \right]. \tag{39}$$

PPO simplifies TRPO by utilizing a surrogate objective with a clipped probability ratio, enabling multiple epochs of mini-batch updates. To ensure stable learning, large policy updates should be avoided. Therefore, the PPO objective is defined as follows:

$$L_{CLIP}(\theta) = \hat{\mathbb{E}}_t \Big[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \Big] \tag{40}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio, and ϵ is a hyperparameter. This objective clips the probability ratio to ensure it stays within a reasonable range, preventing excessively large updates.

The practical implementation of PPO involves the following steps:

1. **Sample Trajectories:** Collect trajectories using the current policy $\pi_{\theta_{\text{old}}}$.
2. **Estimate Advantages:** Compute the advantage function $\hat{A}_t$ using the collected trajectories.

ARTICLE IN PRESS

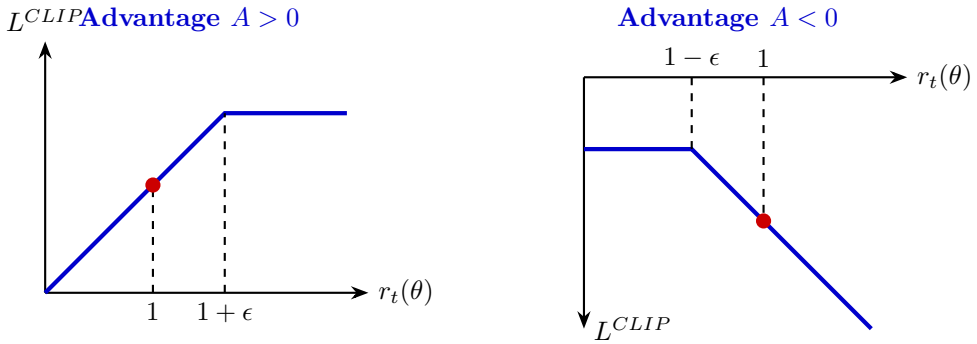


Figure 9: The PPO surrogate objective L^{CLIP} as a function of the probability ratio $r_t(\theta)$. The clipping mechanism prevents destructively large policy updates.

Algorithm 23 PPO [226]

1: Initialize policy parameters θ, and value function parameters ϕ
2: Initialize old policy $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$
3: **for** iteration $i = 1$ to M **do**
4: Collect trajectories $\{(s_t, a_t, r_t, s_{t+1})\}$ by running policy $\pi_{\theta_{\text{old}}}$
5: Compute advantage estimates $\hat{A}_t$ for each trajectory using the value function
6: **for** epoch $j = 1$ to K **do**
7: Compute probability ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$
8: Define the surrogate loss:

$$L_{CLIP}(\theta) = \hat{\mathbb{E}}_t\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\Big]$$

9: Perform mini-batch gradient ascent on the surrogate objective $L_{CLIP}(\theta)$
10: Update the value function parameters ϕ by minimizing:

$$L_V(\phi) = \hat{\mathbb{E}}_t\left[\left(V_\phi(s_t) - R_t\right)^2\right]$$

11: **end for**
12: Update old policy: $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$
13: **end for**

3. **Optimize Surrogate Objective:** Perform several epochs of optimization on the surrogate objective $L_{CLIP}(\theta)$ using minibatch stochastic gradient descent.
4. **Update Policy:** Update the policy parameters to the new parameters θ.

Before diving into related papers, to illustrate the algorithm's functionality, PPO's algorithm is provided in Alg. 23. As illustrated in Algorithm 23, the practical implementation of PPO functionally operates as an Actor-Critic architecture. The algorithm explicitly maintains and initializes both the policy parameters θ (the actor) and the value function parameters ϕ (the critic). The critic is updated during the training process via a squared error loss (Line 10) to provide accurate advantage estimations, thereby guiding the actor's policy updates while maintaining the core proximal clipping objective.

A DRL approach for optimizing traffic flow in mixed-autonomy scenarios, where both Connected Autonomous Vehicles (CAVs) and human-driven vehicles coexist, was introduced in [227]. The authors proposed three distributed learning control policies for CAVs using PPO, a policy gradient DRL method, and conducted experiments with varying traffic settings and CAV penetration rates on the flow framework, a new open-source microscopic traffic simulator. One of the primary strengths of

ARTICLE IN PRESS

the paper was its innovative approach to mixed-autonomy traffic optimization at a network level. The authors hypothesized that controlling distributed CAVs at a network level could outperform individually controlled CAVs, and their experimental results supported this hypothesis. The network-level RL policies for controlling CAVs significantly improved the total rewards and average velocity compared to individual-level RL policies. This finding was crucial for the development of ITS, which aimed to optimize traffic flow and reduce congestion. The use of the flow framework and SUMO environment for experiments was another strength of the paper. These tools provided a realistic and flexible simulation environment for testing the proposed control policies. The comprehensive evaluation, including different traffic settings and CAV penetration rates (10%, 20%, and 30%), added robustness to the findings. The authors' use of three different learning strategies—single-agent asynchronous learning, joint global cooperative learning, and joint local cooperative learning—allowed for a thorough comparison of the effectiveness of network-level control policies. The paper also highlighted the potential high communication overhead associated with the global joint cooperative policy, especially as the penetration rate of CAVs increased. This overhead could have limited the scalability of the solution in real-world scenarios where communication resources were constrained. The authors suggested that the local joint cooperative policy, which required less communication, could have been a more practical choice in such situations, though it might not have performed as well as the global policy.

In [228], the authors presented a novel method for managing power grid line flows in real time using PPO. This approach addressed the challenges posed by the increasing penetration of renewable resources and the unpredictability of power system conditions, aiming to prevent line overloading and potential cascading outages. One of the key strengths of this paper was the application of PPO in a critical real-time grid operation context. PPO was well-suited for this task due to its ability to ensure stable updates within trust regions, thus maintaining the reliability of the control policies. This characteristic was crucial in power systems, where stability and safety were paramount. The authors' decision to use PPO over other algorithms like DDPG and Policy Gradient was justified by PPO's robustness in handling larger and more complex action spaces. A limitation was the reliance on simulation results for validation. While the use of high-fidelity power grid simulators was a strength, real-world deployments could present additional challenges not captured in simulations. Factors such as communication delays, unexpected equipment failures, and human operator interventions could have impacted the effectiveness of the control strategies. Further real-world testing and validation were necessary to fully assess the robustness and practicality of the proposed method in diverse and dynamic power grid environments.

A framework for optimizing metro service schedules and train compositions using PPO within a DRL framework was proposed in [229]. This method was applied to handle the dynamic and complex problem of metro train operations, focusing on minimizing operational costs and improving service regularity. A significant strength of the paper was its innovative application of PPO to metro service scheduling and train composition. PPO, known for its stability and efficiency in handling large-scale optimization problems, was effectively utilized to address the dynamic nature of metro operations. The integration of PPO with ANNs for approximating value functions and policies demonstrated a robust approach to tackling the high-dimensional decision space inherent in metro scheduling. This combination enhanced the framework's ability to adapt to varying passenger demands and operational constraints. The paper's use of a real-world scenario, specifically the Victoria line of the London underground, for testing and validation was another strong point. The authors provided a comprehensive evaluation, comparing their PPO-based method with established meta-heuristic algorithms like the genetic algorithm and differential evolution. The results indicated that the PPO-based approach outperformed these traditional methods in terms of both solution quality and computational efficiency. This practical validation underscored the method's applicability and effectiveness in real-world settings. The reliance on a specific set of operational constraints, such as fixed headways and trailer limits, might have also limited the method's generalizability. While these constraints were necessary for practical implementations, exploring more flexible constraint formulations could have further enhanced the method's adaptability to different metro systems and operational conditions.

Authors in [230] proposed an advanced scheduling algorithm for multitask environments using a combination of PPO and an optimal policy characterization. The main focus was to optimize the scheduling of multiple tasks across multiple servers, taking into account random task arrivals and renewable energy generation. One of the primary strengths of this paper was its innovative combination of PPO with a priority rule, the Earlier Deadline and Less Demand First (ED-LDF). This rule prioritized tasks with earlier deadlines and lower demands, which was shown to be optimal under heavy traffic conditions. The integration of ED-LDF with PPO effectively reduced the dimensionality

ARTICLE IN PRESS

of the action space, making the algorithm scalable and efficient even in large-scale settings. This reduction in complexity was crucial for practical applications where the number of tasks and servers could have been substantial. However, the complexity of implementing the proposed method might have posed challenges. The integration of the ED-LDF rule with PPO required careful tuning and a deep understanding of both RL and optimal scheduling principles. This complexity could have limited the accessibility of the method for practitioners who might not have had advanced expertise in these areas. Additionally, the reliance on heavy-traffic assumptions for the optimality of the ED-LDF rule might have limited the generalizability of the approach to all scheduling environments. Real-world scenarios could have presented varying traffic conditions that did not always align with these assumptions. The paper's focus on renewable energy as a factor in the scheduling decision was another strength, as it aligned with the growing importance of sustainability in computing operations. However, the paper could have benefited from further exploration of how variations in renewable energy availability impacted the scheduling performance. This aspect was crucial for real-world applications where renewable energy sources could have been highly variable.

An innovative approach to enhancing image captioning models using PPO was designed by the authors in [46]. The authors aimed to improve the quality of generated captions by incorporating PPO into the phase of training, specifically targeting the optimization of scores. The study explored various modifications to the PPO algorithm to adapt it effectively for the image captioning task. A significant strength of the paper was its integration of PPO with image captioning models, which traditionally relied on VPG methods. The authors argued that PPO could provide better performance due to its ability to enforce trust-region constraints, thereby improving sample complexity and ensuring stable policy updates. This was particularly important for image captioning, where maintaining high-quality training trajectories was crucial. The authors' experimentation with different regularization techniques and baselines was another strong point. They found that combining PPO with dropout decreased performance, which they attributed to increased KL-divergence of RL policies. This empirical observation was critical as it guided future implementations of PPO in similar contexts. Furthermore, the adoption of a word-level baseline via MC estimation, as opposed to the traditional sentence-level baseline, was a noteworthy innovation. This approach was expected to reduce the variance of policy gradient estimators more effectively, contributing to improved model performance. While the results were promising, they were primarily validated on the MSCOCO dataset. Further validation on other datasets and in real-world applications would have been beneficial to assess the generalizability and robustness of the approach. The paper could have also benefited from a more detailed discussion on the impact of different hyperparameter settings and the specific configurations used in the experiments. Providing this information would have enhanced the reproducibility of the study and allowed other researchers to build on the authors' work more effectively.

In [231], a centralized coordination scheme for CAVs at intersections without traffic signals was developed. The authors introduced the Model Accelerated PPO (MA-PPO) algorithm, which incorporated a prior model into the PPO algorithm to enhance sample efficiency and reduce computational overhead. One of the significant strengths of this paper was its focus on improving computational efficiency, a major challenge in centralized coordination methods. Traditional methods, such as MPC, were computationally demanding, making them impractical for real-time applications with a large number of vehicles. By using MA-PPO, the authors significantly reduced the computation time required for coordination, achieving an impressive reduction to 1/400 of the time needed by MPC. This efficiency gain was crucial for real-time deployment in busy intersections. A limitation was the focus on a specific intersection scenario. While the four-way single-lane intersection was a common setup, real-world intersections could have varied significantly in complexity and traffic patterns. Future work could have explored the applicability of MA-PPO to more complex intersection scenarios and different traffic conditions to ensure broader generalizability.

The application of PPO in developing a DRL controller for the nonlinear attitude control of fixed-wing UAVs was explored in [232]. The study presented a proof-of-concept controller capable of stabilizing a fixed-wing UAV from various initial conditions to desired roll, pitch, and airspeed values. One of the primary strengths of the paper was its innovative use of PPO for UAV attitude control. PPO was known for its stability and efficient policy updates, making it well-suited for complex control tasks like UAV attitude stabilization. The choice of PPO over other RL algorithms was justified by its robust performance and low computational complexity, which were crucial for real-time control applications. The authors also highlighted the practical advantages of using PPO, such as its hyperparameter robustness across different tasks. One of the limitations was the reliance on a single UAV model and specific aerodynamic coefficients for validation. While the Skywalker-X8 was

ARTICLE IN PRESS

Table 15: PPO Papers Review

Application Domain	References
Distributed DRL Control for Mixed-Autonomy Traffic Optimization	[227]
Power Systems and Energy Management	[228]
Transportation and Routing Optimization (EVs)	[229]
Real-time Systems and Hardware	[230]
Image Captioning Models	[46]
Hybrid RL Algorithms	[46]
Intelligent Traffic Signal Control	[231]
Real-time Systems and Hardware	[232], [39]

a popular fixed-wing UAV, the generalizability of the proposed approach to other UAV models with different aerodynamic characteristics remained to be explored. Future work could have benefited from testing the PPO-based controller on a wider range of UAV models to ensure broader applicability.

The use of PPO to control the position of a quadrotor was investigated in [39]. The primary goal was to achieve stable flight control without relying on a predefined mathematical model of the quadrotor's dynamics. One of the major strengths of this paper was its application of PPO in the context of quadrotor control. By using PPO, the authors ensured that the control policy updates remained stable and efficient, which was crucial for real-time applications like quadrotor control. The choice of PPO over other RL algorithms was well justified due to its robustness and low computational complexity. The authors' approach to utilizing a stochastic policy gradient method during training, which was then converted to a deterministic policy for control, was another notable strength. This strategy ensured efficient exploration during training, allowing the quadrotor to learn a robust control policy. The use of a simple reward function that focused on minimizing the position error between the quadrotor and the target further added to the efficiency of the training process. One limitation was the reliance on specific initial conditions and a fixed simulation environment. While the authors showed that the PPO-based controller could recover from harsh initial conditions, the generalizability of the method to different quadrotor models and varying environmental conditions remained to be explored. Future work could have benefited from testing the controller on a wider range of scenarios and incorporating additional environmental factors to ensure broader applicability.

Authors in [233] addressed the development of an intelligent lane change strategy for autonomous vehicles using PPO. This approach uses PPO to manage lane change maneuvers in dynamic and complex traffic environments, focusing on enhancing safety, efficiency, and comfort. The authors' design of a comprehensive reward function that considered multiple aspects of lane change maneuvers was one of the strengths. The reward function incorporated components for safety (avoiding collisions and near-collisions), efficiency (minimizing travel time and aligning with desired speed and position), and comfort (reducing lateral and longitudinal jerks). This multi-faceted approach ensured that the learned policy optimized for a holistic driving experience, balancing the often competing demands of these different aspects. The inclusion of a safety intervention module to prevent catastrophic actions was a particularly noteworthy feature. This module labels actions as "catastrophic" or "safe" and could replace potentially dangerous actions with safer alternatives, enhancing the robustness of the learning process. This safety-centric approach addressed a critical concern in applying DRL to real-world autonomous driving tasks, where safety was paramount. However, the complexity of implementing PPO for lane change maneuvers posed challenges. The need for continuous training and fine-tuning of parameters could have been resource-intensive and might not have been feasible for all developers or organizations, acknowledging that it is true for some other algorithms. Table 15 shows a summary of the analyzed papers. Moreover, table 16 represents all papers reviewed in this section as a package.

4.3.1 Algorithmic Rationale and Selection Criteria: Policy Gradient Methods

Value-based methods inherently struggle with continuous action spaces due to the computationally expensive max operator. Policy-based methods directly parameterize the policy, making them the default choice for continuous control tasks. Selection within this family relies on balancing computational overhead with the need for update stability.

ARTICLE IN PRESS

Table 16: Policy-based Papers Review

Application Domain	References
Distributed DRL Control for Mixed-Autonomy Traffic Optimization	[227]
Power Systems and Energy Management	[228]
Transportation and Routing Optimization (EVs)	[229]
Real-time Systems and Hardware	[230], [232], [39]
Image Captioning Models	[46]
Hybrid RL Algorithms	[46], [219]
Intelligent Traffic Signal Control	[231]
Energy and Power Management	[208], [224], [188] ,[197]
Theoretical Research (Convergence, stability)	[207], [218]
Network Optimization	[210]
Object Recognition	[221]
Multi-agent Systems and Autonomous Behaviors	[220], [42]
Cybersecurity	[222]
Robotics	[223], [211]

- **REINFORCE:** As a foundational algorithm, REINFORCE is selected for its simplicity and applicability to episodic tasks. However, due to its high variance and slow convergence, practitioners typically restrict its use to environments where reward shaping is straightforward or where baseline subtractions sufficiently stabilize learning (e.g., rate adaptation in wireless networks [210]).
- **TRPO:** TRPO is explicitly chosen for safety-critical applications where an erratic policy update could lead to system failure or unsafe behavior. By enforcing a KL-divergence constraint on policy updates, it guarantees monotonic improvement. This makes it ideal for complex, high-stakes domains such as medical interventions, cybersecurity, or power grid management [222, 228], though this stability comes at the cost of high computational complexity due to the calculation of second-order derivatives.
- **PPO:** PPO is currently the industry standard for continuous control because it achieves the stability of TRPO without the second-order mathematical overhead. By using a clipped surrogate objective, it prevents destructively large policy updates. Practitioners heavily favor PPO for robotics, mixed-autonomy traffic control, and Large Language Model fine-tuning (RLHF) because it offers the best balance of ease of implementation, sample complexity, and hyperparameter robustness [227, 232].

We now turn our attention to the final group of methods, actor-critic methods. We begin by introducing the general actor-critic approach, which combines both value-based and policy-based methods.

5 Actor-Critic Methods

Actor-critic methods combine value-based and policy-based approaches. These methods consist of two key components: the actor, which selects actions based on a policy, and the critic, which evaluates the actions based on their value function. By providing feedback on the quality of the actions taken, the critic helps guide the actor in updating the policy. This collaboration enables more stable and efficient learning, addressing some of the limitations found in pure policy or value-based approaches [52, 234].

In the next subsection, we will first introduce two major variants of actor-critic methods, namely Asynchronous Advantage Actor-Critic (A3C) and Advantage Actor-Critic (A2C), and then analyze their various applications.

5.1 A3C & A2C

The A2C algorithm is a synchronous variant of the A3C algorithm, which was introduced by [235]. A2C maintains the key principles of A3C but simplifies the training process by synchronizing the updates of multiple agents, thereby leveraging the strengths of both actor-critic methods and advantage estimation. The actor-critic architecture combines two primary components in both algorithms:

ARTICLE IN PRESS

the actor, which is responsible for selecting actions, and the critic, which evaluates the actions by estimating the value function. The actor updates the policy parameters in a direction that is expected to increase the expected reward, while the critic provides feedback by computing the TD error. This integration allows for more stable and efficient learning compared to using Actor-only or critic-only methods [236]. Advantage estimation is a technique used to reduce the variance of the policy gradient updates. The advantage function $A(s,a)$ represents the difference between the action-value function $Q(s,a)$ and the value function $V(s)$:

$$A(s,a) = Q(s,a) - V(s). \tag{41}$$

By utilizing the advantage function, A2C focuses on actions that yield higher returns than the average, which aids in making more informed policy updates [1]. In contrast to A3C, where multiple agents update the global model asynchronously, A2C synchronizes these updates. Multiple agents operate in parallel environments, gathering experiences and computing gradients, which are then aggregated and used to update the global model in a synchronized manner. This synchronization simplifies the implementation and avoids issues associated with asynchronous updates, such as non-deterministic behavior and the potential overwriting of gradients.

The A3C algorithm operates as follows (see Figure 10): Based on Alg. 24, first, the parameters of the policy network (actor) θ and the value network (critic) ϕ are initialized (line 1). Then, multiple agents are run in parallel, each interacting with its own copy of the environment (lines 2-14). Each agent independently collects a batch of experiences (s_t, a_t, r_t, s_{t+1}) (lines 5-9). For each agent, the advantage is computed using the collected experiences. Subsequently, the gradients of the policy and value networks are calculated using the advantage estimates. **Finally, each agent independently updates the global model parameters θ and ϕ asynchronously using the computed gradients (line 13).**

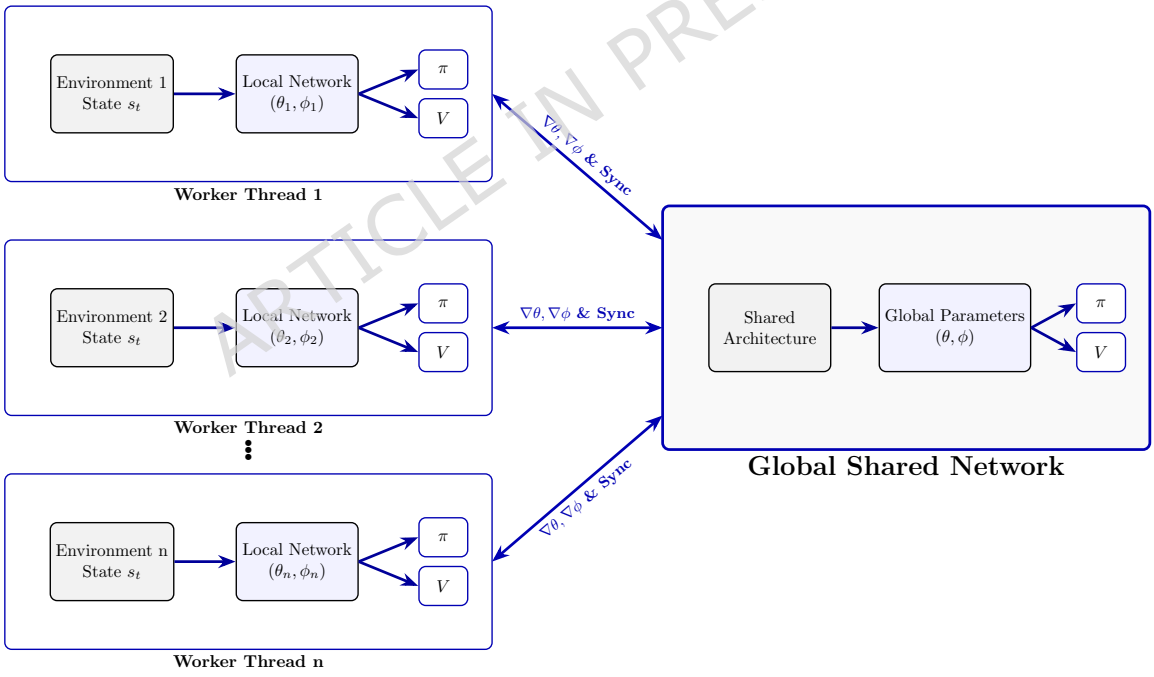


Figure 10: The Asynchronous Advantage Actor-Critic (A3C) architecture. Multiple worker threads interact with parallel environments independently, computing gradients $(\nabla\theta, \nabla\phi)$ to periodically update and synchronize with the shared Global Network [235].

The A2C algorithm, on the other hand, operates as follows: Based on Alg. 25, first, the parameters of the policy network (actor) θ and the value network (critic) ϕ are initialized (line 1). Then, multiple agents are run in parallel, each interacting with its own copy of the environment (line 4). Each agent collects a batch of experiences (s_t, a_t, r_t, s_{t+1}) (line 10). For each agent, the advantage is computed using the collected experiences. Subsequently, the gradients of the policy and value networks are

ARTICLE IN PRESS

Algorithm 24 A3C [237]

```
1: Initialize actor and critic networks with random weights
2: for each episode ∈ [1, n] do
3:     Download weights from the headquarters to each AC
4:     for each AC do
5:         Initialize the random state s_0
6:         for each t ∈ [1, k] do
7:             Select action a_t from the actor network
8:             Execute action a_t and observe reward r_t from the critic network and next state s_t
9:             Update the actor network parameters
10:        end for
11:        Update the critic network parameters
12:    end for
13:    Upload weights of each AC network to the headquarters
14: end for
```

Algorithm 25 A2C [238]

```
1: Initialize policy network (actor) parameters θ and value network (critic) parameters φ
2: Set number of parallel agents N
3: repeat
4:     for each agent i ← 1 to N in parallel do
5:         Get initial state s_i
6:         Initialize local episode storage for agent i
7:         for each step t do
8:             Sample action a_i from policy π_θ(a_i|s_i)
9:             Execute a_i, observe reward r_i and next state s'_i
10:            Store (s_i, a_i, r_i, s'_i) in local storage for agent i
11:            s_i ← s'_i
12:        end for
13:        Compute advantage estimates Â_i for agent i
14:        and policy gradient ∇_θ L_PG(θ) for agent i
15:        and value loss ∇_φ L_V(φ) for agent i
16:    end for
17:    Aggregate gradients from all agents
18:    Update global actor parameters θ and critic parameters φ using aggregated gradients
19: until convergence or maximum steps reached
```

ARTICLE IN PRESS

calculated using the advantage estimates (lines 13-15). **Finally, the gradients from all agents are aggregated and used to update the global model parameters θ and ϕ (lines 17-18).**

Overview of A3C applications in the literature

[50] introduced a scheduler named MOPTSA3C, which prioritized tasks and virtual machines based on various factors such as task length, runtime processing capacities, and electricity unit costs. This approach aimed to optimize makespan, resource utilization, and resource cost using an enhanced A3C. One of the significant strengths of this paper was its comprehensive multi-objective approach. By addressing multiple objectives simultaneously, including minimizing makespan, optimizing resource utilization, and reducing resource costs, the proposed scheduler ensured a balanced and efficient task scheduling process in multi-cloud environments. The use of an improved A3C algorithm enhanced the robustness and efficiency of the scheduling process. The asynchronous nature of A3C allowed for parallel training and faster convergence, which was crucial for dynamic and large-scale cloud environments. The scheduler's ability to prioritize tasks and virtual machines based on multiple factors led to more informed and effective scheduling decisions. Cloud environments were inherently dynamic and could exhibit unpredictable changes in workload, resource availability, and cost structures. The proposed method might have needed further enhancements to adapt to these real-time variations effectively and ensure robust performance under fluctuating conditions. While the paper presented a

ARTICLE IN PRESS

significant advancement in task scheduling for cloud computing, addressing the computational complexity, scalability, and real-world implementation challenges would have been crucial for its practical adoption and effectiveness in diverse cloud environments.

A latency-oriented cooperative caching strategy using A3C was proposed in [239]. This approach aimed to minimize average download latency by predicting content popularity and optimizing caching decisions in real time, thereby improving user experience in fog radio access networks. One strength was the comprehensive evaluation of the proposed method through extensive simulations. The authors compared their A3C-based approach with several baseline algorithms, including greedy caching, random caching, and a cooperative caching strategy that did not consider user equipment caching capacity. The results demonstrated significant reductions in average download latency, highlighting the effectiveness of the proposed strategy in optimizing caching performance. Another limitation was the reliance on accurate and timely content popularity predictions. The effectiveness of the caching strategy depended heavily on the accuracy of these predictions. In real-world applications, user preferences and content popularity could change unpredictably, and any deviations from the predicted values could negatively impact the performance of the caching algorithm. Additionally, the assumption that users did not request the same content repeatedly might not have held true in all scenarios, potentially affecting the reliability of the popularity prediction model.

Authors in [240] presented a comprehensive approach to optimizing resource allocation and pricing in Mobile Edge Computing (MEC)-enabled blockchain systems using the A3C algorithm. The study's strengths lie in its innovative integration of blockchain with MEC to enhance resource management. The A3C algorithm's capability to handle both continuous and high-dimensional discrete action spaces made it well-suited for the dynamic nature of MEC environments. The use of prospect theory to balance risks and rewards based on miner preferences added a nuanced understanding of real-world scenarios. The results demonstrated that the proposed A3C-based algorithm outperformed baseline algorithms in terms of total reward and convergence speed, indicating its effectiveness in optimizing long-term performance. However, the paper had several limitations. The reliance on a specific MEC server configuration and a fixed number of mobile devices might have limited the generalizability of the findings to other settings with different configurations. The assumption that all validators were honest simplified the model but might not have reflected real-world blockchain environments where malicious actors could have existed. The additional complexity introduced by the collaborative local MEC task processing mode might have increased the computational overhead, potentially affecting the scalability of the proposed solution. Moreover, the paper did not address the potential impact of network latency and varying network conditions on the performance of the A3C algorithm, which could have been significant in practical deployments.

In [241], authors explored the application of A3C to create a cognitive agent designed to help Alzheimer's patients play chess. The primary goal was to enhance cognitive skills and boost brain activity through chess, a game known for its cognitive benefits. One of the key strengths of the paper was its innovative approach to leveraging A3C to assist individuals with cognitive disabilities in playing chess. The cognitive agent provided real-time assistance by suggesting offensive and defensive moves, thereby helping players improve their strategies and cognitive abilities. This approach addressed the gap in traditional AI chess opponents, which did not educate players on strategies and tactics. The cognitive agent relied on accurate feedback for consistency. Agent effectiveness was heavily dependent on its ability to provide relevant and timely suggestions. The agent needed to accurately interpret the player's intentions and provide appropriate feedback in real-world applications. Users who had difficulty navigating digital interfaces might also have experienced accessibility issues due to the agent's reliance on digital interfaces. Another concern was scalability. A controlled environment showed promising results, but the system might not have been able to handle a broader range of cognitive impairments. Additional users and interactions could have introduced additional complexity, making it difficult to maintain performance.

Researchers in [242] introduced a novel approach to autonomous valet parking using a combination of PPO and A3C. This method aimed to address the control errors due to the non-linear dynamics of vehicles to optimize parking maneuvers. An important strength of this study was the use of the A3C-PPO algorithm. Combining the advantages of both PPO and A3C, this hybrid approach resulted in a more stable and efficient learning process. A3C's asynchronous nature allowed parallel training, which sped convergence and improved state-action exploration. In addition, PPO prevented drastic changes from destabilizing the learning process by limiting the magnitude of policy updates. Incorporating manual hyperparameter tuning further optimized the training process, resulting in better rewards. One of the limitations was the reliance on specific assumptions about the environment

ARTICLE IN PRESS

Table 17: A3C Paper Review by Domain

Application Domain	References
Cloud-based Control and Encryption Systems	[50], [240]
Games	[241]
Multi-agent Systems and Autonomous Behaviors	[242]
Network Optimization	[243]

and sensor accuracy. The effectiveness of the proposed method depended on the accurate detection and interpretation of the surroundings by sensors such as cameras and LiDAR. Any inaccuracies or deviations in sensor data could have impacted the performance and robustness of the algorithm. In real-world applications, environmental conditions such as lighting, weather, and obstacles could have varied significantly, and ensuring reliable sensor performance under these conditions was crucial. Additionally, the study did not address the potential impact of network latency and communication issues between the vehicle and the central control system, which could have affected the real-time decision-making process. Ensuring robust communication and minimizing latency was critical for the practical implementation of autonomous valet parking systems.

An advanced approach for optimizing content caching in 5G networks using A3C was proposed in [243]. This method aimed to minimize the total transmission cost by learning optimal caching and sharing policies among cooperative Base Stations (BSs) without prior knowledge of content popularity distribution. One of the main strengths of this paper was the use of the A3C algorithm, which leveraged the asynchronous nature of multiple agents to achieve faster convergence and reduce time correlation in learning samples. The algorithm's ability to operate with multiple environment instances in parallel enhanced computational efficiency and significantly improved the learning process. By considering cooperative BSs that could have fetched content from neighboring BSs or the backbone network, the proposed method effectively reduced data traffic and transmission costs in 5G networks. The empirical results demonstrated the superiority of the A3C-based algorithm over classical caching policies such as least recently used, least frequently used, and adaptive replacement cache, showcasing lower transmission costs and faster convergence rates. One limitation of this study was the reliance on accurate and timely updates of content popularity distributions. While the paper assumed that content popularity followed a Zipf distribution and varied over time, the accuracy of these assumptions could have significantly impacted the performance of the caching algorithm. In real-world applications, user preferences and content popularity could have changed unpredictably, and any deviations from the assumed distribution could have affected the effectiveness of the caching policy. Ensuring robust performance under varying content popularity distributions was crucial for the practical implementation of the proposed method.

Table 17 organizes the papers discussed in this section, offering a domain-specific breakdown of the research conducted in the A3C area.

Overview of A2C applications in the literature

In [244], the authors introduced an innovative approach to low-latency task scheduling in edge computing environments, addressing several significant challenges inherent in such settings. The primary focus of the paper was on integrating a hard attention mechanism with the A2C algorithm to enhance task scheduling efficiency and reduce latency. The strengths of the Hard Attention Mechanism and Advantage Actor-Critic (HA-A2C) method lie in its ability to significantly reduce task latency by approximately 40% compared to the DQN method. The hard attention mechanism employed by HA-A2C was particularly effective in reducing computational complexity and increasing efficiency, allowing the model to process tasks more quickly. Additionally, the method showcased improved scalability, maintaining low task latency even as the number of tasks increased. The use of the A2C algorithm, which combined policy gradient and value function estimates, enhanced the stability and effectiveness of the policy network, further contributing to the overall performance of the model. However, there were some limitations to the HA-A2C approach. One notable weakness was the potential complexity of implementing the hard attention mechanism in real-world scenarios, where the dynamic and heterogeneous nature of edge environments might have posed additional challenges. Furthermore, while the HA-A2C method outperformed other DRL methods in terms of latency, it might have still faced difficulties in scenarios with extremely high-dimensional states and action spaces, where further optimization might have been necessary. Another consideration was the

ARTICLE IN PRESS

reliance on accurate and timely data for effective attention allocation, which might not have always been feasible in practical applications.

A task scheduling mechanism in cloud-fog environments that leveraged the A2C algorithm was presented in [245]. This approach aimed to optimize the scheduling process for scalability, reliability, trust, and makespan efficiency. One of the significant strengths of the paper was the holistic approach it took toward task scheduling in heterogeneous cloud-fog environments. The use of the A2C algorithm was particularly effective in handling the dynamic nature of task scheduling, as it allowed the scheduler to make real-time decisions based on the current state of the system. By dynamically adjusting the number of virtual machines according to workload demands, the proposed scheduler ensured efficient resource utilization, which was crucial for maintaining system performance under varying conditions. One limitation was the reliance on specific system parameters and assumptions about the environment. The proposed method assumed accurate estimation of factors such as task priorities and virtual machine capacities, which might not have always been feasible in practical applications. Deviations from these assumptions could have impacted the performance and robustness of the scheduling algorithm. Further research could have explored the robustness of the proposed approach under more relaxed assumptions and in diverse real-world scenarios.

Authors in [246] introduced an A2C-learning-based framework for optimizing beam selection and transmission power in mmWave networks. This approach aimed to improve energy efficiency while maintaining coverage in dynamic and complex network environments. A notable strength of the paper was its innovative application of the A2C algorithm for joint optimization of beam selection and transmission power. This dual optimization was particularly effective in addressing the significant challenge of energy consumption in mmWave networks. By leveraging A2C, the proposed method dynamically adjusted beam selection and power levels based on the current state of the network, which was represented by the Signal-to-Interference-plus-Noise Ratio (SINR) values. The use of A2C ensured stable and efficient learning through policy gradients and value function approximations, making it suitable for real-time applications. One of the limitations was the assumption of specific system parameters and environmental conditions. The method assumed accurate estimation of SINR values and predefined beam angles, which might not have always been feasible in practical applications. Deviations from these assumptions could have impacted the performance and robustness of the optimization algorithm. Future research could have explored the robustness of the proposed method under more relaxed assumptions and in diverse real-world scenarios.

Authors explored the use of the A2C algorithm to estimate the power delay profile (PDP) in 5G New Radio environments in [32]. This approach aimed to enhance channel estimation performance by leveraging DRL techniques. A notable strength of this paper was its innovative application of the A2C algorithm to the problem of PDP estimation. By framing the estimation problem within an RL context, the proposed method directly targeted the minimization of Mean Squared Error in channel estimation, rather than aiming to approximate an ideal PDP. This pragmatic approach allowed the algorithm to adapt to the inherent approximations and imperfections in practical channel estimation processes, leading to improved performance. However, the complexity of implementing the A2C algorithm in real-world scenarios posed challenges. The need for extensive training and parameter tuning required significant computational resources and expertise in RL, which might not have been readily available in all settings. Additionally, while the simulation results were promising, further validation in real-world deployments was necessary to fully assess the robustness and practicality of the proposed approach. Real-world environments could have introduced additional challenges, such as varying traffic patterns and hardware constraints, which were not fully captured in simulations.

The application of various actor-critic algorithms to develop a trading agent for the Indian stock market was investigated in [247]. The study evaluated the performance of PPO, DDPG, A2C, and Twin Delayed DDPG (TD3) algorithms in making trading decisions. One of the primary strengths of this paper was its comprehensive approach to evaluating multiple actor-critic algorithms in a real-world financial trading context. By considering different algorithms, the authors provided a broad perspective on the effectiveness of it in stock trading. The use of historical stock data from the Yahoo Finance API, covering a substantial period (2006-2021), ensured that the models were tested against diverse market conditions, enhancing the robustness and reliability of the results. The study's focus on a single market (the Indian stock market) might have limited the generalizability of the findings. Future research could have explored the application of these algorithms in different financial markets to ensure broader applicability.

The authors presented an innovative framework for optimizing task segmentation and parallel scheduling in edge computing networks using the A2C algorithm in [248]. The approach focused on

ARTICLE IN PRESS

minimizing total task execution delay by splitting multiple computing-intensive tasks into sub-tasks and scheduling them efficiently across different edge servers. A key strength of this paper was its holistic approach to task segmentation and scheduling. By jointly optimizing both processes, the proposed method ensured that tasks were not only divided efficiently but also assigned to the most suitable edge servers for processing. This joint optimization was crucial in dynamic edge computing environments, where both computation capacity and task requirements could have varied significantly over time. The use of A2C allowed the system to adapt to these changes in real-time, enhancing overall system performance. The authors' method of decoupling the complex mixed-integer non-convex problem into more manageable sub-problems was another strength. By first addressing the task segmentation problem and then tackling the sub-tasks parallel scheduling, the paper presented a structured and logical approach to solving the optimization challenge. The introduction of the optimal task split ratio function and its integration into the A2C algorithm further enhanced the efficiency and effectiveness of the proposed solution.

Researchers in [249] presented an innovative approach to enhancing multi-UAV obstacle avoidance using A2C combined with an experience-sharing mechanism. This method aimed to optimize obstacle avoidance strategies in complex, dynamic environments by sharing positive experiences among UAVs to expedite the training process. One of the key strengths of this paper was the introduction of the experience-sharing mechanism to the A2C algorithm. This mechanism significantly enhanced the efficiency and robustness of the training process by allowing UAVs to share positive experiences. This collective learning approach accelerated the convergence of the algorithm, enabling UAVs to quickly learn effective obstacle avoidance strategies. The experience-sharing mechanism was particularly valuable in multi-agent systems, where individual agents could have benefited from the knowledge gained by others, leading to faster and more robust learning. However, the experience-sharing mechanism relied on consistent and reliable communication between UAVs. In practical applications, communication constraints and network reliability issues could have significantly impacted the effectiveness of this mechanism. Inter-UAV communication latency and packet loss could have led to outdated or incomplete information being shared, thereby reducing the overall efficiency of the learning process. Also, the method assumed a certain level of accuracy in modeling the environment and the dynamic obstacles within it. Any deviations from these assumptions, such as unexpected changes in obstacle behavior or environmental conditions, could have affected the performance and robustness of the algorithm. Real-world environments were inherently unpredictable, and the algorithm must have been tested extensively in diverse scenarios to ensure its reliability.

A robust approach to visual navigation using an actor-critic method enhanced with Generalized Advantage Estimation (GAE) was developed by the authors in [250]. This method demonstrated significant strengths in terms of learning efficiency and stability, as well as effective navigation in complex environments like ViZDoom. One major strength of this approach was its ability to rapidly converge and achieve high performance in both basic and complex visual navigation tasks. By employing the A2C method with GAE, the algorithm reduced variance in policy gradient estimates, leading to more stable learning. This was particularly evident in the ViZDoom health gathering scenarios, where the A2C with GAE agent achieved the highest scores with lower variance compared to other methods. Additionally, the use of multiple processes in the A2C method significantly reduced training time, making it more efficient than traditional DQN approaches. However, the method also had notable limitations. One significant drawback was the high computational cost associated with using multiple processes for training, which might not have been feasible in resource-constrained environments. Furthermore, while the approach performed well in the tested ViZDoom scenarios, its generalizability to other, more diverse environments remained uncertain without further validation. The reliance on visual inputs also presented challenges in environments with varying lighting conditions or visual obstructions, which were not extensively tested in this study. Another limitation was the potential for over-fitting to specific task environments. The training setup in controlled ViZDoom scenarios might not have fully captured the complexities of real-world navigation tasks, where environmental dynamics were less predictable. Thus, while the A2C with GAE approach showed promise, its applicability to a broader range of visual navigation tasks would have benefited from additional research and testing in more varied and less controlled environments.

Table 18 summarizes the discussed papers in this section. In the following subsection, we will discuss the Deterministic Policy Gradient (DPG) algorithm, which addresses the challenges associated with **continuous action spaces**.

ARTICLE IN PRESS

Table 18: A2C Paper Review by Domain

Application Domain	References
Edge computing environments	[244], [248]
Network Optimization	[245],[32]
Cloud-based Control and Encryption Systems	[245]
Energy and Power Management (IoT Networks, Smart Energy Systems)	[246]
Financial Applications	[247]
Autonomous UAVs	[249]
Visual Navigation	[250]

5.2 Deterministic Policy Gradient (DPG)

DPG addresses the challenges associated with continuous action spaces and offers significant improvements in sample efficiency over stochastic policy gradient methods. Traditional policy gradient methods in RL use stochastic policies, where the policy $\pi_\theta(a|s)$ is a probability distribution over actions given a state, parameterized by θ. These methods rely on sampling actions from this distribution to compute the policy gradient, which can be computationally expensive and sample inefficient, especially in high-dimensional action spaces [1, 251]. In contrast, the DPG algorithm uses a deterministic policy, denoted by $\mu_\theta(s)$, which directly maps states to actions without involving any randomness. The policy gradient theorem for deterministic policies shows that the gradient of the expected return with respect to the policy parameters can be computed as [251]:

$$\nabla_\theta J(\mu_\theta) = \mathbb{E}_{s\sim\rho^\mu}\left[\nabla_\theta \mu_\theta(s) \nabla_a Q^\mu(s,a)\big|_{a=\mu_\theta(s)}\right] \tag{42}$$

where $Q^\mu(s,a)$ is the action-value function under the deterministic policy μ_θ, and ρ^μ is the discounted state visitation distribution under μ_θ.

By using an off-policy learning approach, DPG ensures adequate exploration while learning a deterministic target policy. A behavior policy, typically a stochastic policy, is employed to generate exploratory actions and collect experiences. The gradients obtained from these experiences are then used to update the deterministic policy. Since the same experiences can be reused to improve the policy in this non-policy setting, the data collected can be utilized more efficiently than in a policy-based setting [252]. The DPG algorithm is generally implemented within an actor-critic framework, where the actor represents the deterministic policy μ_θ, and the critic estimates the action-value function $Q^\mu(s,a)$. The critic is trained using TD learning to minimize the Bellman error:

$$\delta_t = r_t + \gamma Q^\mu(s_{t+1}, \mu_\theta(s_{t+1})) - Q^\mu(s_t, a_t) \tag{43}$$

where δ_t is the TD error, r_t is the reward, γ is the discount factor, and s_t, a_t are the state and action at time step t. The actor updates the policy parameters in the direction suggested by the critic:

$$\theta \leftarrow \theta + \alpha \nabla_\theta \mu_\theta(s_t) \nabla_a Q^\mu(s_t, a_t)\big|_{a=\mu_\theta(s_t)} \tag{44}$$

where α is the learning rate for the actor. A variant of DPG, which is designed to handle continuous action spaces with the help of DL, is analyzed in the next subsection in detail.

5.2.1 Deep Deterministic Policy Gradient (DDPG)

The DDPG algorithm is an extension of the DPG method, designed to handle continuous action spaces effectively, introduced by [253]. DDPG leverages the power of DL to address the challenges associated with high-dimensional continuous control tasks [253]. The foundation of DDPG lies in the DPG algorithm. This approach contrasts with stochastic policy gradients, which sample actions from a probability distribution. The deterministic nature of DPG reduces the variance of gradient estimates and improves sample efficiency, making it suitable for continuous action spaces. DDPG employs an actor-critic architecture, where the actor network represents the policy $\mu(s|\theta^\mu)$ and the critic network estimates the action-value function $Q(s,a|\theta^Q)$. The actor network outputs a specific action for a given state, while the critic network evaluates the action by estimating the expected return. The policy gradient is computed using the chain rule:

ARTICLE IN PRESS

$$\nabla_{\theta^\mu} J \approx \mathbb{E}_{s\sim\rho^\beta}\left[\nabla_a Q(s,a|\theta^Q)\big|_{a=\mu(s|\theta^\mu)} \nabla_{\theta^\mu}\mu(s|\theta^\mu)\right] \tag{45}$$

where ρ^β denotes the state distribution under a behavior policy β. To stabilize learning and address the challenges of training with large, non-linear function approximators, DDPG incorporates two key techniques from the DQN algorithm [8]:

1. Replay Buffer: A replay buffer stores transitions (s_t, a_t, r_t, s_{t+1}) observed during training. By sampling mini-batches of transitions from this buffer, DDPG minimizes the correlations between consecutive samples, which stabilizes training and improves efficiency.
2. Target Networks: DDPG uses target networks for both the actor and critic, which are periodically updated with a soft update mechanism. The target networks provide stable targets for the Q-learning updates, reducing the likelihood of divergence:

$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'} \tag{46}$$

$$\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'} \tag{47}$$

where $\tau < 1$ is the target update rate.

Exploration in continuous action spaces is crucial for effective learning. DDPG employs an exploration policy by adding noise to the actor's deterministic policy. An Ornstein-Uhlenbeck process [254] is typically used to generate temporally correlated noise, promoting exploration in environments with inertia.

The DDPG algorithm operates as follows: As shown in Alg. 26, first, the parameters of the actor network θ^μ and the critic network θ^Q are initialized [255] (line 1-2). Target networks for both the actor and critic are also initialized. Then, multiple agents interact with their respective environments, collecting transitions (s_t, a_t, r_t, s_{t+1}) which are stored in a replay buffer (lines 3-10). For each agent, the actor selects actions based on the current policy with added exploration noise. The critic network is updated using the Bellman equation to minimize the TD error (lines 11-13). The actor network is updated using the policy gradient derived from the critic (line 14). Periodically, the target networks are updated to slowly track the learned networks (line 15). Over the next paragraphs, we analyze some of the papers in the literature that used DDPG.

Authors in [256] presented an innovative method for developing a missile lateral acceleration control system using the DDPG algorithm. This study reframed the autopilot control problem within the RL context, utilizing a 2-degrees-of-freedom nonlinear model of the missile's longitudinal dynamics for training. One strength was the incorporation of performance metrics such as settling time, undershoot, and steady-state error into the reward function. By integrating these key performance indicators, the authors ensured that the trained agent not only learned to control the missile effectively but also adhered to desirable performance standards. This approach enhanced the practical applicability of the method, ensuring that the control system met operational requirements. The method's scalability to more complex scenarios and larger-scale implementations was another concern. The increased number of states and potential interactions in a real-world missile control system could have introduced additional complexities, making it challenging to maintain the same level of performance. Further research was needed to explore the scalability of the approach and develop mechanisms to manage the increased computational load.

A novel approach to fast charging lithium-ion batteries using a combination of a model-based state observer and a DRL optimizer, specifically the DDPG algorithm, was proposed in [257]. This method aimed to balance charging rapidity with the enforcement of thermal safety and degradation constraints. One of the notable strengths of this paper was its innovative application of the DDPG algorithm to the complex problem of fast charging lithium-ion batteries. By formulating a multi-objective optimization problem that included penalties for over-temperature and degradation, the authors effectively addressed the crucial aspects of battery safety and longevity. This approach ensured that the charging strategy not only accelerated the charging process but also maintained the battery within safe operating limits, thus extending its life. The method's scalability to different types of batteries and charging environments was a concern. While the results were promising for the specific battery model used in the study, the ability to generalize the approach to other battery types and configurations remained uncertain. The increased number of variables and potential interactions in more complex systems could have introduced additional complexities, making it challenging to

ARTICLE IN PRESS

Algorithm 26 DDPG [253]

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ.
Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$.
Initialize replay buffer R.
for episode = 1 to M **do**
    Initialize a random process N for action exploration.
    Receive initial observation state s_1.
    for t = 1 to T **do**
        Select action $a_t = \mu(s_t|\theta^\mu) + N_t$ according to the current policy and exploration noise
        Execute action a_t and observe reward r_t and observe new state s_{t+1}
        Store transition (s_t, a_t, r_t, s_{t+1}) in R
        Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
        Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$.
        Update critic by minimizing the loss: $L = \frac{1}{N}\sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$.
        Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \times \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

        Update the target networks:

$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}$$
$$\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'}$$

    end for
end for

maintain the same level of performance. Further research was needed to explore the scalability of the approach and develop mechanisms to manage the increased computational load.

Authors explored the application of DDPG to the problem of obstacle avoidance in the trajectory planning of a robot arm in [258]. The authors proposed using DDPG to plan the trajectory of a robot arm, ensuring smooth and continuous motion while avoiding obstacles. The rewards were specifically designed to overcome the convergence difficulties posed by multiple and potentially antagonistic rewards. One strength of this paper was the careful design of the reward function. By considering both the distance to the target and the proximity to obstacles, the authors ensured that the robot not only reached its goal efficiently but also avoided collisions. This multi-faceted reward structure addressed the convergence issues often encountered in RL tasks with multiple objectives. The empirical validation through simulations in the MuJoCo environment demonstrated the effectiveness of the proposed method, showing that the robot arm could successfully navigate to its target while avoiding obstacles. The method's scalability to more complex scenarios and larger-scale implementations might have impeded the achieved performance. The increased number of states and potential interactions in a real-world robotic system could have introduced additional complexities, making it challenging to maintain the same level of performance. Further research was needed to explore the scalability of the approach and develop mechanisms to manage the increased computational load. Investigating the impact of various hyperparameters and network architectures on the performance of the DDPG algorithm could have provided deeper insights into optimizing the method for different robotic applications.

An enhanced version of DDPG, integrating a Long Short-Term Memory (LSTM) network-based encoder to handle dynamic obstacle avoidance for mobile robots in stochastic environments, is introduced in [259]. One of the notable strengths of this paper was the innovative combination of DDPG with LSTM. This hybrid approach allowed the robot to encode a variable number of obstacles into a fixed-length representation, which addressed the limitation of traditional DDPG that required a fixed number of inputs. By utilizing the LSTM network-based encoder, the proposed method could effectively process and integrate dynamic environmental information, enhancing the robot's adaptability to unpredictable scenarios. The reliance on accurate environmental sensing and real-time data

ARTICLE IN PRESS

processing was a potential limitation. The performance of the LSTM-based encoder and the overall DDPG framework heavily depended on the quality and accuracy of the sensor data. In real-world applications, factors such as sensor noise, varying environmental conditions, and communication delays could have affected the reliability and robustness of the system. Ensuring robust performance under diverse and unpredictable conditions remained a critical challenge.

DDPG combined with prioritized sampling to optimize power control in wireless communication systems, specifically targeting Multiple Sweep Interference (MSI) scenarios, was designed in [260]. Prioritized sampling was another innovative aspect of the paper. By focusing on more valuable experiences during training, the algorithm accelerated the learning process and improved convergence speed. The empirical results showed that the DDPG scheme with prioritized sampling (DDPG-PS) outperformed the traditional DDPG scheme with uniform sampling and the DQN scheme. This was evident in various MSI scenarios, where the DDPG-PS scheme achieved better reward performance and stability. The scalability of the proposed method to more complex scenarios and larger-scale implementations was another concern. While the results were promising in simulated environments, the ability to handle a broader range of interference patterns and larger numbers of channels remained uncertain. The increased number of states and potential interactions could have introduced additional complexities, making it challenging to maintain the same level of performance. Further research was needed to explore the scalability of the approach and develop mechanisms to manage the increased computational load.

Authors in [261] employed DDPG to model the bidding strategies of generation companies in electricity markets. This approach was aimed at overcoming the limitations of traditional game-theoretic methods and conventional RL algorithms, particularly in environments characterized by incomplete information and high-dimensional continuous state/action spaces. One significant strength was the ability of the proposed method to converge to the Nash Equilibrium even in an incomplete information environment. Traditional game-theoretic methods often required complete information and were limited to static games. In contrast, the DDPG-based approach could dynamically simulate repeated games and achieve stable convergence, demonstrating its robustness in modeling real-world market conditions. One limitation was the reliance on accurate modeling of market conditions and real-time data processing. The effectiveness of the proposed method depended heavily on the precision of the input data, such as nodal prices and load demands. In real-world applications, factors such as data inaccuracies, communication delays, and varying environmental conditions could have impacted the reliability and robustness of the system. Ensuring robust performance under diverse and unpredictable conditions remained a critical challenge that needed to be addressed. Additionally, the study assumed a specific structure for the NNs used in the actor and critic models. The performance of the algorithm could have been sensitive to the choice of network architecture and hyperparameters. A more systematic exploration of different architectures and their impact on performance could have provided deeper insights into optimizing the DDPG algorithm for electricity market modeling.

An advanced approach for resource allocation in vehicular communications using a multi-agent DDPG algorithm was studied in [262]. This method was designed to handle the dynamic and high-mobility nature of vehicular environments, specifically targeting the optimization of the sum rate of Vehicle-to-Infrastructure (V2I) communications while ensuring the latency and reliability of Vehicle-to-Vehicle (V2V) communications. One of the significant strengths was the formulation of the resource allocation problem as a decentralized Discrete-time and Finite-state MDP. This approach allowed each V2V communication to act as an independent agent, making decisions based on local observations without requiring global network information. This decentralization was crucial for scalability and real-time adaptability in high-mobility vehicular environments. One potential limitation was the reliance on accurate and timely acquisition of CSI. In high-mobility vehicular environments, obtaining precise CSI could have been challenging due to fast-varying channel conditions. Any inaccuracies in CSI could have impacted the performance and robustness of the proposed resource allocation scheme. Ensuring robust performance under diverse and unpredictable conditions remained a critical challenge. The last algorithm in the category of actor-critic methods to analyze is TD3, which is an enhancement of the DDPG algorithm, designed to address the issues of overestimation bias. This algorithm is analyzed in detail in the next subsection.

Table 19 provides a summary of the analyzed papers based on their domain.

ARTICLE IN PRESS

Table 19: DDPG Papers Review

Application Domain	References
Theoretical Research (Convergence, stability)	[253]
Missile Control Systems	[256]
Battery Charging Optimization	[257]
Robotics	[258], [259]
Network Optimization	[260]
Financial Applications	[261]
Multi-agent Systems and Autonomous Behaviors	[262]

5.2.2 Twin Delayed Deep Deterministic Policy Gradient (TD3)

TD3 is an enhancement of the DDPG algorithm, introduced by [263], designed to address overestimation bias in function approximation within actor-critic methods. TD3 incorporates several innovative techniques to improve the stability and performance of continuous control tasks in RL. Overestimation bias arises when the value estimates for certain actions are consistently higher than their true values due to function approximation errors. This issue is well-recognized in value-based methods such as Q-learning [264]. In actor-critic methods, this bias can result in suboptimal policy updates and divergent behavior [263].

TD3 builds on the concept of double Q-learning, which mitigates overestimation bias by maintaining two separate value estimators and using the minimum of the two estimates for the target update [167]. This approach is adapted to the actor-critic framework by utilizing two critic networks, Q_{θ_1} and Q_{θ_2}, which are trained independently. The target value is computed as:

$$y = r + \gamma \min_{i=1,2} Q_{\theta_i'}(s', \mu_{\theta'}(s')) \tag{48}$$

where $Q_{\theta_i'}$ are the target critic networks and $\mu_{\theta'}$ is the target actor network.

To further minimize error propagation from the critic to the actor, TD3 delays the policy updates in relation to the value updates. Specifically, the policy (actor) network is updated less frequently than the critic networks. This strategy ensures that the value estimates used for policy updates are more accurate and stable. The policy is updated every d iterations, where d is a hyperparameter typically set to 2 or higher [263].

To avoid overfitting to narrow peaks in the value estimates, TD3 introduces target policy smoothing. This technique adds noise to the target policy, promoting smoother value estimates and fostering more robust policy learning. The target value is computed with added noise:

$$y = r + \gamma \min_{i=1,2} Q_{\theta_i'}(s', \mu_{\theta'}(s') + \epsilon) \tag{49}$$

where ϵ is clipped noise sampled from a Gaussian distribution. Based on Alg. 27, the TD3 algorithm operates as follows:

First, the parameters of the actor network θ and the critic networks θ_1 and θ_2 are initialized (line 1). Target networks for both the actor and critics are also initialized (line 2). Multiple agents interact with their respective environments, collecting transitions (s_t, a_t, r_t, s_{t+1}) which are stored in a replay buffer (line 6). The actor selects actions with added exploration noise (line 8). The critic networks are updated by minimizing the TD error using the clipped double Q-learning target (lines 9-10). The actor network is updated using the deterministic policy gradient, but only every d iterations (line 12). Periodically, the target networks are updated to slowly track the learned networks (line 13).

Authors in [265], as the first analyzed paper in the literature, showcased both strengths and limitations in applying the TD3 algorithm for energy management in Hybrid EVs (HEVs). One of the primary strengths of the paper was its innovative application of the TD3 algorithm, which enhanced training efficiency and stability over the previously used methods like Q-learning, DQN, and DDPG. The TD3 algorithm's use of two critic networks helped provide more stable training by mitigating the overestimation bias. The paper also demonstrated significant improvements in fuel economy and battery state-of-charge sustainability, which are crucial metrics for HEVs. By comparing the performance under various driving cycles, the authors provided comprehensive evidence of the TD3 algorithm's effectiveness in real-world scenarios. However, there were notable limitations. The implementation of TD3, while improving stability, introduced complexity in the training process,

ARTICLE IN PRESS

Algorithm 27 TD3 [263]

Initialize critic networks Q_{θ_1}, Q_{θ_2}, and actor network π_ϕ with random parameters θ_1, θ_2, ϕ
Initialize target networks $\theta_1' \leftarrow \theta_1$, $\theta_2' \leftarrow \theta_2$, $\phi' \leftarrow \phi$
Initialize replay buffer $\mathcal{B}$
for $t = 1$ to T **do**
    Select action with exploration noise $a \sim \pi_\phi(s) + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma)$ and observe reward r and new state s'
    Store transition tuple (s, a, r, s') in $\mathcal{B}$
    Sample mini-batch of N transitions (s, a, r, s') from $\mathcal{B}$
    $\tilde{a} \leftarrow \pi_{\phi'}(s') + \epsilon$, $\epsilon \sim \text{clip}(\mathcal{N}(0, \sigma), -c, c)$
    $y \leftarrow r + \gamma \min_{i=1,2} Q_{\theta_i'}(s', \tilde{a})$
    Update critics $Q_{\theta_i} \leftarrow \arg\min_{\theta_i} N^{-1} \sum (y - Q_{\theta_i}(s, a))^2$
    if $t \mod d = 0$ **then**
        Update ϕ by the deterministic policy gradient: $\nabla_\phi J(\phi) = N^{-1} \sum \nabla_a Q_{\theta_1}(s, a)|_{a=\pi_\phi(s)} \nabla_\phi \pi_\phi(s)$
        Update target networks:

$$\theta_i' \leftarrow \tau\theta_i + (1 - \tau)\theta_i'$$
$$\phi' \leftarrow \tau\phi + (1 - \tau)\phi'$$

    end if
end for

requiring careful tuning of hyperparameters to achieve optimal performance. The algorithm's reliance on extensive computational resources for training might have limited its practical applicability in scenarios with constrained resources. Additionally, the paper focused primarily on the simulation results without providing sufficient real-world testing to validate the algorithm's performance under actual driving conditions. This gap raised questions about the robustness of the proposed method when deployed in a real-world environment.

The application of the TD3 algorithm for the target tracking of UAVs was proposed in [266]. The authors integrated several enhancements into the TD3 framework to improve its performance in handling the high nonlinearity and dynamics of UAV control. A significant strength was the novel reward formulation that incorporated exponential functions to limit the effects of velocity and acceleration on the policy function approximation. This approach prevented deformation in the policy function, leading to more stable and robust learning outcomes. Additionally, the concept of multistage training, where the training process was divided into stages focusing on position, velocity, and acceleration sequentially, enhanced the learning efficiency and performance of the UAV in tracking tasks. However, the proposed method also had several limitations. The integration of a PD controller and the novel reward formulation added to the complexity of the training process. The scalability of the proposed method to more complex environments with a higher number of dynamic obstacles or more sophisticated UAV maneuvers was another concern. While the results were promising in the tested scenarios, the ability to handle a broader range of operational conditions and larger numbers of UAVs remained uncertain. The increased number of states and potential interactions could have introduced additional complexities, making it challenging to maintain the same level of performance.

A novel dynamic MsgA channel allocation strategy using TD3 to mitigate the issue of MsgA channel collisions in low-earth-orbit satellite communication systems is investigated in [267]. The paper's approach to dynamically pre-configuring the mapping relationship between PRACH occasions and PUSCH occasions based on historical access information was another strong point. This method allowed the system to adapt to changing access demands effectively, ensuring efficient use of available resources and reducing collision rates. The empirical results were impressive, demonstrating a 39.12% increase in access success probability, which validated the effectiveness of the proposed strategy. The scalability of the proposed method to larger and more complex satellite networks was one of the concerns. While the results were promising in the tested scenarios, the ability to handle a broader range of interference patterns and a larger number of users remained uncertain. The increased number of states and potential interactions could have introduced additional complexities, making it challenging to maintain the same level of performance. Further research was needed to explore the scalability of the approach and develop mechanisms to manage the increased computational load.

ARTICLE IN PRESS

A TD3-based method for optimizing voltage and reactive power in distribution networks with high penetration of Distributed Energy Resources (DERs) such as battery energy storage and solar photovoltaic units was investigated in [268]. The authors' approach of coordinating the reactive power outputs of fast-responding smart inverters and the active power of battery ESS enhanced the overall efficiency of the network. By carefully designing the reward function to ensure a proper voltage profile and effective scheduling of reactive power outputs, the method optimized both voltage regulation and power loss minimization. The results demonstrated that the TD3-based method outperformed traditional methods such as local droop control and DDPG-based approaches, showing significant improvements in reducing voltage fluctuations and minimizing power loss in the IEEE 34- and 123-bus test systems. The scalability of the proposed method to larger and more complex distribution networks was another concern. While the results were promising in the tested IEEE 34- and 123-bus systems, the ability to handle a broader range of network configurations and a larger number of DERs remained uncertain. The increased number of states and potential interactions could have introduced additional complexities, making it challenging to maintain the same level of performance. Further research was needed to explore the scalability of the approach and develop mechanisms to manage the increased computational load.

Authors in [269] presented an innovative approach to quadrotor control, leveraging the TD3 algorithm to address stabilization and position tracking tasks. This study was notable for its application to handle the complex, non-linear dynamics of quadrotor systems. The authors' method of integrating target policy smoothing, twin critic networks, and delayed updates of value networks enhanced the learning efficiency and reduced variance in the policy updates. This comprehensive approach ensured that the quadrotor could achieve precise control in both stabilization and position tracking tasks. The empirical results demonstrated the effectiveness of the TD3-based controllers, showcasing significant improvements in achieving and maintaining target positions under various initial conditions. The scalability of the proposed method to more complex environments with dynamic obstacles and more sophisticated maneuvers was another concern. While the results were promising in the tested scenarios, the ability to handle a broader range of operational conditions and larger-scale implementations remained uncertain. The increased number of states and potential interactions could have introduced additional complexities, making it challenging to maintain the same level of performance. Further research was needed to explore the scalability of the approach and develop mechanisms to manage the increased computational load.

A real-time charging navigation method for multi-autonomous underwater vehicle (AUVs) systems using the TD3 algorithm was designed in [270]. This method was designed to improve the efficiency of navigating AUVs to their respective charging stations by training a trajectory planning model in advance, eliminating the need for recalculating navigation paths for different initial positions and avoiding dependence on sensor feedback or pre-arranged landmarks. The primary strength of this paper lies in its application of the TD3 algorithm to the multi-AUV charging navigation problem. By training the trajectory planning model in advance, the method significantly improved the real-time performance of multi-AUV navigation. However, the paper also had some limitations. One major limitation was the reliance on the accuracy of the AUV motion model and the assumptions made during its formulation. For instance, the model assumed constant velocity and neglected factors like water resistance and system delays, which could have affected the real-world applicability of the results. Moreover, the simulation environment used for training and testing might not have fully captured the complexities and variabilities of real underwater environments. Another potential limitation was the need for extensive computational resources for training the TD3 model, especially given the high number of training rounds (up to 6000) and the large experience replay buffer size.

Authors in [271] presented an advanced approach for Adaptive Cruise Control (ACC) using the TD3 algorithm. This method addressed the complexities of real-time decision-making and control in automotive applications. The authors carefully designed the reward function to consider the velocity error, control input, and additional terms to ensure stability and smooth driving behavior. This reward structure allowed the algorithm to learn an optimal policy that maintained safe distances between vehicles while adapting to changing traffic conditions. The empirical results demonstrated the effectiveness of the TD3-based ACC system in both normal and disturbance scenarios, highlighting its robustness and adaptability. The scalability of the proposed method to more complex driving scenarios and larger-scale implementations was another concern. While the results were promising in the tested scenarios, the ability to handle a broader range of operational conditions and larger numbers of vehicles remained uncertain. The increased number of states and potential interactions could have introduced additional complexities, making it challenging to maintain the same level of

ARTICLE IN PRESS

Table 20: TD3 Paper Review by Domain

Application Domain	References
Energy and Power Management	[265]
Multi-agent Systems and Autonomous UAVs	[266], [270]
Network Resilience and Optimization	[267]
Energy and Power Management	[268]
Real-time Systems and Hardware Implementations	[269], [271]

performance. Further research was needed to explore the scalability of the approach and develop mechanisms to manage the increased computational load. Additionally, the study assumed a specific structure for the NNs used in the actor and critic models. The performance of the algorithm could have been sensitive to the choice of network architecture and hyperparameters. A more systematic exploration of different architectures and their impact on performance could have provided deeper insights into optimizing the TD3 algorithm for adaptive cruise control. Table 20 gives a detailed summary of the discussed papers and the domains of each paper.

5.2.3 Algorithmic Rationale and Selection Criteria: Actor-Critic and Continuous Control

Actor-Critic architectures combine the low-variance advantage of value-based methods with the continuous-action capabilities of policy-based methods. The decision to use specific actor-critic algorithms is largely dictated by whether the environment requires parallelized exploration or mitigation of function approximation bias.

- **A3C and A2C:** These algorithms are selected when wall-clock training time is the primary bottleneck. By deploying multiple agents in parallel environments, A3C/A2C decorrelates the training data without needing a memory-intensive replay buffer. Practitioners utilize these for fog computing, cloud scheduling, and edge networks [50, 248], where distributed data collection naturally aligns with the algorithmic architecture. A2C is often preferred over A3C when GPU utilization and synchronous batching are prioritized.
- **DDPG:** DDPG extends the DQN framework to continuous action spaces. It is chosen for deterministic physical control tasks, such as missile lateral acceleration or basic robotic arm manipulation [256, 258]. However, because it is highly sensitive to hyperparameter tuning and prone to overestimating Q-values, it is increasingly being replaced by its successors in highly unstable environments.
- **TD3:** TD3 is explicitly chosen when DDPG fails due to overestimation bias and policy instability. By utilizing twin critics and delaying policy updates, TD3 smooths out target networks and prevents the agent from exploiting narrow peaks in the value function. Practitioners select TD3 for the most unforgiving continuous control tasks, such as multi-AUV navigation, dynamic UAV target tracking, and autonomous adaptive cruise control [266, 270, 271], where precision and stability are absolute requirements.

6 Recent Trends in the Literature

To conclude, we highlight some of the recent trends in RL and specifically DRL in this section. We acknowledge that we are only covering a selection of the recent, significant trends, as the field is rapidly evolving, and a complete overview would require another dedicated paper. We begin by focusing on RLHF, which is increasingly used in Large Language Models (LLMs) to fine-tune language models [272]. We then address HRL, which introduces an inductive bias to the final policy by explicitly factorizing it into multiple levels. Later, we explore an interesting new trend, Offline RL, before covering transfer learning.

6.1 RL from Human Feedback (RLHF)

RLHF is an approach that incorporates human input to define and refine an agent's objectives, rather than relying solely on predefined reward functions. This method places RLHF at the intersection

ARTICLE IN PRESS

Table 21: Overview of Algorithms Across Different Application Domains

Application Domain	Algorithm(s)	References
Energy Efficiency and Power Management	MC, Q-learning, Double Q-learning, SARSA, Dyna-Q, AMF, DDQN, TRPO, Dueling DQN, PPO, A2C, TD3	[27], [153], [161], [162], [28], [165], [170], [112], [110], [188], [197], [29], [202], [208], [224], [228], [246], [265], [268]
Cloud-based Systems	SARSA, A3C, A2C	[47], [50], [240], [244], [248], [245]
Optimization	TD-learning DQN, DDQN, Dueling DQN, DDPG	[25], [179], [26], [199], [257]
Multi-agent Systems	SARSA, MCTS, Prioritized Sweeping, Dyna-Q, DDQN, A3C, TD3	[48], [49], [105], [117], [118], [194], [270]
Algorithmic RL	TD-learning, MC, SARSA, Prioritized Sweeping	[123], [122], [124], [125], [127], [168], [172], [91], [93], [95], [102]
General RL	TD-learning, Q-learning, DQN, Dueling DQN, TRPO, PPO	[130], [4], [141], [147], [177], [201], [222]
Robotics	MC, Q-learning, Dyna-Q, DDQN, Dueling DQN, REINFORCE, TRPO, DDPG	[14], [15], [16], [17], [23], [150], [152], [119], [190], [24], [192], [211], [223], [258], [259]
Financial Applications	Q-learning, A2C, DDPG	[40], [247], [261]
Games	TD-learning, SARSA, MCTS, Dyna-Q, DDQN, DQN, A3C	[10], [11], [12], [13], [126], [131], [144], [155], [88], [90], [92], [94], [96], [97], [98], [99], [37], [108], [113], [181], [182], [184], [241]
Signal Processing	TD-learning	[43]
Networks	TD-learning, Q-learning, SARSA, Dyna-Q, Prioritized Sweeping, DQN, Dueling DQN, REINFORCE, A3C, A2C, DDPG, TD3	[30], [151], [154], [169], [31], [116], [106], [178], [180], [200], [203], [210], [243], [245], [32], [260], [267]
ITS	Q-learning, SARSA, Prioritized Sweeping, Dyna-Q, DDQN, Dueling DQN, TRPO, PPO, A2C, DDPG	[18], [19], [20], [21], [22], [145], [149], [166], [171], [41], [196], [191], [193], [205], [114], [115], [204], [220], [42], [227], [229], [231], [242], [249], [250], [256], [262], [266]
Theoretical Research	Q-learning, TD-learning, Prioritized Sweeping, REINFORCE, TRPO, DDPG	[132], [133], [136], [137], [138], [139], [5], [101], [104], [207], [218], [253]
Dynamic Environments	TD-learning, Q-learning, Dyna-Q	[33], [134], [109], [34], [113]
Partially Observable Environments	TD-learning, SARSA	[35], [36], [156]
Real-time Systems and Hardware Implementations	Q-learning, DQN, DDQN, PPO, TD3	[142], [38], [148], [159], [164], [195], [230], [232], [39], [269], [271]
Benchmark Tasks	TD-learning	[107], [44]
Data Management and Processing	Q-learning, DQN, PPO	[146], [45], [163], [46]

of AI and human-computer interaction, offering a promising path to enhance the performance and adaptability of intelligent systems while aligning their objectives with human values [273, 274].

Traditional RL systems depend on engineered reward functions to guide agent behavior, which can be difficult to design accurately for complex tasks. RLHF addresses this by directly incorporating human feedback into the learning process, allowing agents to learn from human preferences and judgments. This approach has been crucial in training LLMs, where RLHF has been instrumental in guiding the model's capabilities toward human-aligned objectives [272, 275]. One of the most widely used algorithms in RLHF is PPO, which was discussed in subsection 4.3.

6.2 Hierarchical RL (HRL)

Similar to how DL uses hierarchical feature representations, HRL builds upon hierarchies of policies [276]. Early advancements in HRL introduced the concept of options, where policies go beyond simple, single-timestep actions to include multi-timestep behaviors that encapsulate other policies [277]. This structure allows higher-level policies to address broader goals while delegating more precise control to lower-level policies. Numerous studies in DRL have explored HRL by using a primary policy that

ARTICLE IN PRESS

selects among sub-policies. The segmentation of states or objectives into sub-policies is achieved either manually [278, 279] or through automated methods [280–282].

6.3 Offline RL

Offline RL focuses on using pre-collected datasets to train policies without further interaction with the environment [283]. This paradigm shift is particularly useful in situations where real-time data collection is costly, unsafe, or impractical, such as in healthcare, robotics, and autonomous driving systems [283, 284]. Also known as batch RL, offline RL relies on datasets generated from prior agent-environment interactions, which may contain suboptimal or noisy trajectories. Unlike online RL, where agents continuously interact with the environment to refine their policies, offline RL focuses on maximizing performance based solely on the static dataset. Key challenges in offline RL include distributional shift and extrapolation errors, where policies attempt to generalize beyond the state-action pairs seen in the dataset [285]. These problems arise because traditional RL algorithms assume continuous interaction with the environment and tend to overestimate the value of out-of-distribution actions. To address these limitations, various advancements, such as conservative Q-learning [286] and behavior-regularized Q-learning [287], have been proposed to constrain the learned policy, ensuring it stays close to the behavior policy underlying the dataset.

6.4 Transfer Learning

Transfer learning involves leveraging knowledge gained from solving one task to improve learning efficiency and performance on a different, but related task. In the context of RL, transfer learning addresses the challenges posed by training agents from scratch in complex environments, such as high computational costs, sparse rewards, and slow convergence rates [288]. The core principle of transfer learning is to reuse knowledge encapsulated in policies, value functions, or representations learned in a source task to speed up learning in a target task. Transfer learning techniques in RL can be divided into various strategies, such as transferring policies, reward functions, feature representations, or transition dynamics [288, 289]. Policy transfer directly involves reusing or fine-tuning policies learned in the source task, while representation transfer focuses on sharing latent features across tasks to improve generalization [290]. Additionally, model-based approaches allow for transferring environment dynamics, which can reduce the data requirements for learning in the target task [291]. A major challenge in transfer learning is the problem of negative transfer, where knowledge from the source task hinders performance on the target task due to significant differences between the tasks [292]. To mitigate this, methods like successor representations [293, 294] and generalized value functions [290] have been introduced to separate task-specific information from reusable knowledge structures.

ARTICLE IN PRESS

6.5 AI Ethics

RL agents optimize expected cumulative reward, but "what gets optimized" is only as ethical as the objective specification and the enforcement mechanisms around it. A large portion of ethical RL work therefore, operationalizes ethics either (i) deontologically, via explicit constraints, shielding, and runtime monitors that prevent forbidden behaviors, or (ii) consequentially, via reward design, shaping, and preference-based learning that try to encode value into the return [295–297]. Both pathways have well-known structural fragilities: rule-based approaches can be brittle under ambiguity and distribution shift, while scalar reward optimization can collapse plural moral considerations into a single proxy and invite reward hacking. A virtue-ethics lens reframes the target as policy-level dispositions, stable behavioral habits that persist when incentives, contexts, and interaction partners change, shifting evaluation from one-off constraint satisfaction or high return to trait-like summaries, robustness under interventions, and explicit accounting of ethical trade-offs (e.g., via multi-objective and risk-aware formulations) [298, 299].

A complementary ethical failure mode arises when the "reward" itself is socially constructed. In many alignment settings, the agent does not observe a ground-truth moral signal; instead, it receives feedback from humans or other evaluators whose judgments may be biased, lazy, or strategically sycophantic. In such social feedback environments, treating aggregate feedback as exogenous truth (as is commonly assumed in RLHF-style pipelines) can induce a structural misalignment where the learned objective separates from the latent ground truth (an "objective decoupling" that persists asymptotically) [297, 300]. Recent work formalizes this as a Social MDP and proposes Epistemic

ARTICLE IN PRESS

Source Alignment: rather than trusting consensus, the agent uses sparse internal axioms (e.g., safety spot-checks) to adjudicate evaluator reliability online and reweight feedback sources, enabling recovery even under majority bias [300]. Ethically, this emphasizes that responsible RL must not only encode values (or virtues) but also govern the epistemics of supervision—auditing who/what is being trusted, reporting how trade-offs are made, and demonstrating robustness of aligned behavior under feedback corruption and distributional shift [298, 301].

7 Future Directions and Open Challenges

Building on the foundational algorithms discussed, the landscape of RL is rapidly expanding toward resolving the "sim-to-real" gap and ensuring safe deployment in high-stakes environments. While RLHF serves as the primary mechanism for aligning models with human intent, the field is increasingly prioritizing *Offline RL* to leverage static, historical datasets $\mathcal{D} = \{(s_i, a_i, r_i, s'_i)\}$. This paradigm utilizes conservative value estimates to prevent the overestimation of out-of-distribution actions, a common failure mode in domains where live interaction is hazardous.

Beyond offline optimization, *World Models* and *HRL* are redefining how agents navigate long-horizon tasks [302]. Modern World Models learn a latent transition model $p(z_{t+1}|z_t, a_t)$, allowing the agent to "dream" trajectories and optimize its policy entirely within an internal state-space. This improves sample efficiency via a variational objective:

$$\mathcal{L}_{\text{world}} = \mathbb{E}_q \left[\sum_{t=1}^{T} \ln p(x_t|z_t) + \ln p(r_t|z_t) - D_{KL}(q(z_t|x_t) \| p(z_t|z_{t-1}, a_{t-1})) \right] \tag{50}$$

Simultaneously, HRL decomposes tasks into a multi-level structure where meta-controllers select goals for sub-policies to execute. These developments move the field toward "Generalist Agents" capable of transferring knowledge across disparate tasks with minimal retraining.

8 Discussion

Throughout this survey, we explored a variety of RL algorithms and their applications across several domains, including but not limited to robotics, ITS, games, wireless networks, and more. However, there is still much more to explore, both in terms of the number of papers analyzed and the diversity of algorithms. Several algorithms and methods were not covered in this survey for a range of reasons. The algorithms discussed here are those that have been applied across multiple domains and are more widely used in the research community. Additionally, due to time constraints, resources, and page limitations, it was not possible to analyze every algorithm and method in a single paper. Understanding these algorithms, however, provides insight into the different variations being regularly introduced by the research community.

The purpose of this survey is not to determine which algorithm is superior to others. As there is no universal solution to RL, it is not feasible to claim that "for problem X, algorithm Y performs better than others." This would require implementing new algorithms for the same problem and replicating the original work. Furthermore, results achieved using RL and specifically DRL may vary due to different extrinsic and intrinsic factors, as noted in [303], which makes direct comparisons difficult.

Finally, we strongly encourage readers to go through the chapters in this paper one by one, read the introductions to each algorithm, and, if necessary, consolidate their knowledge by reviewing the referenced papers. This will enable readers to gain insights from the analysis of the papers that used these algorithms. The tables provided are particularly useful for readers who are not interested in reading the entire paper. By summarizing key information in the tables, we offer a concise overview of the material presented throughout the survey.

Our goal has been to provide an overview of RL in terms of both theory and applications, offering a comprehensive understanding of various algorithms. This survey serves as a valuable resource for those wishing to deepen their knowledge of RL theory, as well as for readers interested in exploring how these algorithms are applied across different fields.

In the final section of the survey, we present a comprehensive table that highlights the application of RL algorithms across various domains in Table 21. Given the wide-reaching impact of RL, we have grouped these domains into broader categories to offer a more organized and concise overview. This categorization helps to effectively demonstrate the relevance and use of specific RL

ARTICLE IN PRESS

algorithms across different research areas while managing the constraints of space. The **Energy Efficiency and Power Management** category encompasses research areas such as train control, IoTs, WBAN, PID controllers, and smart energy systems, all of which focus on optimizing energy usage and improving power management. **Cloud-based systems** include works focused on cloud-based control and encryption systems, as well as edge computing environments, reflecting the growing importance of RL in managing and optimizing cloud resources. The **Optimization** category captures studies that leverage RL for solving complex optimization problems across various applications. **multi-agent systems** is a category that emphasizes RL's role in enabling autonomous behaviors, covering research involving shepherding, virtual agents, and other multi-agent Systems. **Algorithmic RL** covers advanced RL methodologies and hybrid approaches, including renewal theory, rough set theory, Bayesian RL, and MPC tuning. The General RL category encompasses broad RL applications, including policy learning, cybersecurity, and learning from raw experience. **Robotics** research, focusing on the application of RL in robotics, includes trajectory control, learning, routing, and more. In the **Financial Applications** category, studies on portfolio re-balancing and other financial strategies using RL are included. The **Games** category features research on game strategies in Chess, StarCraft, video games, and Card games, illustrating RL's success in complex strategic environments. signal processing research, which uses RL for signal processing and parameter estimation, is grouped under its own category. The **Networks** category covers studies focused on network reliability, blocking probabilities, optical transport networks, fog RAN, and network resilience and optimization. **ITS** includes RL applications in railway systems, EVs, intelligent traffic signal control, UAVs, and other transportation-related technologies. The **Theoretical Research** category includes studies focused on the theoretical aspects of RL, such as convergence and stability. **Dynamic Environments** research involves RL in environments like mazes, the Mountain Car problem, and Atari games. **Partially Observable Environments** includes studies on predictions, POMDPs, and Swarm intelligence in optimization problems. Research on applying RL in FPGA, Real-time Systems, and other hardware implementations is grouped under **Real-time Systems and Hardware Implementations**. Studies using benchmark tasks like Mountain Car and Acrobot to test RL algorithms are included in the **Benchmark Tasks** category. **Data Management and Processing** involves research applying RL in data management and processing environments, such as Hadoop and pathological image analysis. Finally, the object recognition category encompasses studies focusing on using RL for object recognition tasks.

Table 21 acts as a quick reference for researchers, helping them identify relevant work in their specific area of interest. It highlights the diversity and adaptability of RL algorithms across various domains. The categorization streamlines the information, making it more accessible and easier to understand the different applications of RL.

ARTICLE IN PRESS

9 Conclusion

This survey provided a thorough analysis of RL algorithms, categorizing them into value-based, policy-based, and actor-critic methods. Reviewing a wide range of research papers, it highlighted the strengths, weaknesses, and applications of each algorithm, offering valuable insights across various domains. From classical methods like Q-learning to advanced DRL and algorithmic variations tailored for specific domains, the paper presented a comprehensive overview. In addition to classifying RL algorithms based on model-free/model-based approaches, scalability, and sample efficiency, it also served as a practical guide for researchers and practitioners regarding the algorithms most suitable for different domains. Furthermore, this survey explored the practical implementation and performance of RL algorithms in fields such as games, robotics, autonomous systems, and more, providing a balanced assessment of their effectiveness.

Acknowledgements. This work was supported by the Natural Sciences and Engineering Research Council of Canada (NSERC) Discovery Grant, and Wilfrid Laurier University.

Declarations

Author Contribution.

- Majid Ghasemi: Conceptualization, Methodology, Writing – Original Draft, Investigation.
- Amir Hossein Moosavi: Writing – Review and Editing, Validation, Formal Analysis.

ARTICLE IN PRESS

- Dariush Ebrahimi: Supervision, Project Administration, Writing – Review and Editing, Funding Acquisition.

All authors reviewed and approved the final manuscript.

References

[1] Andrew, B., Richard S, S.: Reinforcement learning: an introduction (2018)

[2] Bellman, R.: The theory of dynamic programming. Bulletin of the American Mathematical Society **60**(6), 503–515 (1954)

[3] Ghasemi, M., Moosavi, A.H., Sorkhoh, I., Agrawal, A., Alzhouri, F., Ebrahimi, D.: An introduction to reinforcement learning: Fundamental concepts and practical applications. arXiv preprint arXiv:2408.07712 (2024)

[4] Sutton, R.S.: Learning to predict by the methods of temporal differences. Machine learning **3**, 9–44 (1988)

[5] Watkins, C.J.C.H.: Learning from delayed rewards. In: PhD Thesis, King's College, Cambridge (1989)

[6] Kaelbling, L.P., Littman, M.L., Moore, A.W.: Reinforcement learning: A survey. Journal of artificial intelligence research **4**, 237–285 (1996)

[7] Auer, P., Cesa-Bianchi, N., Fischer, P.: Finite-time analysis of the multiarmed bandit problem. Machine learning **47**(2), 235–256 (2002)

[8] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., *et al.*: Human-level control through deep reinforcement learning. Nature **518**(7540), 529–533 (2015)

[9] Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., *et al.*: Mastering the game of go with deep neural networks and tree search. nature **529**(7587), 484–489 (2016)

[10] Souchleris, K., Sidiropoulos, G.K., Papakostas, G.A.: Reinforcement learning in game industry—review, prospects and challenges. Applied Sciences **13**(4), 2443 (2023)

[11] Koyamada, S., Okano, S., Nishimori, S., Murata, Y., Habara, K., Kita, H., Ishii, S.: Pgx: Hardware-accelerated parallel game simulators for reinforcement learning. Advances in Neural Information Processing Systems **36** (2024)

[12] Xu, Z., Yu, C., Fang, F., Wang, Y., Wu, Y.: Language agents with reinforcement learning for strategic play in the werewolf game. arXiv preprint arXiv:2310.18940 (2023)

[13] Qu, X., Gan, W., Song, D., Zhou, L.: Pursuit-evasion game strategy of usv based on deep reinforcement learning in complex multi-obstacle environment. Ocean Engineering **273**, 114016 (2023)

[14] Rana, K., Xu, M., Tidd, B., Milford, M., Sünderhauf, N.: Residual skill policies: Learning an adaptable skill-based action space for reinforcement learning for robotics. In: Conference on Robot Learning, pp. 2095–2104 (2023). PMLR

[15] Kober, J., Bagnell, J.A., Peters, J.: Reinforcement learning in robotics: A survey. The International Journal of Robotics Research **32**(11), 1238–1274 (2013)

[16] Balasubramanian, S.: Intrinsically motivated multi-goal reinforcement learning using robotics environment integrated with openai gym. Journal of Science & Technology **4**(5), 46–60 (2023)

[17] Abeyruwan, S.W., Graesser, L., D'Ambrosio, D.B., Singh, A., Shankar, A., Bewley, A., Jain,

ARTICLE IN PRESS

D., Choromanski, K.M., Sanketi, P.R.: i-sim2real: Reinforcement learning of robotic policies in tight human-robot interaction loops. In: Conference on Robot Learning, pp. 212–224 (2023). PMLR

[18] Zhu, R., Li, L., Wu, S., Lv, P., Li, Y., Xu, M.: Multi-agent broad reinforcement learning for intelligent traffic light control. Information Sciences **619**, 509–525 (2023)

[19] Yazdani, M., Sarvi, M., Bagloee, S.A., Nassir, N., Price, J., Parineh, H.: Intelligent vehicle pedestrian light (ivpl): A deep reinforcement learning approach for traffic signal control. Transportation research part C: emerging technologies **149**, 103991 (2023)

[20] Liu, Y., Huo, L., Wu, J., Bashir, A.K.: Swarm learning-based dynamic optimal management for traffic congestion in 6g-driven intelligent transportation system. IEEE Transactions on Intelligent Transportation Systems **24**(7), 7831–7846 (2023)

[21] Chen, D., Hajidavalloo, M.R., Li, Z., Chen, K., Wang, Y., Jiang, L., Wang, Y.: Deep multi-agent reinforcement learning for highway on-ramp merging in mixed traffic. IEEE Transactions on Intelligent Transportation Systems **24**(11), 11623–11638 (2023)

[22] Ghosh, P., Oliveira, T.E.A., Alzhouri, F., Ebrahimi, D.: Maximizing group-based vehicle communications and fairness: A reinforcement learning approach. In: 2024 IEEE Wireless Communications and Networking Conference (WCNC), pp. 1–7 (2024). IEEE

[23] Aghaei, V.T., Ağababaoğlu, A., Yıldırım, S., Onat, A.: A real-world application of markov chain monte carlo method for bayesian trajectory control of a robotic manipulator. ISA transactions **125**, 580–590 (2022)

[24] Yu, Y., Liu, Y., Wang, J., Noguchi, N., He, Y.: Obstacle avoidance method based on double dqn for agricultural robots. Computers and Electronics in Agriculture **204**, 107546 (2023)

[25] Koning, M., Cai, W., Sadigh, B., Oppelstrup, T., Kalos, M.H., Bulatov, V.V.: Adaptive importance sampling monte carlo simulation of rare transition events. The Journal of chemical physics **122**(7) (2005)

[26] Li, R., Gong, W., Wang, L., Lu, C., Pan, Z., Zhuang, X.: Double dqn-based coevolution for green distributed heterogeneous hybrid flowshop scheduling with multiple priorities of jobs. IEEE Transactions on Automation Science and Engineering (2023)

[27] Liu, W., Tang, T., Su, S., Cao, Y., Bao, F., Gao, J.: An intelligent train control approach based on the monte carlo reinforcement learning algorithm. In: 2018 21st International Conference on Intelligent Transportation Systems (ITSC), pp. 1944–1949 (2018). IEEE

[28] Jaiswal, A., Kumar, S., Dohare, U.: Green computing in heterogeneous internet of things: Optimizing energy allocation using sarsa-based reinforcement learning. In: 2020 IEEE 17th India Council International Conference (INDICON), pp. 1–6 (2020). IEEE

[29] Xuan, Z., Wei, G., Ni, Z.: Power allocation in multi-agent networks via dueling dqn approach. In: 2021 IEEE 6th International Conference on Signal and Image Processing (ICSIP), pp. 959–963 (2021). IEEE

[30] Lassila, P., Karvo, J., Virtamo, J.: Efficient importance sampling for monte carlo simulation of multicast networks. In: Proceedings IEEE INFOCOM 2001. Conference on Computer Communications. Twentieth Annual Joint Conference of the IEEE Computer and Communications Society (Cat. No. 01CH37213), vol. 1, pp. 432–439 (2001). IEEE

[31] Oh, E., Wang, H.: Reinforcement-learning-based energy storage system operation strategies to manage wind power forecast uncertainty. IEEE Access **8**, 20965–20976 (2020)

[32] Kwon, H.: Learning-based power delay profile estimation for 5g nr via advantage actor-critic (a2c). In: 2022 IEEE 95th Vehicular Technology Conference:(VTC2022-Spring), pp. 1–6 (2022).

ARTICLE IN PRESS

ARTICLE IN PRESS

IEEE

[33] De Asis, K., Sutton, R.S.: Per-decision multi-step temporal difference learning with control variates. arXiv preprint arXiv:1807.01830 (2018)

[34] Li, X., Yang, C., Song, J., Feng, S., Li, W., He, H.: A motion control method for agent based on dyna-q algorithm. In: 2023 4th International Conference on Computer Engineering and Application (ICCEA), pp. 274–278 (2023). IEEE

[35] Sutton, R.S., Tanner, B.: Temporal-difference networks. Advances in neural information processing systems **17** (2004)

[36] Zuters, J.: Realizing undelayed n-step td prediction with neural networks. In: Melecon 2010-2010 15th IEEE Mediterranean Electrotechnical Conference, pp. 102–106 (2010). IEEE

[37] Moerland, T.M., Broekens, J., Plaat, A., Jonker, C.M.: Monte carlo tree search for asymmetric trees. arXiv preprint arXiv:1805.09218 (2018)

[38] Nasreen, M.A., Ravindran, S.: An overview of q-learning based energy efficient power allocation in wban (q-eepa). In: 2022 2nd International Conference on Innovative Sustainable Computational Technologies (CISCT), pp. 1–5 (2022). IEEE

[39] Lopes, G.C., Ferreira, M., Silva Simões, A., Colombini, E.L.: Intelligent control of a quadrotor with proximal policy optimization reinforcement learning. In: 2018 Latin American Robotic Symposium, 2018 Brazilian Symposium on Robotics (SBR) and 2018 Workshop on Robotics in Education (WRE), pp. 503–508 (2018). IEEE

[40] Darapaneni, N., Basu, A., Savla, S., Gururajan, R., Saquib, N., Singhavi, S., Kale, A., Bid, P., Paduri, A.R.: Automated portfolio rebalancing using q-learning. In: 2020 11th IEEE Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON), pp. 0596–0602 (2020). IEEE

[41] Desai, R.M., Patil, B.: Prioritized sweeping reinforcement learning based routing for manets. Indonesian Journal of Electrical Engineering and Computer Science **5**(2), 383–390 (2017)

[42] Santoso, J., *et al.*: Multiagent simulation on hide and seek games using policy gradient trust region policy optimization. In: 2020 7th International Conference on Advance Informatics: Concepts, Theory and Applications (ICAICTA), pp. 1–5 (2020). IEEE

[43] Saha, S., Kay, S.M.: Maximum likelihood parameter estimation of superimposed chirps using monte carlo importance sampling. IEEE Transactions on Signal Processing **50**(2), 224–230 (2002)

[44] Wu, L., Chen, K.: Bias resilient multi-step off-policy goal-conditioned reinforcement learning. arXiv preprint arXiv:2311.17565 (2023)

[45] Yao, M., Gao, X., Wang, J., Wang, M.: Improving nuclei segmentation in pathological image via reinforcement learning. In: 2022 International Conference on Machine Learning, Cloud Computing and Intelligent Mining (MLCCIM), pp. 290–295 (2022). IEEE

[46] Zhang, L., Zhang, Y., Zhao, X., Zou, Z.: Image captioning via proximal policy optimization. Image and Vision Computing **108**, 104126 (2021)

[47] Suh, J., Tanaka, T.: Sarsa (0) reinforcement learning over fully homomorphic encryption. In: 2021 SICE International Symposium on Control Systems (SICE ISCS), pp. 1–7 (2021). IEEE

[48] Go, C.K., Lao, B., Yoshimoto, J., Ikeda, K.: A reinforcement learning approach to the shepherding task using sarsa. In: 2016 International Joint Conference on Neural Networks (IJCNN), pp. 3833–3836 (2016). IEEE

[49] Zerbel, N., Yliniemi, L.: Multiagent monte carlo tree search. In: Proceedings of the 18th

ARTICLE IN PRESS

International Conference on Autonomous Agents and MultiAgent Systems, pp. 2309–2311 (2019)

[50] Mangalampalli, S., Karri, G.R., Mohanty, S.N., Ali, S., Khan, M.I., Abdullaev, S., AlQahtani, S.A.: Multi-objective prioritized task scheduler using improved asynchronous advantage actor critic (a3c) algorithm in multi cloud environment. IEEE Access (2024)

[51] Li, Y.: Deep reinforcement learning: An overview. arXiv preprint arXiv:1701.07274 (2017)

[52] Arulkumaran, K., Deisenroth, M.P., Brundage, M., Bharath, A.A.: Deep reinforcement learning: A brief survey. IEEE Signal Processing Magazine **34**(6), 26–38 (2017)

[53] Wang, X., Wang, S., Liang, X., Zhao, D., Huang, J., Xu, X., Dai, B., Miao, Q.: Deep reinforcement learning: A survey. IEEE Transactions on Neural Networks and Learning Systems **35**(4), 5064–5078 (2022)

[54] Wang, H.-n., Liu, N., Zhang, Y.-y., Feng, D.-w., Huang, F., Li, D.-s., Zhang, Y.-m.: Deep reinforcement learning: a survey. Frontiers of Information Technology & Electronic Engineering **21**(12), 1726–1744 (2020)

[55] Moerland, T.M., Broekens, J., Plaat, A., Jonker, C.M., *et al.*: Model-based reinforcement learning: A survey. Foundations and Trends® in Machine Learning **16**(1), 1–118 (2023)

[56] Polydoros, A.S., Nalpantidis, L.: Survey of model-based reinforcement learning: Applications on robotics. Journal of Intelligent & Robotic Systems **86**(2), 153–173 (2017)

[57] Luo, F.-M., Xu, T., Lai, H., Chen, X.-H., Zhang, W., Yu, Y.: A survey on model-based reinforcement learning. Science China Information Sciences **67**(2), 121101 (2024)

[58] Sato, Y.: Model-free reinforcement learning for financial portfolios: a brief survey. arXiv preprint arXiv:1904.04973 (2019)

[59] Ramírez, J., Yu, W., Perrusquía, A.: Model-free reinforcement learning from expert demonstrations: a survey. Artificial Intelligence Review **55**(4), 3213–3241 (2022)

[60] Yang, N., Chen, S., Zhang, H., Berry, R.: Beyond the edge: An advanced exploration of reinforcement learning for mobile edge computing, its applications, and future research trajectories. IEEE Communications Surveys & Tutorials **27**(1), 546–594 (2025) https://doi.org/10.1109/COMST.2024.3405075

[61] Yang, N., Fan, M., Wang, W., Zhang, H.: Decision-making large language model for wireless communication: A comprehensive survey on key techniques. IEEE Communications Surveys & Tutorials (2025)

[62] Eschmann, J.: Reward function design in reinforcement learning. Reinforcement Learning Algorithms: Analysis and Applications, 25–33 (2021)

[63] Sutton, R.S., Barto, A.G., *et al.*: Reinforcement learning. Journal of Cognitive Neuroscience **11**(1), 126–134 (1999)

[64] Foster, D.J., Rakhlin, A.: Foundations of reinforcement learning and interactive decision making. arXiv preprint arXiv:2312.16730 (2023)

[65] Puterman, M.L.: Chapter 8 markov decision processes. In: Stochastic Models. Handbooks in Operations Research and Management Science, vol. 2, pp. 331–434. Elsevier, ??? (1990). https://doi.org/10.1016/S0927-0507(05)80172-0 . https://www.sciencedirect.com/science/article/pii/S0927050705801720

[66] Rummery, G.A., Niranjan, M.: On-line q-learning using connectionist systems. Technical report, University of Cambridge, Department of Engineering Cambridge (1994)

ARTICLE IN PRESS

[67] Wang, J.X., Kurth-Nelson, Z., Tirumala, D., Soyer, H., Leibo, J.Z., Munos, R., Blundell, C., Kumaran, D., Botvinick, M.: Learning to reinforcement learn. arXiv preprint arXiv:1611.05763 (2016)

[68] Ding, Z., Huang, Y., Yuan, H., Dong, H.: Introduction to reinforcement learning. Deep reinforcement learning: fundamentals, research and applications, 47–123 (2020)

[69] McFarlane, R.: A survey of exploration strategies in reinforcement learning. McGill University **3**, 17–18 (2018)

[70] Badia, A.P., Sprechmann, P., Vitvitskyi, A., Guo, D., Piot, B., Kapturowski, S., Tieleman, O., Arjovsky, M., Pritzel, A., Bolt, A., et al.: Never give up: Learning directed exploration strategies. arXiv preprint arXiv:2002.06038 (2020)

[71] Whitney, W.F., Bloesch, M., Springenberg, J.T., Abdolmaleki, A., Cho, K., Riedmiller, M.: Decoupled exploration and exploitation policies for sample-efficient reinforcement learning. arXiv preprint arXiv:2101.09458 (2021)

[72] Caelen, O., Bontempi, G.: A dynamic programming strategy to balance exploration and exploitation in the bandit problem. Annals of Mathematics and Artificial Intelligence **60**, 3–24 (2010)

[73] Liu, F., Viano, L., Cevher, V.: Understanding deep neural function approximation in reinforcement learning via epsilon-greedy exploration. Advances in Neural Information Processing Systems **35**, 5093–5108 (2022)

[74] Wunder, M., Littman, M.L., Babes, M.: Classes of multiagent q-learning dynamics with epsilon-greedy exploration. In: Proceedings of the 27th International Conference on Machine Learning (ICML-10), pp. 1167–1174 (2010)

[75] Cesa-Bianchi, N., Gentile, C., Lugosi, G., Neu, G.: Boltzmann exploration done right. Advances in neural information processing systems **30** (2017)

[76] Wang, S., Pu, Y., Yang, S., Yao, X., Li, B.: Boltzmann exploration for deterministic policy optimization. In: Neural Information Processing: 27th International Conference, ICONIP 2020, Bangkok, Thailand, November 23–27, 2020, Proceedings, Part II 27, pp. 214–222 (2020). Springer

[77] Usama, M., Chang, D.E.: Learning-driven exploration for reinforcement learning. In: 2021 21st International Conference on Control, Automation and Systems (ICCAS), pp. 1146–1151 (2021). IEEE

[78] White, D.A., Sofge, D.A.: The role of exploration in learning control. Handbook of Intelligent Control: Neural, Fuzzy and Adaptive Approaches, 1–27 (1992)

[79] Kearns, M., Singh, S.: Near-optimal performance for reinforcement learning in polynomial time. URL: http://www. research. att. com/˜ mkearns (1998)

[80] Sutton, R.S.: Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In: Machine Learning Proceedings 1990, pp. 216–224. Elsevier, ??? (1990)

[81] Ishii, S., Yoshida, W., Yoshimoto, J.: Control of exploitation–exploration meta-parameter in reinforcement learning. Neural networks **15**(4-6), 665–687 (2002)

[82] Schäfer, L., Christianos, F., Hanna, J., Albrecht, S.V.: Decoupling exploration and exploitation in reinforcement learning. In: ICML 2021 Workshop on Unsupervised Reinforcement Learning (2021)

[83] Wang, H., Zariphopoulou, T., Zhou, X.: Exploration versus exploitation in reinforcement

ARTICLE IN PRESS

learning: A stochastic control approach. arXiv preprint arXiv:1812.01552 (2018)

[84] Thompson, W.R.: On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. Biometrika **25**(3-4), 285–294 (1933)

[85] Tamar, A., Wu, Y., Thomas, G., Levine, S., Abbeel, P.: Value iteration networks. Advances in neural information processing systems **29** (2016)

[86] Silver, D., Veness, J.: Monte-carlo planning in large pomdps. Advances in neural information processing systems **23** (2010)

[87] Rossi, L., Winands, M.H., Butenweg, C.: Monte carlo tree search as an intelligent search tool in structural design problems. Engineering with Computers **38**(4), 3219–3236 (2022)

[88] Coulom, R.: Efficient selectivity and backup operators in monte-carlo tree search. In: International Conference on Computers and Games, pp. 72–83 (2006). Springer

[89] Fu, M.C.: A tutorial introduction to monte carlo tree search. In: 2020 Winter Simulation Conference (WSC), pp. 1178–1193 (2020). IEEE

[90] Wang, W., Sebag, M.: Multi-objective monte-carlo tree search. In: Asian Conference on Machine Learning, pp. 507–522 (2012). PMLR

[91] Su, X., Huang, T., Li, Y., You, S., Wang, F., Qian, C., Zhang, C., Xu, C.: Prioritized architecture sampling with monto-carlo tree search. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 10968–10977 (2021)

[92] Perez, D., Mostaghim, S., Samothrakis, S., Lucas, S.M.: Multiobjective monte carlo tree search for real-time games. IEEE Transactions on Computational Intelligence and AI in Games **7**(4), 347–360 (2014)

[93] Baier, H., Winands, M.H.: Monte-carlo tree search and minimax hybrids. In: 2013 IEEE Conference on Computational Inteligence in Games (CIG), pp. 1–8 (2013). IEEE

[94] De Waard, M., Roijers, D.M., Bakkes, S.C.: Monte carlo tree search with options for general video game playing. In: 2016 IEEE Conference on Computational Intelligence and Games (CIG), pp. 1–8 (2016). IEEE

[95] Lanctot, M., Winands, M.H., Pepels, T., Sturtevant, N.R.: Monte carlo tree search with heuristic evaluations using implicit minimax backups. In: 2014 IEEE Conference on Computational Intelligence and Games, pp. 1–8 (2014). IEEE

[96] Winands, M.H., Bjornsson, Y., Saito, J.-T.: Monte carlo tree search in lines of action. IEEE Transactions on Computational Intelligence and AI in Games **2**(4), 239–250 (2010)

[97] Santos, A., Santos, P.A., Melo, F.S.: Monte carlo tree search experiments in hearthstone. In: 2017 IEEE Conference on Computational Intelligence and Games (CIG), pp. 272–279 (2017). IEEE

[98] Cowling, P.I., Powley, E.J., Whitehouse, D.: Information set monte carlo tree search. IEEE Transactions on Computational Intelligence and AI in Games **4**(2), 120–143 (2012)

[99] Ciancarini, P., Favini, G.P.: Monte carlo tree search in kriegspiel. Artificial Intelligence **174**(11), 670–684 (2010)

[100] Moore, A.W., Atkeson, C.G.: Prioritized sweeping: Reinforcement learning with less data and less time. Machine learning **13**, 103–130 (1993)

[101] Peng, J., Williams, R.J.: Efficient learning and planning within the dyna framework. Adaptive behavior **1**(4), 437–454 (1993)

ARTICLE IN PRESS

ARTICLE IN PRESS

[102] Li, R., Wang, Q., Dong, C., *et al.*: Morphing strategy design for uav based on prioritized sweeping reinforcement learning. In: IECON 2020 The 46th Annual Conference of the IEEE Industrial Electronics Society, pp. 2786–2791 (2020). IEEE

[103] Zajdel, R.: Epoch-incremental dyna-learning and prioritized sweeping algorithms. Neurocomputing **319**, 13–20 (2018)

[104] Van Seijen, H., Sutton, R.: Planning by prioritized sweeping with small backups. In: International Conference on Machine Learning, pp. 361–369 (2013). PMLR

[105] Bargiacchi, E., Verstraeten, T., Roijers, D.M., Nowé, A.: Model-based multi-agent reinforcement learning with cooperative prioritized sweeping. arXiv preprint arXiv:2001.07527 (2020)

[106] Dearden, R.: Structured prioritized sweeping. In: ICML, pp. 82–89 (2001). Citeseer

[107] Watkins, C.J., Dayan, P.: Q-learning. Machine learning **8**, 279–292 (1992)

[108] Santos, M., López, V., Botella, G., *et al.*: Dyna-h: A heuristic planning reinforcement learning algorithm applied to role-playing game strategy decision systems. Knowledge-Based Systems **32**, 28–36 (2012)

[109] Chai, Y., Zeng, X.-J.: A multi-objective dyna-q based routing in wireless mesh network. Applied Soft Computing **108**, 107486 (2021)

[110] Del Giorno, S., D'Antoni, F., Piemonte, V., Merone, M.: A new glycemic closed-loop control based on dyna-q for type-1-diabetes. Biomedical Signal Processing and Control **81**, 104492 (2023)

[111] Faycal, T., Zito, C.: Dyna-t: Dyna-q and upper confidence bounds applied to trees. arXiv preprint arXiv:2201.04502 (2022)

[112] Xu, Z., Zhu, F., Fu, Y., Liu, Q., You, S.: A dyna-q based multi-path load-balancing routing algorithm in wireless sensor networks. In: 2015 IEEE Trustcom/BigDataSE/ISPA, vol. 3, pp. 1–4 (2015). IEEE

[113] Pei, M., An, H., Liu, B., Wang, C.: An improved dyna-q algorithm for mobile robot path planning in unknown dynamic environment. IEEE Transactions on Systems, Man, and Cybernetics: Systems **52**(7), 4415–4425 (2021)

[114] Wang, F., Gao, J., Li, M., Zhao, L.: Autonomous pev charging scheduling using dyna-q reinforcement learning. IEEE Transactions on Vehicular Technology **69**(11), 12609–12620 (2020)

[115] Liu, Y., Yan, S., Zhao, Y., Song, C., Li, F.: Improved dyna-q: a reinforcement learning method focused via heuristic graph for agv path planning in dynamic environments. Drones **6**(11), 365 (2022)

[116] Zhang, G., Li, Y., Niu, Y., Zhou, Q.: Anti-jamming path selection method in a wireless communication network based on dyna-q. Electronics **11**(15), 2397 (2022)

[117] Hwang, K.-S., Jiang, W.-C., Chen, Y.-J.: Model learning and knowledge sharing for a multiagent system with dyna-q learning. IEEE transactions on cybernetics **45**(5), 978–990 (2014)

[118] Huang, J., Tan, Q., Ma, J., Han, L.: Path planning method using dyna-q algorithm under complex urban environment. In: 2022 China Automation Congress (CAC), pp. 6776–6781 (2022). IEEE

[119] Vitolo, E., San Miguel, A., Civera, J., Mahulea, C.: Performance evaluation of the dyna-q algorithm for robot navigation. In: 2018 IEEE 14th International Conference on Automation

ARTICLE IN PRESS

Science and Engineering (CASE), pp. 322–327 (2018). IEEE

[120] Wang, C., Yuan, S., Shao, K., Ross, K.: On the convergence of the monte carlo exploring starts algorithm for reinforcement learning. arXiv preprint arXiv:2002.03585 (2020)

[121] Peters, J.F., Henry, C.: Approximation spaces in off-policy monte carlo learning. Engineering applications of artificial intelligence **20**(5), 667–675 (2007)

[122] Peters, J.F., Lockery, D., Ramanna, S.: Monte carlo off-policy reinforcement learning: A rough set approach. In: Fifth International Conference on Hybrid Intelligent Systems (HIS'05), p. 6 (2005). IEEE

[123] Subramanian, J., Mahajan, A.: Renewal monte carlo: Renewal theory-based reinforcement learning. IEEE Transactions on Automatic Control **65**(8), 3663–3670 (2019)

[124] Wang, Y., Won, K.S., Hsu, D., Lee, W.S.: Monte carlo bayesian reinforcement learning. arXiv preprint arXiv:1206.6449 (2012)

[125] Wu, B., Feng, Y.: Monte-carlo bayesian reinforcement learning using a compact factored representation. In: 2017 4th International Conference on Information Science and Control Engineering (ICISCE), pp. 466–469 (2017). IEEE

[126] Baykal, O., Alpaslan, F.N.: Reinforcement learning in card game environments using monte carlo methods and artificial neural networks. In: 2019 4th International Conference on Computer Science and Engineering (UBMK), pp. 1–6 (2019). IEEE

[127] Bulteau, S., El Khadiri, M.: A new importance sampling monte carlo method for a flow network reliability problem. Naval Research Logistics (NRL) **49**(2), 204–228 (2002)

[128] Tokdar, S.T., Kass, R.E.: Importance sampling: a review. Wiley Interdisciplinary Reviews: Computational Statistics **2**(1), 54–60 (2010)

[129] Siegmund, D.: Importance sampling in the monte carlo study of sequential tests. The Annals of Statistics, 673–684 (1976)

[130] Altahhan, A.: Td (0)-replay: An efficient model-free planning with full replay. In: 2018 International Joint Conference on Neural Networks (IJCNN), pp. 1–7 (2018). IEEE

[131] Baxter, J., Tridgell, A., Weaver, L.: Knightcap: a chess program that learns by combining td (lambda) with game-tree search. arXiv preprint cs/9901002 (1999)

[132] Dayan, P.: The convergence of td (λ) for general λ. Machine learning **8**, 341–362 (1992)

[133] Dayan, P., Sejnowski, T.J.: Td (λ) converges with probability 1. Machine Learning **14**, 295–301 (1994)

[134] Wiering, M.A., Van Hasselt, H.: Two novel on-policy reinforcement learning algorithms based on td (λ)-methods. In: 2007 IEEE International Symposium on Approximate Dynamic Programming and Reinforcement Learning, pp. 280–287 (2007). IEEE

[135] De Asis, K., Hernandez-Garcia, J., Holland, G., Sutton, R.: Multi-step reinforcement learning: A unifying algorithm. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32 (2018)

[136] Chen, Z., Maguluri, S.T., Shakkottai, S., Shanmugam, K.: A lyapunov theory for finite-sample guarantees of asynchronous q-learning and td-learning variants. arXiv preprint arXiv:2102.01567 (2021)

[137] Lee, D.: Analysis of off-policy multi-step td-learning with linear function approximation. arXiv preprint arXiv:2402.15781 (2024)

ARTICLE IN PRESS

[138] De Asis, K.: A unified view of multi-step temporal difference learning. PhD thesis, University of Alberta (2018)

[139] Wang, Y., Tan, X.: Greedy multi-step off-policy reinforcement learning (2020)

[140] Mahmood, A.R., Yu, H., Sutton, R.S.: Multi-step off-policy learning without importance sampling ratios. arXiv preprint arXiv:1702.03006 (2017)

[141] Bi, Y., Thomas-Mitchell, A., Zhai, W., Khan, N.: A comparative study of deterministic and stochastic policies for q-learning. In: 2023 4th International Conference on Artificial Intelligence, Robotics and Control (AIRC), pp. 1–5 (2023). IEEE

[142] Spano, S., Cardarilli, G.C., Di Nunzio, L., Fazzolari, R., Giardino, D., Matta, M., Nannarelli, A., Re, M.: An efficient hardware implementation of reinforcement learning: The q-learning algorithm. Ieee Access **7**, 186340–186351 (2019)

[143] Huang, D., Zhu, H., Lin, X., Wang, L.: Application of massive parallel computation based q-learning in system control. In: 2022 5th International Conference on Pattern Recognition and Artificial Intelligence (PRAI), pp. 1–5 (2022). IEEE

[144] Daswani, M., Sunehag, P., Hutter, M.: Q-learning for history-based reinforcement learning. In: Asian Conference on Machine Learning, pp. 213–228 (2013). PMLR

[145] Shou, B., Zhang, H., Long, Z., Xie, Y., Zhang, K., Gu, Q.: Design and applications of q-learning adaptive pid algorithm for maglev train levitation control system. In: 2023 35th Chinese Control and Decision Conference (CCDC), pp. 1947–1953 (2023). IEEE

[146] Akshay, G., Naik, N.S., Vardhan, J.: Enhancing hadoop performance with q-learning for optimal parameter tuning. In: TENCON 2023-2023 IEEE Region 10 Conference (TENCON), pp. 617–622 (2023). IEEE

[147] Jin, C., Allen-Zhu, Z., Bubeck, S., Jordan, M.I.: Is q-learning provably efficient? Advances in neural information processing systems **31** (2018)

[148] Da Silva, L.M., Torquato, M.F., Fernandes, M.A.: Parallel implementation of reinforcement learning q-learning technique for fpga. IEEE Access **7**, 2782–2798 (2018)

[149] Wang, S., Zhang, L.: Q-learning based handover algorithm for high-speed rail wireless communications. In: 2023 IEEE Wireless Communications and Networking Conference (WCNC), pp. 1–6 (2023). IEEE

[150] Wang, S.: Taxi scheduling research based on q-learning. In: 2021 3rd International Conference on Machine Learning, Big Data and Business Intelligence (MLBDBI), pp. 700–703 (2021). IEEE

[151] Xiao, W., Chen, J., Li, X., Wang, M., Huang, D., Zhang, D.: Random walk routing algorithm based on q-learning in optical transport network. In: 2022 18th International Conference on Computational Intelligence and Security (CIS), pp. 88–92 (2022). IEEE

[152] Qu, X., Yao, M.: Visual novelty based internally motivated q-learning for mobile robot scene learning and recognition. In: 2011 4th International Congress on Image and Signal Processing, vol. 3, pp. 1461–1466 (2011). IEEE

[153] Paterova, T., Prauzek, M., Konecny, J.: Robustness analysis of data-driven self-learning controllers for iot environmental monitoring nodes based on q-learning approaches. In: 2022 IEEE Symposium Series on Computational Intelligence (SSCI), pp. 721–727 (2022). IEEE

[154] Nassar, A., Yilmaz, Y.: Reinforcement learning for adaptive resource allocation in fog ran for iot with heterogeneous latency requirements. IEEE Access **7**, 128014–128025 (2019)

[155] Wender, S., Watson, I.: Applying reinforcement learning to small scale combat in the real-time

ARTICLE IN PRESS

strategy game starcraft: Broodwar. In: 2012 Ieee Conference on Computational Intelligence and Games (cig), pp. 402–408 (2012). IEEE

[156] Iima, H., Kuroe, Y.: Swarm reinforcement learning algorithms based on sarsa method. In: 2008 SICE Annual Conference, pp. 2045–2049 (2008). IEEE

[157] Hasselt, H.: Double q-learning. Advances in neural information processing systems **23** (2010)

[158] Ren, Z., Zhu, G., Hu, H., Han, B., Chen, J., Zhang, C.: On the estimation bias in double q-learning. Advances in Neural Information Processing Systems **34**, 10246–10259 (2021)

[159] Ben-Akka, M., Tanougast, C., Diou, C., Chaddad, A.: An efficient hardware implementation of the double q-learning algorithm. In: 2023 3rd International Conference on Electrical, Computer, Communications and Mechatronics Engineering (ICECCME), pp. 1–6 (2023). IEEE

[160] Jamshidi, F., Zhang, L., Nezhadalinaei, F.: Autonomous driving systems: Developing an approach based on a* and double q-learning. In: 2021 7th International Conference on Web Research (ICWR), pp. 82–85 (2021). IEEE

[161] Paterova, T., Prauzek, M., Konecny, J.: Data-driven self-learning controller design approach for power-aware iot devices based on double q-learning strategy. In: 2021 IEEE Symposium Series on Computational Intelligence (SSCI), pp. 01–07 (2021). IEEE

[162] Fan, J., Yang, T., Zhao, J., Cui, Z., Ning, J., Wang, P.: Double q learning multi-agent routing method for maritime search and rescue. In: 2023 International Conference on Ubiquitous Communication (Ucom), pp. 367–372 (2023). IEEE

[163] Konecny, J., Prauzek, M., Paterova, T.: Double q-learning adaptive wavelet compression method for data transmission at environmental monitoring stations. In: 2022 IEEE Symposium Series on Computational Intelligence (SSCI), pp. 567–572 (2022). IEEE

[164] Huang, H., Lin, M., Zhang, Q.: Double-q learning-based dvfs for multi-core real-time systems. In: 2017 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData), pp. 522–529 (2017). IEEE

[165] Wang, D., Liu, B., Jia, H., Zhang, Z., Chen, J., Huang, D.: Peer-to-peer electricity transaction decisions of the user-side smart energy system based on the sarsa reinforcement learning. CSEE Journal of Power and Energy Systems **8**(3), 826–837 (2020)

[166] Aljohani, T.M., Mohammed, O.: A real-time energy consumption minimization framework for electric vehicles routing optimization based on sarsa reinforcement learning. Vehicles **4**(4), 1176–1194 (2022)

[167] Van Seijen, H., Van Hasselt, H., Whiteson, S., Wiering, M.: A theoretical and empirical analysis of expected sarsa. In: 2009 Ieee Symposium on Adaptive Dynamic Programming and Reinforcement Learning, pp. 177–184 (2009). IEEE

[168] Moradimaryamnegari, H., Frego, M., Peer, A.: Model predictive control-based reinforcement learning using expected sarsa. IEEE Access **10**, 81177–81191 (2022)

[169] Gonzalez, I.A.M., Turau, V.: Comparison of wifi interference mitigation strategies in dsme networks: Leveraging reinforcement learning with expected sarsa. In: 2023 IEEE International Mediterranean Conference on Communications and Networking (MeditCom), pp. 270–275 (2023). IEEE

[170] Muduli, R., Jena, D., Moger, T.: Application of expected sarsa-learning for load frequency control of multi-area power system. In: 2023 5th International Conference on Energy, Power and Environment: Towards Flexible Green Energy Technologies (ICEPE), pp. 1–6 (2023). IEEE

ARTICLE IN PRESS

ARTICLE IN PRESS

[171] Kekuda, A., Anirudh, R., Krishnan, M.: Reinforcement learning based intelligent traffic signal control using n-step sarsa. In: 2021 International Conference on Artificial Intelligence and Smart Systems (ICAIS), pp. 379–384 (2021). IEEE

[172] Kuchibhotla, V., Harshitha, P., Goyal, S.: An n-step look ahead algorithm using mixed (on and off) policy reinforcement learning. In: 2020 3rd International Conference on Intelligent Sustainable Systems (ICISS), pp. 677–681 (2020). IEEE

[173] Degris, T., Pilarski, P.M., Sutton, R.S.: Model-free reinforcement learning with continuous action in practice. In: 2012 American Control Conference (ACC), pp. 2177–2182 (2012). IEEE

[174] Liu, Y., Halev, A., Liu, X.: Policy learning with constraints in model-free reinforcement learning: A survey. In: The 30th International Joint Conference on Artificial Intelligence (ijcai) (2021)

[175] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., Riedmiller, M.: Playing atari with deep reinforcement learning. arXiv preprint arXiv:1312.5602 (2013)

[176] Ahn, K.U., Park, C.S.: Application of deep q-networks for model-free optimal control balancing between different hvac systems. Science and Technology for the Built Environment **26**(1), 61–74 (2020)

[177] Stember, J.N., Shalu, H.: Reinforcement learning using deep q networks and q learning accurately localizes brain tumors on mri with very small training sets. BMC Medical Imaging **22**(1), 224 (2022)

[178] Guo, H., Wu, R., Qi, B., Xu, C.: Deep-q-networks-based adaptive dual-mode energy-efficient routing in rechargeable wireless sensor networks. IEEE Sensors Journal **22**(10), 9956–9966 (2022)

[179] Talaat, F.M.: Effective deep q-networks (edqn) strategy for resource allocation based on optimized reinforcement learning algorithm. Multimedia Tools and Applications **81**(28), 39945–39961 (2022)

[180] Song, C., Lee, H., Kim, K., Cha, S.W.: A power management strategy for parallel phev using deep q-networks. In: 2018 IEEE Vehicle Power and Propulsion Conference (VPPC), pp. 1–5 (2018). IEEE

[181] Yoon, S., Kim, K.-J.: Deep q networks for visual fighting game ai. In: 2017 IEEE Conference on Computational Intelligence and Games (CIG), pp. 306–308 (2017). IEEE

[182] Lv, L., Zhang, S., Ding, D., Wang, Y.: Path planning via an improved dqn-based learning policy. IEEE Access **7**, 67319–67330 (2019)

[183] Zamzam, A.S., Yang, B., Sidiropoulos, N.D.: Energy storage management via deep q-networks. In: 2019 IEEE Power & Energy Society General Meeting (PESGM), pp. 1–5 (2019). IEEE

[184] Gao, N., Qin, Z., Jing, X., Ni, Q., Jin, S.: Anti-intelligent uav jamming strategy via deep q-networks. IEEE Transactions on Communications **68**(1), 569–581 (2019)

[185] Van Hasselt, H., Guez, A., Silver, D.: Deep reinforcement learning with double q-learning. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 30 (2016)

[186] Peng, B., Sun, Q., Li, S.E., Kum, D., Yin, Y., Wei, J., Gu, T.: End-to-end autonomous driving through dueling double deep q-network. Automotive Innovation **4**, 328–337 (2021)

[187] Zhang, X., Shi, X., Zhang, Z., Wang, Z., Zhang, L.: A ddqn path planning algorithm based on experience classification and multi steps for mobile robots. Electronics **11**(14), 2120 (2022)

[188] Iqbal, A., Tham, M.-L., Chang, Y.C.: Double deep q-network-based energy-efficient resource allocation in cloud radio access network. IEEE Access **9**, 20440–20449 (2021)

ARTICLE IN PRESS

ARTICLE IN PRESS

[189] Zuo, G., Du, T., Lu, J.: Double dqn method for object detection. In: 2017 Chinese Automation Congress (CAC), pp. 6727–6732 (2017). IEEE

[190] Zhang, W., Gai, J., Zhang, Z., Tang, L., Liao, Q., Ding, Y.: Double-dqn based path smoothing and tracking control method for robotic vehicle navigation. Computers and Electronics in Agriculture **166**, 104985 (2019)

[191] Zhang, S., Wu, Y., Ogai, H., Inujima, H., Tateno, S.: Tactical decision-making for autonomous driving using dueling double deep q network with double attention. IEEE Access **9**, 151983–151992 (2021)

[192] Xue, X., Li, Z., Zhang, D., Yan, Y.: A deep reinforcement learning method for mobile robot collision avoidance based on double dqn. In: 2019 IEEE 28th International Symposium on Industrial Electronics (ISIE), pp. 2131–2136 (2019). IEEE

[193] Mo, S., Pei, X., Chen, Z.: Decision-making for oncoming traffic overtaking scenario using double dqn. In: 2019 3rd Conference on Vehicle Control and Intelligence (CVCI), pp. 1–4 (2019). IEEE

[194] Xiaofei, Y., Yilun, S., Wei, L., Hui, Y., Weibo, Z., Zhengrong, X.: Global path planning algorithm based on double dqn for multi-tasks amphibious unmanned surface vehicle. Ocean Engineering **266**, 112809 (2022)

[195] Lee, C., Jung, J., Chung, J.-M.: Intelligent dual active protocol stack handover based on double dqn deep reinforcement learning for 5g mmwave networks. IEEE Transactions on Vehicular Technology **71**(7), 7572–7584 (2022)

[196] Zhang, Y., Sun, P., Yin, Y., Lin, L., Wang, X.: Human-like autonomous vehicle speed control by deep reinforcement learning with double q-learning. In: 2018 IEEE Intelligent Vehicles Symposium (IV), pp. 1251–1256 (2018). IEEE

[197] Li, D., Xu, S., Zhao, J.: Partially observable double dqn based iot scheduling for energy harvesting. In: 2019 IEEE International Conference on Communications Workshops (ICC Workshops), pp. 1–6 (2019). IEEE

[198] Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., Freitas, N.: Dueling network architectures for deep reinforcement learning. In: International Conference on Machine Learning, pp. 1995–2003 (2016). PMLR

[199] Han, B.-A., Yang, J.-J.: Research on adaptive job shop scheduling problems based on dueling double dqn. Ieee Access **8**, 186474–186495 (2020)

[200] Ban, T.-W.: An autonomous transmission scheme using dueling dqn for d2d communication networks. IEEE transactions on vehicular technology **69**(12), 16348–16352 (2020)

[201] Liu, Y., Zhang, C.: Application of dueling dqn and decga for parameter estimation in variogram models. IEEE Access **8**, 38112–38122 (2020)

[202] Liu, W., Si, P., Sun, E., Li, M., Fang, C., Zhang, Y.: Green mobility management in uav-assisted iot based on dueling dqn. In: ICC 2019-2019 IEEE International Conference on Communications (ICC), pp. 1–6 (2019). IEEE

[203] Tadele, S.B., Kar, B., Wakgra, F.G., Khan, A.U.: Optimization of end-to-end aoi in edge-enabled vehicular fog systems: A dueling-dqn approach. arXiv preprint arXiv:2407.02815 (2024)

[204] Jiang, W., Bao, C., Xu, G., Wang, Y.: Research on autonomous obstacle avoidance and target tracking of uav based on improved dueling dqn algorithm. In: 2021 China Automation Congress (CAC), pp. 5110–5115 (2021). IEEE

[205] Huang, Z., Liu, S., Zhang, G.: The usv path planning of dueling dqn algorithm based on tree

ARTICLE IN PRESS

sampling mechanism. In: 2022 IEEE Asia-Pacific Conference on Image Processing, Electronics and Computers (IPEC), pp. 971–976 (2022). IEEE

[206] Williams, R.J.: Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine learning **8**, 229–256 (1992)

[207] Zhang, J., Kim, J., O'Donoghue, B., Boyd, S.: Sample efficient reinforcement learning with reinforce. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 35, pp. 10887–10895 (2021)

[208] Guo, L., Li, Z., Outbib, R., Gao, F.: Function approximation reinforcement learning of energy management with the fuzzy reinforce for fuel cell hybrid electric vehicles. Energy and AI **13**, 100246 (2023)

[209] Lauffenburger, J.C., Yom-Tov, E., Keller, P.A., McDonnell, M.E., Bessette, L.G., Fontanet, C.P., Sears, E.S., Kim, E., Hanken, K., Buckley, J.J., *et al.*: Reinforcement learning to improve non-adherence for diabetes treatments by optimising response and customising engagement (reinforce): study protocol of a pragmatic randomised trial. BMJ open **11**(12), 052091 (2021)

[210] Tao, Y., Tan, W.L.: A reinforcement learning approach to wi-fi rate adaptation using the reinforce algorithm. In: 2024 IEEE Wireless Communications and Networking Conference (WCNC), pp. 1–6 (2024). IEEE

[211] Weerakoon, K., Chakraborty, S., Karapetyan, N., Sathyamoorthy, A.J., Bedi, A.S., Manocha, D.: Htron: Efficient outdoor navigation with sparse rewards via heavy tailed adaptive reinforce algorithm. arXiv preprint arXiv:2207.03694 (2022)

[212] Schulman, J., Levine, S., Abbeel, P., Jordan, M., Moritz, P.: Trust region policy optimization. In: International Conference on Machine Learning, pp. 1889–1897 (2015). PMLR

[213] Bertsekas, D.: Dynamic Programming and Optimal Control: Volume I vol. 4. Athena scientific, ??? (2012)

[214] Peters, J., Schaal, S.: Reinforcement learning of motor skills with policy gradients. Neural networks **21**(4), 682–697 (2008)

[215] Szita, I., Lörincz, A.: Learning tetris using the noisy cross-entropy method. Neural computation **18**(12), 2936–2941 (2006)

[216] Liu, B., Cai, Q., Yang, Z., Wang, Z.: Neural trust region/proximal policy optimization attains globally optimal policy. Advances in neural information processing systems **32** (2019)

[217] Achiam, J.: Advanced policy gradient methods. Dostupné tiež z: http://rail. eecs. berkeley. edu/deeprlcourse-fa17/f17docs ... (2017)

[218] Yuan, Q., Xiao, N.: A monotonic policy optimization algorithm for high-dimensional continuous control problem in 3d mujoco. Multimedia Tools and Applications **78**, 28665–28680 (2019)

[219] Roostaie, S., Ebadzadeh, M.M.: Entrpo: Trust region policy optimization method with entropy regularization. arXiv preprint arXiv:2110.13373 (2021)

[220] Kuba, J.G., Chen, R., Wen, M., Wen, Y., Sun, F., Wang, J., Yang, Y.: Trust region policy optimisation in multi-agent reinforcement learning. arXiv preprint arXiv:2109.11251 (2021)

[221] Liu, H., Wu, Y., Sun, F.: Extreme trust region policy optimization for active object recognition. IEEE transactions on neural networks and learning systems **29**(6), 2253–2258 (2018)

[222] Mondal, B., Banerjee, A., Gupta, S.: Xss filter detection using trust region policy optimization. In: 2023 1st International Conference on Advanced Innovations in Smart Cities (ICAISC), pp. 1–4 (2023). IEEE

ARTICLE IN PRESS

ARTICLE IN PRESS

[223] Erens, J.: Universal robot policy: Using a surrogate model in combination with trpo. Master's thesis, University of Twente (2024)

[224] Thattai, K., Ravishankar, J., Li, C.: Consumer-centric home energy management system using trust region policy optimization-based multi-agent deep reinforcement learning. In: 2023 IEEE Belgrade PowerTech, pp. 1–6 (2023). IEEE

[225] Peng, N., Lin, Y., Zhang, Y., Li, J.: Aoi-aware joint spectrum and power allocation for internet of vehicles: A trust region policy optimization-based approach. IEEE Internet of Things Journal **9**(20), 19916–19927 (2022)

[226] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347 (2017)

[227] Wei, H., Liu, X., Mashayekhy, L., Decker, K.: Mixed-autonomy traffic control with proximal policy optimization. In: 2019 IEEE Vehicular Networking Conference (VNC), pp. 1–8 (2019). IEEE

[228] Zhang, B., Lu, X., Diao, R., Li, H., Lan, T., Shi, D., Wang, Z.: Real-time autonomous line flow control using proximal policy optimization. In: 2020 IEEE Power & Energy Society General Meeting (PESGM), pp. 1–5 (2020). IEEE

[229] Ying, C.-S., Chow, A.H., Wang, Y.-H., Chin, K.-S.: Adaptive metro service schedule and train composition with a proximal policy optimization approach based on deep reinforcement learning. IEEE Transactions on Intelligent Transportation Systems **23**(7), 6895–6906 (2021)

[230] Jin, J., Xu, Y.: Optimal policy characterization enhanced proximal policy optimization for multitask scheduling in cloud computing. IEEE Internet of Things Journal **9**(9), 6418–6433 (2021)

[231] Guan, Y., Ren, Y., Li, S.E., Sun, Q., Luo, L., Li, K.: Centralized cooperation for connected and automated vehicles at intersections by proximal policy optimization. IEEE Transactions on Vehicular Technology **69**(11), 12597–12608 (2020)

[232] Bøhn, E., Coates, E.M., Moe, S., Johansen, T.A.: Deep reinforcement learning attitude control of fixed-wing uavs using proximal policy optimization. In: 2019 International Conference on Unmanned Aircraft Systems (ICUAS), pp. 523–533 (2019). IEEE

[233] Ye, F., Cheng, X., Wang, P., Chan, C.-Y., Zhang, J.: Automated lane change strategy using proximal policy optimization-based deep reinforcement learning. In: 2020 IEEE Intelligent Vehicles Symposium (IV), pp. 1746–1752 (2020). IEEE

[234] Grondman, I., Busoniu, L., Lopes, G.A., Babuska, R.: A survey of actor-critic reinforcement learning: Standard and natural policy gradients. IEEE Transactions on Systems, Man, and Cybernetics, part C (applications and reviews) **42**(6), 1291–1307 (2012)

[235] Mnih, V., Badia, A.P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., Kavukcuoglu, K.: Asynchronous methods for deep reinforcement learning. In: International Conference on Machine Learning, pp. 1928–1937 (2016). PMLR

[236] Konda, V., Tsitsiklis, J.: Actor-critic algorithms. Advances in neural information processing systems **12** (1999)

[237] Zou, Y., Xing, Q.-Z., Wang, B.-C., Zheng, S.-X., Cheng, C., Wang, Z.-M., Wang, X.-W.: Application of the asynchronous advantage actor–critic machine learning algorithm to real-time accelerator tuning. Nuclear Science and Techniques **30**, 1–9 (2019)

[238] Day, M.-Y., Yang, C.-Y., Ni, Y.: Portfolio dynamic trading strategies using deep reinforcement learning. Soft Computing **28**(15), 8715–8730 (2024)

ARTICLE IN PRESS

[239] Jiang, F., Han, S., Sun, C.: Asynchronous advantage actor-critic algorithm based cooperative caching strategy for fog radio access networks. In: 2023 IEEE Wireless Communications and Networking Conference (WCNC), pp. 1–6 (2023). IEEE

[240] Du, J., Cheng, W., Lu, G., Cao, H., Chu, X., Zhang, Z., Wang, J.: Resource pricing and allocation in mec enabled blockchain systems: An a3c deep reinforcement learning approach. IEEE Transactions on Network Science and Engineering **9**(1), 33–44 (2021)

[241] Joypriyanka, M., Surendran, R.: Chess game to improve the mental ability of alzheimer's patients using a3c. In: 2023 Fifth International Conference on Electrical, Computer and Communication Technologies (ICECCT), pp. 1–6 (2023). IEEE

[242] Tiong, T., Saad, I., Teo, K.T.K., Lago, H.B.: Autonomous valet parking with asynchronous advantage actor-critic proximal policy optimization. In: 2022 IEEE 12th Annual Computing and Communication Workshop and Conference (CCWC), pp. 0334–0340 (2022). IEEE

[243] Shi, Z., Li, L., Xu, Y., Li, X., Chen, W., Han, Z.: Content caching policy for 5g network based on asynchronous advantage actor-critic method. In: 2019 IEEE Global Communications Conference (GLOBECOM), pp. 1–6 (2019). IEEE

[244] Yang, J., Lu, J., Zhou, X., Li, S., Xiong, C., Hu, J.: Ha-a2c: Hard attention and advantage actor-critic for addressing latency optimization in edge computing. IEEE Transactions on Green Communications and Networking (2024)

[245] Choppara, P., Mangalampalli, S.: Reliability and trust aware task scheduler for cloud-fog computing using advantage actor critic (a2c) algorithm. IEEE Access (2024)

[246] Dantas, Y., Iturria-Rivera, P.E., Zhou, H., Bavand, M., Elsayed, M., Gaigalas, R., Erol-Kantarci, M.: Beam selection for energy-efficient mmwave network using advantage actor critic learning. In: ICC 2023-IEEE International Conference on Communications, pp. 5285–5290 (2023). IEEE

[247] Vishal, M., Satija, Y., Babu, B.S.: Trading agent for the indian stock market scenario using actor-critic based reinforcement learning. In: 2021 IEEE International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS), pp. 1–5 (2021). IEEE

[248] Sun, Y., Zhang, X.: A2c learning for tasks segmentation with cooperative computing in edge computing networks. In: GLOBECOM 2022-2022 IEEE Global Communications Conference, pp. 2236–2241 (2022). IEEE

[249] Han, X., Wang, J., Zhang, Q., Qin, X., Sun, M.: Multi-uav automatic dynamic obstacle avoidance with experience-shared a2c. In: 2019 International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob), pp. 330–335 (2019). IEEE

[250] Shao, K., Zhao, D., Zhu, Y., Zhang, Q.: Visual navigation with actor-critic deep reinforcement learning. In: 2018 International Joint Conference on Neural Networks (IJCNN), pp. 1–6 (2018). IEEE

[251] Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., Riedmiller, M.: Deterministic policy gradient algorithms. In: International Conference on Machine Learning, pp. 387–395 (2014). Pmlr

[252] Degris, T., White, M., Sutton, R.S.: Off-policy actor-critic. arXiv preprint arXiv:1205.4839 (2012)

[253] Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., Wierstra, D.: Continuous control with deep reinforcement learning. arXiv preprint arXiv:1509.02971 (2015)

[254] Uhlenbeck, G.E., Ornstein, L.S.: On the theory of the brownian motion. Physical review **36**(5),

ARTICLE IN PRESS

823 (1930)

[255] Kong, Y., Li, Y., Wang, J., Yin, S.: Edge computing task unloading decision optimization algorithm based on deep reinforcement learning. In: China Conference on Wireless Sensor Networks, pp. 189–201 (2023). Springer

[256] Candeli, A., De Tommasi, G., Lui, D.G., Mele, A., Santini, S., Tartaglione, G.: A deep deterministic policy gradient learning approach to missile autopilot design. IEEE Access **10**, 19685–19696 (2022)

[257] Wei, Z., Quan, Z., Wu, J., Li, Y., Pou, J., Zhong, H.: Deep deterministic policy gradient-drl enabled multiphysics-constrained fast charging of lithium-ion battery. IEEE Transactions on Industrial Electronics **69**(3), 2588–2598 (2021)

[258] Wen, S., Chen, J., Wang, S., Zhang, H., Hu, X.: Path planning of humanoid arm based on deep deterministic policy gradient. In: 2018 IEEE International Conference on Robotics and Biomimetics (ROBIO), pp. 1755–1760 (2018). IEEE

[259] Gao, X., Yan, L., Li, Z., Wang, G., Chen, I.-M.: Improved deep deterministic policy gradient for dynamic obstacle avoidance of mobile robot. IEEE Transactions on Systems, Man, and Cybernetics: Systems **53**(6), 3675–3682 (2023)

[260] Zhou, S., Cheng, Y., Lei, X., Duan, H.: Deep deterministic policy gradient with prioritized sampling for power control. IEEE Access **8**, 194240–194250 (2020)

[261] Liang, Y., Guo, C., Ding, Z., Hua, H.: Agent-based modeling in electricity market using deep deterministic policy gradient algorithm. IEEE transactions on power systems **35**(6), 4180–4192 (2020)

[262] Xu, Y.-H., Yang, C.-C., Hua, M., Zhou, W.: Deep deterministic policy gradient (ddpg)-based resource allocation scheme for noma vehicular communications. IEEE Access **8**, 18797–18807 (2020)

[263] Fujimoto, S., Hoof, H., Meger, D.: Addressing function approximation error in actor-critic methods. In: International Conference on Machine Learning, pp. 1587–1596 (2018). PMLR

[264] Thrun, S.B.: Efficient Exploration in Reinforcement Learning. Carnegie Mellon University, ??? (1992)

[265] Yazar, O., Coskun, S., Li, L., Zhang, F., Huang, C.: Actor-critic td3-based deep reinforcement learning for energy management strategy of hev. In: 2023 5th International Congress on Human-Computer Interaction, Optimization and Robotic Applications (HORA), pp. 1–6 (2023). IEEE

[266] Mosali, N.A., Shamsudin, S.S., Alfandi, O., Omar, R., Al-Fadhali, N.: Twin delayed deep deterministic policy gradient-based target tracking for unmanned aerial vehicle with achievement rewarding and multistage training. IEEE Access **10**, 23545–23559 (2022)

[267] Han, X., Li, Z., Xie, Z.: Two-step random access optimization for 5g-and-beyond leo satellite communication system: a td3-based msga channel allocation strategy. IEEE Communications Letters **27**(6), 1570–1574 (2023)

[268] Hossain, R., Gautam, M., Lakouraj, M.M., Livani, H., Benidris, M.: Volt-var optimization in distribution networks using twin delayed deep reinforcement learning. In: 2022 IEEE Power & Energy Society Innovative Smart Grid Technologies Conference (ISGT), pp. 1–5 (2022). IEEE

[269] Shehab, M., Zaghloul, A., El-Badawy, A.: Low-level control of a quadrotor using twin delayed deep deterministic policy gradient (td3). In: 2021 18th International Conference on Electrical Engineering, Computing Science and Automatic Control (CCE), pp. 1–6 (2021). IEEE

[270] Yu, J., Sun, H., Sun, Q.: Multi-auv charging navigation trajectory planning based on twin

ARTICLE IN PRESS

delayed deep deterministic policy gradient. In: 2023 42nd Chinese Control Conference (CCC), pp. 8521–8526 (2023). IEEE

[271] Bishen, H.K., Shihabudheen, K., Shanir, P.M.: Adaptive cruise control using twin delayed deep deterministic policy gradient. In: 2023 5th International Conference on Energy, Power and Environment: Towards Flexible Green Energy Technologies (ICEPE), pp. 1–6 (2023). IEEE

[272] Bai, Y., Jones, A., Ndousse, K., Askell, A., Chen, A., DasSarma, N., Drain, D., Fort, S., Ganguli, D., Henighan, T., et al.: Training a helpful and harmless assistant with reinforcement learning from human feedback. arXiv preprint arXiv:2204.05862 (2022)

[273] Kaufmann, T., Weng, P., Bengs, V., Hüllermeier, E.: A survey of reinforcement learning from human feedback. arXiv preprint arXiv:2312.14925 (2023)

[274] Lambert, N., Castricato, L., Werra, L., Havrilla, A.: Illustrating reinforcement learning from human feedback (rlhf). Hugging Face Blog (2022). https://huggingface.co/blog/rlhf

[275] Griffith, S., Subramanian, K., Scholz, J., Isbell, C.L., Thomaz, A.L.: Policy shaping: Integrating human feedback with reinforcement learning. Advances in neural information processing systems **26** (2013)

[276] Arulkumaran, K., Deisenroth, M.P., Brundage, M., Bharath, A.A.: A brief survey of deep reinforcement learning. arXiv preprint arXiv:1708.05866 (2017)

[277] Sutton, R.S., Precup, D., Singh, S.: Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. Artificial intelligence **112**(1-2), 181–211 (1999)

[278] Tessler, C., Givony, S., Zahavy, T., Mankowitz, D., Mannor, S.: A deep hierarchical approach to lifelong learning in minecraft. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 31 (2017)

[279] Kulkarni, T.D., Narasimhan, K., Saeedi, A., Tenenbaum, J.: Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation. Advances in neural information processing systems **29** (2016)

[280] Vezhnevets, A., Mnih, V., Osindero, S., Graves, A., Vinyals, O., Agapiou, J., et al.: Strategic attentive writer for learning macro-actions. Advances in neural information processing systems **29** (2016)

[281] Vezhnevets, A.S., Osindero, S., Schaul, T., Heess, N., Jaderberg, M., Silver, D., Kavukcuoglu, K.: Feudal networks for hierarchical reinforcement learning. In: International Conference on Machine Learning, pp. 3540–3549 (2017). PMLR

[282] Bacon, P.-L., Harb, J., Precup, D.: The option-critic architecture. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 31 (2017)

[283] Levine, S., Kumar, A., Tucker, G., Fu, J.: Offline reinforcement learning: Tutorial, review, and perspectives on open problems. arXiv preprint arXiv:2005.01643 (2020)

[284] Prudencio, R.F., Maximo, M.R., Colombini, E.L.: A survey on offline reinforcement learning: Taxonomy, review, and open problems. IEEE Transactions on Neural Networks and Learning Systems (2023)

[285] Fujimoto, S., Meger, D., Precup, D.: Off-policy deep reinforcement learning without exploration. In: International Conference on Machine Learning, pp. 2052–2062 (2019). PMLR

[286] Kumar, A., Zhou, A., Tucker, G., Levine, S.: Conservative q-learning for offline reinforcement learning. Advances in Neural Information Processing Systems **33**, 1179–1191 (2020)

[287] Wu, Y., Tucker, G., Nachum, O.: Behavior regularized offline reinforcement learning. arXiv preprint arXiv:1911.11361 (2019)

ARTICLE IN PRESS

[288] Taylor, M.E., Stone, P.: Transfer learning for reinforcement learning domains: A survey. Journal of Machine Learning Research **10**(7) (2009)

[289] Fernández, F., Veloso, M.: Probabilistic policy reuse in a reinforcement learning agent. In: Proceedings of the Fifth International Joint Conference on Autonomous Agents and Multiagent Systems, pp. 720–727 (2006)

[290] Barreto, A., Dabney, W., Munos, R., Hunt, J.J., Schaul, T., Hasselt, H.P., Silver, D.: Successor features for transfer in reinforcement learning. Advances in neural information processing systems **30** (2017)

[291] Gupta, A., Mendonca, R., Liu, Y., Abbeel, P., Levine, S.: Meta-reinforcement learning of structured exploration strategies. Advances in neural information processing systems **31** (2018)

[292] Gulcehre, C., Wang, Z., Novikov, A., Paine, T., Gómez, S., Zolna, K., Agarwal, R., Merel, J.S., Mankowitz, D.J., Paduraru, C., *et al.*: Rl unplugged: A suite of benchmarks for offline reinforcement learning. Advances in Neural Information Processing Systems **33**, 7248–7259 (2020)

[293] Ma, C., Wen, J., Bengio, Y.: Universal successor representations for transfer reinforcement learning. arXiv preprint arXiv:1804.03758 (2018)

[294] Madarasz, T., Behrens, T.: Better transfer learning with inferred successor maps. Advances in neural information processing systems **32** (2019)

[295] Alshiekh, M., Bloem, R., Ehlers, R., Könighofer, B., Niekum, S., Topcu, U.: Safe reinforcement learning via shielding. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32 (2018)

[296] Abel, D., MacGlashan, J., Littman, M.L.: Reinforcement learning as a framework for ethical decision making. In: AAAI Workshop: AI, Ethics, and Society, vol. 16 (2016). Phoenix, AZ

[297] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., *et al.*: Training language models to follow instructions with human feedback. Advances in neural information processing systems **35**, 27730–27744 (2022)

[298] Ghasemi, M., Crowley, M.: Toward virtuous reinforcement learning. arXiv preprint arXiv:2512.04246 (2025)

[299] Ecoffet, A., Lehman, J.: Reinforcement learning under moral uncertainty. In: International Conference on Machine Learning, pp. 2926–2936 (2021). PMLR

[300] Ghasemi, M., Crowley, M.: Objective decoupling in social reinforcement learning: Recovering ground truth from sycophantic majorities. arXiv preprint arXiv:2602.08092 (2026)

[301] Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., et al.: Constitutional ai: Harmlessness from ai feedback. arXiv preprint arXiv:2212.08073 (2022)

[302] Gupta, L., Li, L., Liu, Y., Ganapathi, S., Zhao, M., Lin, X.V., Tandon, N., Roussanaly, R., Zhang, C., Chen, A., Liang, P., Gao, T., Yeh, R., Kıcıman, E., Kambadur, M.: World of workflows: a benchmark for bringing world models to enterprise systems. arXiv preprint arXiv:2601.22130 (2026) arXiv:2601.22130 [cs.LG]

[303] Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., Meger, D.: Deep reinforcement learning that matters. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32 (2018)